1 Annexes

Pour plus d'information voici le lien Github du projet.

1.1 Partie de Canin Antoine

RFIDReader.cpp :

```cpp
#include "RFIDReader.h"

RFIDReader::RFIDReader(const std::string& port) : portUSB(port) {}

bool RFIDReader::openPort(int baudRate) {
    char erreur = serial.openDevice(portUSB.c_str(), baudRate);
    return erreur == 1;
}

std::string RFIDReader::readBadge() {
    std::string badgeData = "";
    char caracRecu;

    while (true) {
        char erreur = serial.readChar(&caracRecu, 2000); // Timeout de 2s

        if (erreur > 0) {
            if (caracRecu == 0x02) { // Début de transmission (STX)
                badgeData = "";
            }
            else if (caracRecu == 0x03) { // Fin de transmission (ETX)
                return badgeData; // Retourne les données complètes
            }
            else {
                badgeData += caracRecu;
            }
        }
        else if (erreur < 0) {
            return ""; // Erreur de lecture -> Retourne une chaîne vide
        }
    }
}

void RFIDReader::closePort() {
    serial.closeDevice();
}
```

RFIDReader.h :

```
#ifndef RFIDREADER_H
#define RFIDREADER_H

#include <iostream>
#include <string>
#include "serialib.h"

class RFIDReader {
private:
    serialib serial;
    std::string portUSB;

public:
    RFIDReader(const std::string& port);
    bool openPort(int baudRate = 9600);
    std::string readBadge();
    void closePort();
};

#endif // RFIDREADER_H
```

main.cppl :

```
#include <iostream>
#include <string>

// Inclusions des headers nécessaires
#include "RFIDReader.h"
#include "requete.h"
#include "serialib.h"

using namespace std;

void programme1() {
    cout << "=== Programme 1 : Lecture RFID avec base de donnees ===" << endl;

    RFIDReader reader("/dev/ttyUSB0");
    Requete dbManager("tcp://10.187.52.4", "casier", "casier", "casier_b"); // Connexion à la base en ligne

    if (!dbManager.connexion()) {
        cerr << "Echec de connexion a la base de donnees." << endl;
        return;
    }

    if (!reader.openPort()) {
        cerr << "Echec d'ouverture du port RFID." << endl;
        return;
    }
```

```

    while (true) {
        std::string badge = reader.readBadge();
        if (!badge.empty()) {
            cout << "Badge detecte : " << badge << endl;
            if (dbManager.badgeExiste(badge)) {
                cout << "Badge deja enregistre." << endl;
            }
        }
    }

    reader.closePort();
    dbManager.deconnexion();
}

void programme2() {
    cout << "=== Programme 2 : Lecture RFID avec serialib ==="
<< endl;

    serialib serial;
    const string portUSB("/dev/ttyUSB0");

    if (serial.openDevice(portUSB.c_str(), 9600) != 1) {
        cerr << "Ouverture du port serie KO !" << endl;
        return;
    }

    cout << "Ouverture du port serie OK. Lecture en cours..."
<< endl;

    string badgeData = "";

    while (true) {
        char caracRecu;
        int erreur = serial.readChar(&caracRecu, 2000);

        if (erreur > 0) {
            if (caracRecu == 0x02) {
                badgeData = "";
            }
            else if (caracRecu == 0x03) {
                cout << "Donnee recue : " << badgeData << endl;
                badgeData = "";
            }
            else {
                badgeData += caracRecu;
            }
        }
        else if (erreur < 0) {
            cerr << "Erreur de lecture." << endl;
            break;
```

```
        }
    }

    serial.closeDevice();
}

int main() {
    cout << "Choisis un programme a tester :" << endl;
    cout << "1 - RFID avec base de donnees" << endl;
    cout << "2 - RFID simple avec serialib" << endl;
    cout << "Ton choix : ";

    int choix;
    cin >> choix;

    if (choix == 1) {
        programme1();
    }
    else if (choix == 2) {
        programme2();
    }
    else {
        cerr << "Choix invalide." << endl;
    }

    return 0;
}
```

requete.cpp :

```
#include "requete.h"

// Constructeur de la classe Requete : initialise la connexion à la base de données MySQL
Requete::Requete(const std::string& host, const std::string& user, const std::string& password, const std::string& database)
{
    try
    {
        // Obtention d'une instance du driver MySQL
        driver = get_driver_instance();

        // Création d'une connexion avec les informations fournies (hôte, utilisateur, mot de passe)
        con = std::unique_ptr<sql::Connection>(driver->connect(host, user, password));

        // Sélection de la base de données à utiliser
        con->setSchema(database);

        std::cout << "Connexion a la base de donnees reussie !" << std::endl;
```

```
    }
    catch (sql::SQLException& e)
    {
        // Gestion des erreurs en cas d'échec de connexion
        std::cerr << "Erreur de connexion MySQL: " << e.what() << " (Code: " << e.getErrorCode() << ")" << std::endl;
    }
}

// Destructeur de la classe Requete : ferme la connexion à la base de données
Requete::~Requete()
{
    deconnexion();
}

// Vérifie si la connexion à la base de données est bien établie
bool Requete::connexion()
{
    return con != nullptr; // Retourne vrai si la connexion existe, faux sinon
}

// Ferme proprement la connexion à la base de données
void Requete::deconnexion()
{
    if (con)
    {
        con.reset(); // Réinitialisation du pointeur unique, libérant ainsi la connexion
        std::cout << "Connexion MySQL fermee." << std::endl;
    }
}

// Vérifie si un badge existe dans la base de données
bool Requete::badgeExiste(const std::string& badgeID)
{
    try
    {
        if (!con)
        {
            std::cerr << "Erreur : connexion MySQL non établie." << std::endl;
            return false;
        }

        std::unique_ptr<sql::PreparedStatement> pstmt(con->prepareStatement("SELECT COUNT(*) FROM Tag WHERE Tag = ?"));
        pstmt->setString(1, badgeID);

```

```
        std::unique_ptr<sql::ResultSet> res(pstmt->executeQuery());
        res->next();

        return res->getInt(1) > 0; // Retourne vrai si le badge existe déjà
    }
    catch (sql::SQLException& e)
    {
        std::cerr << "Erreur lors de la vérification du badge : " << e.what() << " (Code: " << e.getErrorCode() << ")" << std::endl;
        return false;
    }
}
```

requete.h :

```
#pragma once

#include <iostream>
#include <string>
#include <memory>

// Inclusion du connecteur MySQL C++
#include <cppconn/driver.h>
#include <cppconn/exception.h>
#include <cppconn/resultset.h>
#include <cppconn/statement.h>
#include <cppconn/prepared_statement.h>

class Requete {
private:
    std::unique_ptr<sql::Connection> con; // Connexion MySQL
    std::unique_ptr<sql::Statement> stmt;
    std::unique_ptr<sql::PreparedStatement> pstmt;
    sql::Driver* driver;

public:
    Requete(const std::string& host, const std::string& user, const std::string& password, const std::string& database);
    ~Requete();
    bool connexion();
    void deconnexion();
    bool badgeExiste(const std::string& badgeID);
};
```

1.2 Partie de Pinalie William

1.2.1 Code

RGBLED.cpp :

```cpp
#include "RGBLED.h"

DelRGBChainable::DelRGBChainable(unsigned char clk_pin,
    unsigned char data_pin, unsigned char number_of_leds) :
    _clk_pin(clk_pin), _data_pin(data_pin), _num_leds(number_of_leds)
{
    gpioInitialise();
    gpioSetMode(_clk_pin, PI_OUTPUT);
    gpioSetMode(_data_pin, PI_OUTPUT);

    _led_state = new unsigned char[_num_leds * 3];

    for (unsigned char i = 0; i < _num_leds; i++)
        setColorRGB(i, 0, 0, 0);
}

DelRGBChainable::~DelRGBChainable() {
    delete[] _led_state;
    gpioTerminate();
}

int DelRGBChainable::initGpio() {
    if (gpioInitialise() < 0) {
        std::cerr << "Erreur d’initialisation de pigpio" << std::endl;
        return -1;
    }
}

void DelRGBChainable::clk(void) {
    gpioWrite(_clk_pin, 0);
    gpioDelay(_CLK_PULSE_DELAY);
    gpioWrite(_clk_pin, 1);
    gpioDelay(_CLK_PULSE_DELAY);
}

void DelRGBChainable::sendByte(unsigned char b) {
    // Send one bit at a time, starting with the MSB
    for (unsigned char i = 0; i < 8; i++) {
        // If MSB is 1, write one and clock it, else write 0 and clock
        if ((b & 0x80) != 0) {
            gpioWrite(_data_pin, 1);
        }
        else {
            gpioWrite(_data_pin, 0);
```

```
        }
        clk();

        // Advance to the next bit to send
        b <<= 1;
    }
}

void DelRGBChainable::sendColor(unsigned char red, unsigned char green, unsigned char blue) {
    // Start by sending a byte with the format "1 1 /B7 /B6 /G7 /G6 /R7 /R6"
    unsigned char prefix = 0xC0; // B11000000;
    if ((blue & 0x80) == 0) {
        prefix |= 0x20; // B00100000;
    }
    if ((blue & 0x40) == 0) {
        prefix |= 0x10; // B00010000;
    }
    if ((green & 0x80) == 0) {
        prefix |= 0x08; // B00001000;
    }
    if ((green & 0x40) == 0) {
        prefix |= 0x04; // B00000100;
    }
    if ((red & 0x80) == 0) {
        prefix |= 0x02; // B00000010;
    }
    if ((red & 0x40) == 0) {
        prefix |= 0x01; // B00000001;
    }
    sendByte(prefix);

    // Now must send the 3 colors
    sendByte(blue);
    sendByte(green);
    sendByte(red);
}

void DelRGBChainable::setColorRGB(unsigned char led, unsigned char red, unsigned char green, unsigned char blue) {
    // Send data frame prefix (32x "0")
    sendByte(0x00);
    sendByte(0x00);
    sendByte(0x00);
    sendByte(0x00);

    // Send color data for each one of the leds
    for (unsigned char i = 0; i < _num_leds; i++) {
        if (i == led) {
            _led_state[i * 3] = red;
```

```
            _led_state[i * 3 + 1] = green;
            _led_state[i * 3 + 2] = blue;
        }

        sendColor(_led_state[i * 3],
            _led_state[i * 3 + 1],
            _led_state[i * 3 + 2]);
    }
}

void DelRGBChainable::FlashesRedLed(unsigned char num_led) {
    for (int i = 0; i < 10; i++) {
        // Allumer la LED en rouge
        setColorRGB(num_led, 255, 0, 0);
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        // éteindre la LED
        setColorRGB(num_led, 0, 0, 0);
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
}

void DelRGBChainable::FlashesGreenLed(unsigned char num_led) {
    for (int i = 0; i < 10; i++) {
        // Allumer la LED en vert
        setColorRGB(num_led, 0, 255, 0);
        std::this_thread::sleep_for(std::chrono::milliseconds(100));

        // éteindre la LED
        setColorRGB(num_led, 0, 0, 0);
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
}


void DelRGBChainable::GreenLed(unsigned char num_led) {

    setColorRGB(num_led, 0, 255, 0);
}

void DelRGBChainable::RedLed(unsigned char num_led) {

    setColorRGB(num_led, 255, 0, 0);
}
```

RGBLED.h :

```
#include <iostream>
```

```
#include <pigpio.h>
#include <atomic>
#include <chrono>
#include <thread>
#include <chrono>
using namespace std;

#ifndef _DelRGBChainable_
#define _DelRGBChainable_


class DelRGBChainable
{

public:
    DelRGBChainable(unsigned char clk_pin, unsigned char  data_pin, unsigned char  number_of_leds);
    ~DelRGBChainable();

    int initGpio();
    void setColorRGB(unsigned char led, unsigned char  red, unsigned char  green, unsigned char  blue);
    void FlashesRedLed(unsigned char num_led);
    void FlashesGreenLed(unsigned char num_led);
    void GreenLed(unsigned char num_led);
    void RedLed(unsigned char num_led);

private:
    const unsigned int _CLK_PULSE_DELAY = 1;
    unsigned char _clk_pin;
    unsigned char _data_pin;
    unsigned char _num_leds;

    unsigned char* _led_state;

    void clk(void);
    void sendByte(unsigned char b);
    void sendColor(unsigned char red, unsigned char green, unsigned char blue);
};
#endif
```

main_test.cpp :

```
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <cstdint>  // type int8_t, int16_t, uint8_t, etc...
#include <thread>   // Pour std::this_thread::sleep_for
#include <mutex>
#include <fstream>
```

```
#include <sstream>
#include <vector>

using namespace std;

#include "serialib.h"
#include "servo_class.h"
#include "RGBLED.h"
#include "requete.h"
#include "RFIDReader.h"

int main(int argc, char** argv) {

  DelRGBChainable d(16, 17, 5);
  uint16_t status = 0;
  ServoH servoh;
  serialib VS;
  ServoH::ServoValuePos servopos;
  ServoH::ServoValueVolt servovolt;


    servopos = servoh.readInfoServoPos();

    servovolt = servoh.readInfoServoVolt();

      // Initialisation des objets
      RFIDReader reader("/dev/ttyUSB0");
      Requete dbManager("tcp://10.187.52.4", "casier", "casier"
, "casier_b");

      // Connexion à la base de données
      if (!dbManager.connexion()) {
          cerr << "Erreur de connexion à la base de données."
<< endl;
          return -1;
      }

      d.initGpio();  // Initialisation des GPIO


      cout << "Avant ouverture" << endl;


      if (!servoh.openConnection("/dev/ttyS0", 115200, VS)) {
          cerr << "Erreur d'ouverture de la connexion série."
<< endl;
          return -1;
      }

      cout << "Après ouverture" << endl;
      cout << "avant scan" << endl;
```

```
      /*servoh.scanServos(&VS);*/  // Scanner les servos
    std::vector<uint8_t> servosOnBus;
    if (ServoH::scanBus(&VS, servosOnBus, 20)) {
      for (uint8_t i = 0; i < servosOnBus.size(); i++) {
        std::cout << "Id servo : " << static_cast<int>(
servosOnBus.at(i)) << std::endl;
      }
    }
      cout << "Après scan" << endl;

      //Création des servos
    ServoH s0(&VS, 0);
    ServoH s1(&VS, 1);
      ServoH s2(&VS, 2);
    ServoH s3(&VS, 3);
    ServoH s4(&VS, 4);
    ServoH s5(&VS, 5);

    d.GreenLed(0);
    d.GreenLed(1);
    d.GreenLed(2);
    d.GreenLed(3);
    d.GreenLed(4);

      while (true) {

      if (!reader.openPort()) {
        cerr << "Erreur d'ouverture du port RFID." << endl;
        return -1;
      }

          string badge = reader.readBadge();

          if (!badge.empty()) {
              cout << "Badge lu : " << badge << endl;

        reader.closePort();

              if (dbManager.doesTagExist(badge)) {
                  // Badge existant
                  cout << "Badge déjà enregistré." << endl;

                  int casierId = dbManager.getCasierIdFromTag(
badge);

          servoh.openRfidServo(servopos.servo0, s0, 3000);
          std::this_thread::sleep_for(std::chrono::milliseconds
(1000));  // Pause de 1000 ms
          servoh.closeRfidServo(s0, 1000);


```

```
                    if (casierId != -1) {
                        cout << "Casier ID associé : " << casierId << endl;

                        // Selon l'ID du casier, actionner le servomoteur correspondant
                        if (casierId == 1) {
                            // Casier 1 : ouvrir le servomoteur 1
                servoh.openServo(s1, 1000);
                d.FlashesGreenLed(0);
                            std::this_thread::sleep_for(std::chrono::milliseconds(1000));
                servoh.closeServo(servopos.servo1, s1);     // Fermer le servo
                d.RedLed(0);
                        }
                        else if (casierId == 2) {
                            // Casier 2 : ouvrir le servomoteur 2
                servoh.openServo(s2, 0);
                d.FlashesGreenLed(1);
                            std::this_thread::sleep_for(std::chrono::milliseconds(1000));  // Pause de 1000 ms
                servoh.closeServo(servopos.servo2, s2);     // Fermer le servo
                d.RedLed(1);
                        }
              else if (casierId == 3) {
                // Casier 3 : ouvrir le servomoteur 3
                servoh.openServo(s3, 1000);
                d.FlashesGreenLed(2);
                std::this_thread::sleep_for(std::chrono::milliseconds(1000));  // Pause de 1000 ms
                servoh.closeServo(servopos.servo3, s3);     // Fermer le servo
                d.RedLed(2);
              }
              else if (casierId == 4) {
                // Casier 4 : ouvrir le servomoteur 4
                servoh.openServo(s4, 1000);
                d.FlashesGreenLed(3);
                std::this_thread::sleep_for(std::chrono::milliseconds(1000));  // Pause de 1000 ms
                servoh.closeServo(servopos.servo4, s4);     // Fermer le servo
                d.RedLed(3);
                std::this_thread::sleep_for(std::chrono::seconds(5));
              }
                    }

                    }
```

```
                    else {
                        // Badge non enregistré
                        cout << "Badge non enregistré !" << endl;

              servoh.openRfidServo(servopos.servo0, s0, 3000);
              std::this_thread::sleep_for(std::chrono::
milliseconds(1000));  // Pause de 1000 ms
              servoh.closeRfidServo(s0, 1000);

                    }
                }
            }

// Fermer la connexion série et la connexion GPIO
    servoh.closeConnection(VS);
    gpioTerminate();
    reader.closePort();
    dbManager.deconnexion();
    return 0;
}
```

servo_class.cpp :

```
#include "servo_class.h"

ServoH::ServoH(serialib* VS, uint8_t id) {
    this->VS = VS;
    this->servoCasiers = 0;
    this->servoRFID = 0;
    this->baudSpeed = HERKULEX_BAUD_RATE::BD115200;
    this->controlMode = HERKULEX_CONTROL_MODE::POSITION;
    if (id > MAX_ID)
        this->id = MAX_ID;
    else
        this->id = id;
    this->playtime = 60;
    this->position = HERKULEX_MID_POSITION;
    this->velocity = 0;
    this->torque = HERKULEX_TORQUE_VALUE::TORQUE_FREE;
    this->errorMessage = "";
}

ServoH::ServoH()
{
    ServoValuePos servopos;
    ServoValueVolt servovolt;

    this->servoCasiers = 0;
    this->servoRFID = 0;
}

ServoH::ServoH(serialib* VS)
```

```
{
    this->VS = VS;
}

ServoH::~ServoH()
{

}

void ServoH::scanServos(serialib* VS)
{
    std::vector<uint8_t> servosOnBus;
    if (ServoH::scanBus(VS, servosOnBus, 20)) {
        for (uint8_t i = 0; i < servosOnBus.size(); i++) {
            std::cout << "Id servo : " << static_cast<int>(
servosOnBus.at(i)) << std::endl;
        }
    }
}

bool ServoH::readStatusServo(uint16_t status, ServoH& s)
{
    if (!s.readStatus(status)) {
        cout << "Erreur" << s.getLastError() << endl;
        return -1;
    }
    else {
        cout << "Status : " << status << endl;
    }

    if (s.readStatus(status)) {
        cout << "status F : " << hex << (status >> 8) << endl;
        cout << "status f : " << hex << static_cast<int>(status
 & 0x00FF) << endl;
    }
}

bool ServoH::openConnection(const std::string& port, int
baudrate, serialib& VS)
{
    int codeRetour = VS.openDevice(port.c_str(), baudrate);
    if (codeRetour != 1) {
        std::cout << "Erreur à l'ouverture du port série." <<
std::endl;
        return false;
    }
    return true;
}

void ServoH::closeConnection(serialib& VS)
{
```

```
    VS.closeDevice();
}

void ServoH::openServo(ServoH& s, int temp_milli)
{
    float servoCasiers = 0;

    servoCasiers = getservoCasiers();
    s.init();
    s.fixTorqueOn();
    s.fixDegree(0);
    s.stopServo(s, servoCasiers);
    std::this_thread::sleep_for(std::chrono::milliseconds(temp_milli));
}

void ServoH::closeServo(int degree, ServoH& s)
{
    float servoCasiers = 0;
    servoCasiers = getservoCasiers();
    s.fixTorqueOn();
    s.fixDegree(degree);
    s.stopServo(s, servoCasiers);
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));
}

void ServoH::openServoTestVolt(ServoH& s, float& lastVolt, float& minVolt, int degree, int temp_milli)
{
    s.init();
    s.fixTorqueOn();
    s.fixDegree(degree);

    for (int j = 0; j < 20; j++) {
        s.readVoltage(lastVolt);

        if (lastVolt < minVolt) {
            minVolt = lastVolt;
        }

        /*cout << "[open] volt lu : " << lastVolt << " | tension min : " << minVolt << endl;*/
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(temp_milli));
}

void ServoH::closeServoTestVolt(ServoH& s, float& lastVolt, float& minVolt, int degree, int temp_milli)
```

```
{
    s.fixTorqueOn();
    s.fixDegree(degree);

    minVolt = std::numeric_limits<float>::max(); // Ré
initialise la valeur minimale

    for (int j = 0; j < 20; j++) {
        s.readVoltage(lastVolt);

        if (lastVolt < minVolt) {
            minVolt = lastVolt;
        }

        /*cout << "[close] volt lu : " << lastVolt << " |
tension min : " << minVolt << endl;*/
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(temp_
milli));
}

void ServoH::changeID(uint8_t oldId, uint8_t newId, ServoH s)
{

    // Tente de changer l'ID
    if (s.fixNewId(oldId, newId)) {
        std::cout << "L'ID du servomoteur est modifié de " <<
static_cast<int>(oldId)
                << " a " << static_cast<int>(newId) << std::endl;
    }
    else {
        std::cout << "Erreur lors du changement d'ID : " << s.
getLastError() << std::endl;
    }
}

void ServoH::openRfidServo(int degree, ServoH& s, int temp_
milli)
{
    float servoRFID = 0;

    servoRFID = getservoRFID();

    s.init();
    s.fixTorqueOn();
    s.fixDegree(degree);
    s.stopServo(s, servoRFID);
    std::this_thread::sleep_for(std::chrono::milliseconds(temp_
milli));
}
```

```

void ServoH::closeRfidServo(ServoH& s, int temp_milli)
{
    float servoRFID = 0;

    servoRFID = getservoRFID();
    s.fixDegree(0);
    s.stopServo(s, servoRFID);
    std::this_thread::sleep_for(std::chrono::milliseconds(temp_milli));
}

int ServoH::convertPositionToDegree(int position, const int midPosition)
{
    return (position - midPosition) * 0.325f;
}

float ServoH::getservoCasiers()
{
    return this->servoCasiers;
}

float ServoH::getservoRFID()
{
    return this->servoRFID;
}

float ServoH::getservoTest()
{
    return this->servoTest;
}

ServoH::ServoValuePos ServoH::readInfoServoPos()

{
    const string filePath = "../../../fichiers/position_servo.txt";
    std::ifstream file(filePath);
    std::vector<int> valuespos;

    ServoValuePos resultpos = { 0, 0, 0, 0, 0 };

    if (!file.is_open()) {
        std::cerr << "Erreur d'ouverture du fichier : " << filePath << std::endl;
        return resultpos; // Retourne un vecteur vide en cas d'erreur
    }

    std::string line;
```

```
    while (std::getline(file, line)) {
        std::istringstream iss(line);
        std::string word1, servoName, colon;
        float value;
        int res_val;

        iss >> word1 >> servoName >> colon >> value;

        // Si la ligne suit le bon format
        if ((word1 == "closeID" || word1 == "openID") && colon
== ":" && !servoName.empty()) {
            int res_val = value;
            valuespos.push_back(res_val);
        }
        else std::cerr << "Ligne ignorée (format incorrect) : "
 << line << std::endl;
    }

    file.close();

    for (size_t i = 0; i < valuespos.size(); ++i) {
        switch (i) {
        case 0: resultpos.servo0 = valuespos[i]; break;
        case 1: resultpos.servo1 = valuespos[i]; break;
        case 2: resultpos.servo2 = valuespos[i]; break;
        case 3: resultpos.servo3 = valuespos[i]; break;
        case 4: resultpos.servo4 = valuespos[i]; break;
        case 5: resultpos.servo5 = valuespos[i]; break;
        }
    }

    cout << "Positions des servos :" << endl;
    for (size_t i = 0; i < valuespos.size(); ++i) {
        std::cout << "Servo " << (i) << " = " << valuespos[i]
<< std::endl;
    }
    return resultpos;
}

ServoH::ServoValueVolt ServoH::readInfoServoVolt()
{
    const string filePath = "../../../fichiers/position_servo.
txt";
    std::ifstream file(filePath);
    std::vector<int> valuesvolt;

    ServoValueVolt resultvolt = { 0, 0, 0, 0, 0 };

    if (!file.is_open()) {
        std::cerr << "Erreur d'ouverture du fichier : " <<
filePath << std::endl;
```

```
        return resultvolt; // Retourne un vecteur vide en cas d'erreur
    }

    std::string line;
    while (std::getline(file, line)) {
        std::istringstream iss(line);
        std::string word1, servoName, colon;
        float value;
        int res_val;

        iss >> word1 >> servoName >> colon >> value;

        // Si la ligne suit le bon format
        if (word1 == "voltID" && colon == ":" && !servoName.empty()) {
            int res_val = value;
            valuesvolt.push_back(res_val);
        }
        else std::cerr << "Ligne ignorée (format incorrect) : " << line << std::endl;
    }

    file.close();

    for (size_t i = 0; i < valuesvolt.size(); ++i) {
        switch (i) {
        case 0: resultvolt.servo0 = valuesvolt[i]; break;
        case 1: resultvolt.servo1 = valuesvolt[i]; break;
        case 2: resultvolt.servo2 = valuesvolt[i]; break;
        case 3: resultvolt.servo3 = valuesvolt[i]; break;
        case 4: resultvolt.servo4 = valuesvolt[i]; break;
        case 5: resultvolt.servo5 = valuesvolt[i]; break;
        }
    }

    cout << "Positions des servos :" << endl;
    for (size_t i = 0; i < valuesvolt.size(); ++i) {
        std::cout << "Servo " << (i) << " = " << valuesvolt[i] << std::endl;
    }
    return resultvolt;
}

void ServoH::changeTXT(float newval, string text, string id)
{
    std::string filename = "../../../fichiers/position_servo.txt";
    std::ifstream infile(filename);
    std::ofstream outfile("temp.txt");
    std::string line;
```

```
    float newServoCasiersVolt = 12; //Nouvelle valeur à définir

    if (!infile || !outfile) {
        std::cerr << "Erreur lors de l'ouverture du fichier."
<< std::endl;
    }

    while (std::getline(infile, line)) {
        std::istringstream iss(line);
        std::string word1, servoName, colon, value;

        iss >> word1 >> servoName >> colon >> value;

        std::cout << line << std::endl;
        try {
            if (text == "voltID" && servoName == id) {
                outfile << word1 << " " << servoName << " " <<
colon << " " << newval << std::endl;
            }
            else if (text == "closeID" && servoName == id) {
                outfile << word1 << " " << servoName << " " <<
colon << " " << newval << std::endl;
            }
            else if (text == "closeID" && servoName == id) {
                outfile << word1 << " " << servoName << " " <<
colon << " " << newval << std::endl;
            }
            else if (text == "changeID" && servoName == id) {
                int num = newval;
                outfile << word1 << " " << num << " " << colon
<< " " << value << std::endl;
            }
            else {
                outfile << line << std::endl;
            }
        }
        catch (const std::exception& e) {
            std::cerr << "L'ID n'existe pas dans le fichier !
Aucune modification. " << line << "\n"
                << "Exception : " << e.what() << std::endl;
            outfile << line << std::endl;
        }
    }

    infile.close();
    outfile.close();

    // Remplace le fichier original
    std::remove(filename.c_str());
    std::rename("temp.txt", filename.c_str());

```

```
    std::cout << "Modification terminée." << std::endl;
}


void ServoH::stopServo(ServoH s, float typeServo)
{
    float voltage = 0;
    uint16_t pos;
    int degree;

    for (int j = 0; j < 20; j++) {
        s.readVoltage(voltage);
        if (voltage < typeServo) {
            s.readPosition(pos);
            degree = convertPositionToDegree(pos);
            s.fixTorqueOff();
            cout << "voltage : " << voltage << endl;
            cout << "position : " << pos << endl;
            cout << "degree : " << degree << endl;
            break;  // Quitter la boucle actuelle
        }
        else {
            cout << "voltage : " << voltage << endl;
        }
    }
}

bool ServoH::sendSimpleIJOGData(HERKULEX_CONTROL_MODE mode,
uint16_t position, uint8_t playtime)
{
    frameToSend.clear();
    frameToSend.push_back(HERKULEX_FRAME_HEADER1_VALUE);
    frameToSend.push_back(HERKULEX_FRAME_HEADER2_VALUE);
    frameToSend.push_back(HERKULEX_IJOG_MINIMUM_LENGHT);
    frameToSend.push_back(this->id);
    frameToSend.push_back(static_cast<uint8_t>(HERKULEX_CMD::I_
JOG));
    frameToSend.push_back(0);
    frameToSend.push_back(0);
    frameToSend.push_back(static_cast<uint8_t>(position & 0
x00FF));
    frameToSend.push_back(static_cast<uint8_t>((position & 0
xFF00) >> 8));
    // ne fonctionne pas si je mets la ligne suivante donc je
force à 0 pour passer en mode position
    //uint8_t byteSet = ((getLeds()<<2) | (static_cast<uint8_t
>(HERKULEX_CONTROL_MODE::POSITION)<<1));
    if (mode == HERKULEX_CONTROL_MODE::POSITION)
        frameToSend.push_back(0);
    else
        frameToSend.push_back(2);
```

```
    frameToSend.push_back(this->id);
    frameToSend.push_back(playtime);
    frameToSend.at(static_cast<int>(HERKULEX_FRAME::CRC1)) = calcCRC1(frameToSend);
    frameToSend.at(static_cast<int>(HERKULEX_FRAME::CRC2)) = calcCRC2(frameToSend);

    if (!sendFrame()) {
        errorMessage += "\nError sending simple I_JOG command\n";
        return false;
    }
    return true;
}

bool ServoH::writeEEPROM(HERKULEX_ROM_REGISTERS startReg, vector<uint8_t> data)
{
    return writeRegs(HERKULEX_REGISTERS::EEPROM, static_cast<uint8_t>(startReg), data);
}

bool ServoH::writeRegs(HERKULEX_REGISTERS memType, uint8_t address, vector<uint8_t> data)
{
    frameToSend.push_back(HERKULEX_FRAME_HEADER1_VALUE);
    frameToSend.push_back(HERKULEX_FRAME_HEADER2_VALUE);
    frameToSend.push_back(HERKULEX_EEPROM_RAM_WRITE_LENGHT + data.size());
    frameToSend.push_back(this->id);
    if (memType == HERKULEX_REGISTERS::EEPROM)
        frameToSend.push_back(static_cast<uint8_t>(HERKULEX_CMD::EEP_WRITE));
    else
        frameToSend.push_back(static_cast<uint8_t>(HERKULEX_CMD::RAM_WRITE));
    frameToSend.push_back(0);
    frameToSend.push_back(0);
    frameToSend.push_back(address);
    frameToSend.push_back(data.size());
    for (uint8_t i = 0; i < data.size(); i++) frameToSend.push_back(data.at(i));
    frameToSend.at(static_cast<int>(HERKULEX_FRAME::CRC1)) = calcCRC1(frameToSend);
    frameToSend.at(static_cast<int>(HERKULEX_FRAME::CRC2)) = calcCRC2(frameToSend);

    //printFrameToSend();
    if (!sendFrame()) {
        errorMessage += "\nError while sending frame for writing EEPROM/RAM registers";
```

```
        return false;
    }
    return true;
}

uint8_t     ServoH::calcCRC1(vector<uint8_t> frame) {
    uint8_t     CRC1 = 0;

    for (unsigned int i = static_cast<int>(HERKULEX_FRAME::
LENTGH); i < frame.size(); i++) {
        switch (i) {
        case static_cast<int>(HERKULEX_FRAME::CRC1):
        case static_cast<int>(HERKULEX_FRAME::CRC2):
            break;
        default:
            CRC1 ^= frame.at(i);
        }
    }
    CRC1 = CRC1 & 0xFE;
    return CRC1;
}

uint8_t     ServoH::calcCRC2(vector<uint8_t> frame) {
    return (~calcCRC1(frame) & 0xFE);
}

bool ServoH::init()
{
    if (!readMaximumPosition(this->maxPosition)) return false;
    if (!readMinimumPosition(this->minPosition)) return false;
    if (!readControlMode(this->controlMode)) return false;
    if (!readPosition(this->position)) return false;

    return true;
}

bool ServoH::getId() const
{
    return this->id;
}

bool ServoH::fixNewId(uint8_t oldId, uint8_t newId)
{
    vector<uint8_t> data;
    bool state;

    if (oldId != this->id) {
        this->errorMessage += "\nWrong actual id";
        return false;
    }

```

```
    if (newId > MAX_ID) {
        this->errorMessage += "\nNew servo id too large (>253)";
        return false;
    }

    this->frameToSend.clear();
    data.push_back(newId);
    state = writeEEPROM(HERKULEX_ROM_REGISTERS::ROM_ID, data);
    if (state) this->id = newId;

    return state;

}

bool ServoH::fixPosition(uint16_t position, uint8_t playtime)
{
    HERKULEX_CONTROL_MODE mode;

    if (!readControlMode(mode))
        return false;

    if (mode != HERKULEX_CONTROL_MODE::POSITION) {
        errorMessage += "\nCan't set position when servo is in velocity mode";
        return false;
    }

    if (position < this->minPosition || position > this->maxPosition) {
        this->errorMessage += "\nPosition must be between " + to_string(minPosition) +
            " and " + to_string(maxPosition) + "\n";
        return false;
    }

    if (!sendSimpleIJOGData(mode, position, playtime)) {
        errorMessage += "Can\'t change servo position";
        return false;
    }

    this->position = position;
    this->playtime = playtime;

    return true;
}

bool ServoH::recvFrame(const int timeOut) {
    char car;
    int result;

```

```
    frameRecv.clear();

    do {
        // lecture d'un caractère sur la voie série avec timeout
        result = VS->readChar(&car, timeOut);
        // on rajoute le caractère recu dans le vecteur si on est pas en timeout
        if (result != 0) frameRecv.push_back(car);
    } while (result != 0);

    if (result == 0 && frameRecv.empty()) {
        HERKULEX_CMD cmdSend;
        cmdSend = static_cast<HERKULEX_CMD>(frameToSend.at(static_cast<int>(HERKULEX_FRAME::CMD)));
        if (cmdSend == HERKULEX_CMD::EEP_WRITE ||
            cmdSend == HERKULEX_CMD::RAM_WRITE)
            return true;
        else
            errorMessage += "\nNo response from servo";
        return false;
    }

    if (frameRecv.at(static_cast<int>(HERKULEX_FRAME::HEADER1)) != HERKULEX_FRAME_HEADER1_VALUE ||
        frameRecv.at(static_cast<int>(HERKULEX_FRAME::HEADER2)) != HERKULEX_FRAME_HEADER2_VALUE) {
        errorMessage += "\nReceived frame header error";
        return false;
    }

    if (frameRecv.at(static_cast<int>(HERKULEX_FRAME::LENTGH)) != frameRecv.size()) {
        errorMessage += "\nReceived frame size error";
        return false;
    }

    if (calcCRC1(frameRecv) != frameRecv.at(static_cast<int>(HERKULEX_FRAME::CRC1))) {
        errorMessage += "\nReceived frame CRC1 error";
        return false;
    }

    if (calcCRC2(frameRecv) != frameRecv.at(static_cast<int>(HERKULEX_FRAME::CRC2))) {
        errorMessage += "\nReceived frame CRC2 error";
        return false;
    }

    return true;
}
```

```

vector<uint8_t> ServoH::readRAM(HERKULEX_RAM_REGISTERS startReg, uint8_t nbBytes)
{
    return readRegs(HERKULEX_REGISTERS::RAM, static_cast<uint8_t>(startReg), nbBytes);
}

vector<uint8_t> ServoH::readRegs(HERKULEX_REGISTERS memType, uint8_t address, uint8_t nbBytes)
{
    vector<uint8_t> reg;

    frameToSend.push_back(HERKULEX_FRAME_HEADER1_VALUE);
    frameToSend.push_back(HERKULEX_FRAME_HEADER2_VALUE);
    frameToSend.push_back(HERKULEX_EEPROM_RAM_READ_LENGHT);
    frameToSend.push_back(this->id);
    if (memType == HERKULEX_REGISTERS::EEPROM)
        frameToSend.push_back(static_cast<uint8_t>(HERKULEX_CMD::EEP_READ));
    else
        frameToSend.push_back(static_cast<uint8_t>(HERKULEX_CMD::RAM_READ));
    frameToSend.push_back(0);
    frameToSend.push_back(0);
    frameToSend.push_back(address);
    frameToSend.push_back(nbBytes);
    frameToSend.at(static_cast<int>(HERKULEX_FRAME::CRC1)) = calcCRC1(frameToSend);
    frameToSend.at(static_cast<int>(HERKULEX_FRAME::CRC2)) = calcCRC2(frameToSend);

    if (!sendFrame()) {
        errorMessage += "\nError while sending frame for reading EEPROM/RAM registers";
        return reg;
    }

    if (!recvFrame()) {
        errorMessage += "\nError while receiving frame after reading EEPROM/RAM registers";
        return reg;
    };

    for (uint8_t i = 9; i < 9 + nbBytes; i++) reg.push_back(frameRecv.at(i));
    return reg;
}

vector<uint8_t> ServoH::readEEPROM(HERKULEX_ROM_REGISTERS startReg, uint8_t nbBytes)
```

```
{
    return readRegs(HERKULEX_REGISTERS::EEPROM, static_cast<
uint8_t>(startReg), nbBytes);
}

bool ServoH::fixTorque(HERKULEX_TORQUE_VALUE torque) {
    vector<uint8_t> data;
    bool state;

    this->frameToSend.clear();
    data.push_back(static_cast<uint8_t>(torque));
    state = writeRAM(HERKULEX_RAM_REGISTERS::RAM_TORQUE_CONTROL
, data);
    if (!state)
        this->errorMessage += "Can\'t change torque\n";
    else
        this->torque = torque;

    return state;
}

bool ServoH::fixTorqueOn()
{
    return fixTorque(HERKULEX_TORQUE_VALUE::TORQUE_ON);
}

bool ServoH::fixTorqueOff() {
    return fixTorque(HERKULEX_TORQUE_VALUE::TORQUE_FREE);
}

bool ServoH::readControlMode(HERKULEX_CONTROL_MODE& mode)
{
    vector<uint8_t> response;

    frameToSend.clear();
    response = readRAM(HERKULEX_RAM_REGISTERS::RAM_CURRENT_
CONTROL_MODE, ONE_BYTE_REGISTER);

    if (response.size() == ONE_BYTE_REGISTER) {
        mode = static_cast<HERKULEX_CONTROL_MODE>(response.at
(0));
        return true;
    }
    else {
        errorMessage += "\nError reading current control mode";
        return false;
    }
}

bool ServoH::readStatus(uint16_t& statusValue)
{
```

```
    frameToSend.clear();
    frameToSend.push_back(HERKULEX_FRAME_HEADER1_VALUE);
    frameToSend.push_back(HERKULEX_FRAME_HEADER2_VALUE);
    frameToSend.push_back(HERKULEX_STATUS_FRAME_LENGHT);
    frameToSend.push_back(this->id);
    frameToSend.push_back(static_cast<uint8_t>(HERKULEX_CMD::STAT));
    frameToSend.push_back(calcCRC1(frameToSend));
    frameToSend.push_back(calcCRC2(frameToSend));

    if (!sendFrame()) {
        errorMessage += "\nError getting STATUS\n";
        return false;
    }

    if (!recvFrame()) return false;

    statusValue = frameRecv.at(7) << 8 | frameRecv.at(8);
    return true;
}

bool ServoH::readPosition(uint16_t& position)
{
    vector<uint8_t> response;

    frameToSend.clear();
    response = readRAM(HERKULEX_RAM_REGISTERS::RAM_CALIBRATED_POSITION, TWO_BYTE_REGISTER);

    if (response.size() == TWO_BYTE_REGISTER) {
        uint16_t result = response.at(1) << 8 | response.at(0);
        // Masquage du resultat sur 10 bits (gamme 0-1023)
        position = result & 0x03FF;
        return true;
    }
    else {
        errorMessage += "\nError reading calibrated position";
        return false;
    }
}

bool ServoH::readVoltage(float& voltage) {
    vector<uint8_t> response;

    frameToSend.clear();
    response = readRAM(HERKULEX_RAM_REGISTERS::RAM_VOLTAGE, ONE_BYTE_REGISTER);

    if (response.size() == ONE_BYTE_REGISTER) {
        // le nombre 0.074 vient de la documentation page 29
        voltage = (static_cast<float>(response.at(0)) * 0.074);
```

```
        return true;
    }
    else {
        errorMessage += "\nError reading servo voltage";
        return false;
    }
}

bool ServoH::readMinimumPosition(uint16_t& minPos)
{
    vector<uint8_t> response;

    frameToSend.clear();
    response = readEEPROM(HERKULEX_ROM_REGISTERS::ROM_MIN_POSITION, TWO_BYTE_REGISTER);

    if (response.size() == TWO_BYTE_REGISTER) {
        minPos = response.at(1) << 8 | response.at(0);
        return true;
    }
    else {
        errorMessage += "\nError reading minimum servo position";
        return false;
    }
}

bool ServoH::readMaximumPosition(uint16_t& maxPos)
{
    vector<uint8_t> response;

    frameToSend.clear();
    response = readEEPROM(HERKULEX_ROM_REGISTERS::ROM_MAX_POSITION, TWO_BYTE_REGISTER);

    if (response.size() == TWO_BYTE_REGISTER) {
        maxPos = response.at(1) << 8 | response.at(0);
        return true;
    }
    else {
        errorMessage += "\nError reading maximum servo position";
        return false;
    }

}

bool ServoH::scanBus(serialib* VS, vector<uint8_t>& idServo, uint8_t idMax)
{
    vector<uint8_t> frame;
```

```
    vector<uint8_t> tempFrame;
    char car;

    frame.push_back(HERKULEX_FRAME_HEADER1_VALUE);
    frame.push_back(HERKULEX_FRAME_HEADER2_VALUE);
    frame.push_back(HERKULEX_STATUS_FRAME_LENGHT);
    frame.push_back(0); // id inconnu
    frame.push_back(static_cast<uint8_t>(HERKULEX_CMD::STAT));
    frame.push_back(0); // CRC1 non calcule
    frame.push_back(0); // CRC2 non calcule

    for (uint8_t i = 0; i <= idMax; i++) {
        frame.at(static_cast<int>(HERKULEX_FRAME::PID)) = i;
        frame.at(static_cast<int>(HERKULEX_FRAME::CRC1)) = calcCRC1(frame);
        frame.at(static_cast<int>(HERKULEX_FRAME::CRC2)) = calcCRC2(frame);
        int result = VS->writeBytes(frame.data(), frame.size());
        if (result < 0)
            return false;

        tempFrame.clear();
        do {
            // lecture d'un caractère sur la voie série avec timeout
            result = VS->readChar(&car, 100);
            // on rajoute le caractère recu dans le vecteur si on est pas en timeout
            if (result != 0) tempFrame.push_back(car);
        } while (result != 0);

        if (result == 0 && !tempFrame.empty()) idServo.push_back(i);
    }
    return true;
}

bool ServoH::fixDegree(float degree, uint8_t playtime)
{
    float minDegree = (this->minPosition - HERKULEX_MID_POSITION) * 0.325;
    float maxDegree = (this->maxPosition - HERKULEX_MID_POSITION) * 0.325;

    if (degree < minDegree || degree > maxDegree) {
        errorMessage += "\nDegree must be between " + to_string(minDegree) +
            " and " + to_string(maxDegree) + "\n";
        return false;
    }
```

```
    int position = static_cast<uint16_t>(HERKULEX_MID_POSITION
+ (degree / 0.325));
    return this->fixPosition(position, playtime);
}

bool ServoH::sendFrame()
{
    int result = VS->writeBytes(frameToSend.data(), frameToSend
.size());
    if (result < 0) {
        errorMessage += "\nError writing on serial port";
        return false;
    }
    else
        return true;
}

string ServoH::getLastError()
{
    string tempErrorMessage = errorMessage;
    errorMessage = "No error";
    return tempErrorMessage;
}

bool ServoH::writeRAM(HERKULEX_RAM_REGISTERS startReg, vector<
uint8_t> data) {
    return writeRegs(HERKULEX_REGISTERS::RAM, static_cast<uint8
_t>(startReg), data);
}
```

servo_class.h :

```
#include <iostream>
#include <vector>

#include <string>
#include <cstdlib>
#include <cstdint>      // type int8_t, int16_t, uint8_t, etc
...
#include "serialib.h"
#include <fstream>
#include <sstream>
#include <thread>
#include <chrono>

using namespace std;

enum class HERKULEX_ROM_REGISTERS : std::uint8_t {
    ROM_MIN_POSITION = 26,
    ROM_MAX_POSITION = 28,
    ROM_ID = 6
};
```

```

enum class HERKULEX_BAUD_RATE : std::uint8_t {
    BD666666 = 0x02,
    BD500000 = 0x03,
    BD400000 = 0x04,
    BD250000 = 0x07,
    BD200000 = 0x09,
    BD115200 = 0x10,
    BD57600 = 0x22
};

enum class HERKULEX_TORQUE_VALUE : std::uint8_t {
    TORQUE_FREE = 0x00,
    BREAK_ON = 0x40,
    TORQUE_ON = 0x60
};

enum class HERKULEX_CONTROL_MODE : std::uint8_t {
    POSITION = 0x00,
    VELOCITY
};

enum class HERKULEX_RAM_REGISTERS : std::uint8_t {
    RAM_TORQUE_CONTROL = 52,
    RAM_CURRENT_CONTROL_MODE = 56,
    RAM_CALIBRATED_POSITION = 58,
    RAM_VOLTAGE = 54

};

enum class HERKULEX_REGISTERS : std::uint8_t {
    RAM = 0,
    EEPROM
};

enum class HERKULEX_CMD : std::uint8_t {
    EEP_WRITE = 1,
    EEP_READ,
    RAM_WRITE,
    RAM_READ,
    I_JOG,
    STAT = 7
};

enum class HERKULEX_FRAME : std::uint8_t {
    HEADER1 = 0,
    HEADER2,
    LENTGH,
    PID,
    CMD,
    CRC1,
```

```
    CRC2
};

const uint16_t  HERKULEX_MID_POSITION = 512;
const uint8_t   DEFAULT_ID = 0xFD;
const uint8_t   HERKULEX_FRAME_HEADER1_VALUE = 0xFF;
const uint8_t   HERKULEX_FRAME_HEADER2_VALUE = 0xFF;
const uint8_t   HERKULEX_STATUS_FRAME_LENGHT = 7;
const int       HERKULEX_TIMEOUT = 100;
const uint8_t   MAX_ID = 0xFD;
const uint8_t   HERKULEX_EEPROM_RAM_WRITE_LENGHT = 9;
const uint8_t   HERKULEX_IJOG_MINIMUM_LENGHT = 12;
const uint8_t   HERKULEX_EEPROM_RAM_READ_LENGHT = 9;
const uint8_t   ONE_BYTE_REGISTER = 1;
const uint8_t   TWO_BYTE_REGISTER = 2;

class ServoH {
private:

    float servoCasiers;
    float servoRFID;
    float servoTest;

    serialib* VS;  // Objet de communication série
    uint8_t             id;
    vector<uint8_t>     frameToSend, frameRecv;
    int16_t             velocity;
    string              errorMessage;
    uint16_t            maxPosition;
    uint16_t            minPosition;
    HERKULEX_CONTROL_MODE controlMode;
    HERKULEX_BAUD_RATE    baudSpeed;
    uint16_t            position;
    uint8_t             playtime;
    HERKULEX_TORQUE_VALUE torque;

    bool            sendSimpleIJOGData(HERKULEX_CONTROL_MODE
mode, uint16_t position, uint8_t playtime = 60);
    bool            writeEEPROM(HERKULEX_ROM_REGISTERS startReg
, vector<uint8_t> data);
    bool            writeRegs(HERKULEX_REGISTERS memType, uint8
_t address, vector<uint8_t> data);
    static  uint8_t calcCRC1(vector<uint8_t> frame);
    static  uint8_t calcCRC2(vector<uint8_t> frame);
    bool            recvFrame(const int timeOut = HERKULEX_
TIMEOUT);
    vector<uint8_t> readRAM(HERKULEX_RAM_REGISTERS startReg,
uint8_t nbBytes);
    vector<uint8_t> readRegs(HERKULEX_REGISTERS memType, uint8_
t address, uint8_t nbBytes);
    vector<uint8_t> readEEPROM(HERKULEX_ROM_REGISTERS startReg,
```

```
 uint8_t nbBytes);

public:

    struct ServoValuePos {

        int servo0;
        int servo1;
        int servo2;
        int servo3;
        int servo4;
        int servo5;
    };

    struct ServoValueVolt {

        int servo0;
        int servo1;
        int servo2;
        int servo3;
        int servo4;
        int servo5;
    };

    // Constructeur par défaut
    ServoH();
    ServoH(serialib* VS);/*: isConnected(false) {}*/
    ServoH(serialib* VS, uint8_t id);
    ~ServoH();

    //methodes utiles
    void scanServos(serialib* VS);
    bool readStatusServo(uint16_t status, ServoH& s);
    bool openConnection(const std::string& port, int baudrate,
serialib& VS);
    void closeConnection(serialib& VS);
    void openServo(ServoH& s, int temp_milli);
    void openServoTestVolt(ServoH& s, float& volt1, float&
volt2, int degree, int temp_milli);
    void closeServoTestVolt(ServoH& s, float& volt1, float&
volt2, int degree, int temp_milli);
    void changeID(uint8_t oldId, uint8_t newId, ServoH s);
    void closeServo(int degree, ServoH& s);
    void openRfidServo(int degree, ServoH& s, int temp_milli);
    void closeRfidServo(ServoH& s, int temp_milli);
    void stopServo(ServoH s, float typeServo);
    int convertPositionToDegree(int position, const int
midPosition = 512);
    float getservoCasiers();
    float getservoRFID();
    float getservoTest();
```

```
    ServoValuePos readInfoServoPos();
    ServoValueVolt readInfoServoVolt();
    void changeTXT(float newval, string text, string oldid);



    //autres methodes
    bool init();
    bool getId() const;
    bool fixNewId(uint8_t oldId, uint8_t newId);
    bool fixPosition(uint16_t position, uint8_t playtime = 60);
    bool fixTorque(HERKULEX_TORQUE_VALUE torque);
    bool fixTorqueOn();
    bool fixTorqueOff();
    bool readControlMode(HERKULEX_CONTROL_MODE& mode);
    bool readStatus(uint16_t& statusValue);
    bool readPosition(uint16_t& position);
    bool readVoltage(float& voltage);

    bool readMinimumPosition(uint16_t& minPos);
    bool readMaximumPosition(uint16_t& maxPos);

    static bool scanBus(serialib* VS, vector<uint8_t>& idServo,
 uint8_t idMax = MAX_ID);
    bool fixDegree(float degree, uint8_t playtime = 60);
    bool writeRAM(HERKULEX_RAM_REGISTERS startReg, vector<uint8
_t> data);
    bool sendFrame();

    string      getLastError();
};
```

1.2.2 Bonus

main.cpp :

```
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <cstdint>  // type int8_t, int16_t, uint8_t, etc...
#include <thread>   // Pour std::this_thread::sleep_for
#include <mutex>
#include <chrono>   // Pour std::chrono::milliseconds
#include <fstream>
#include <sstream>
#include <vector>

using namespace std;

#include "serialib.h"
```

```
#include "servo_class.h"


int main(int argc, char** argv) {
  //bite
  string pos, volt, modif;
  uint16_t status = 0;
  ServoH servoh;
  serialib VS;
  ServoH::ServoValuePos servopos;
  ServoH::ServoValueVolt servovolt;

  servopos = servoh.readInfoServoPos();
  servovolt = servoh.readInfoServoVolt();

  /*for (size_t i = 0; i < servoValues.size(); ++i) {
    std::cout << "Servo " << (i + 1) << " = " << servoValues[i] << std::endl;
  }*/

  cout << "Avant ouverture" << endl;


  if (!servoh.openConnection("/dev/ttyS0", 115200, VS)) {
    cerr << "Erreur d'ouverture de la connexion série." << endl;
    return -1;
  }

  cout << "Après ouverture" << endl;
  // Scanner les servos sur le bus
  cout << "<< Avant scan >>" << endl;
  /*servoh.scanServos(&VS);*/
  std::vector<uint8_t> servosOnBus;
  if (ServoH::scanBus(&VS, servosOnBus, 20)) {
    for (uint8_t i = 0; i < servosOnBus.size(); i++) {
      std::cout << "Id servo : " << static_cast<int>(servosOnBus.at(i)) << std::endl;
    }
  }
  else {
    cout << "Aucun servo trouvé sur le bus !" << endl;
  }
  cout << "<< Après scan >>" << endl;

  // Demander les ID à l'utilisateur
  bool exita = true;
  string option;
  string exits = "exit";
  while (exita == true) {
    servopos = servoh.readInfoServoPos();
```

```
    servovolt = servoh.readInfoServoVolt();
    cout << "choisissez une option : " << endl << "1. Modifier l'id du servomoteur"
      << endl << "2. modifier la position des servomoteurs" << endl << "3. Modifier le voltes d'un servomoteur" << endl
      << "4. Quitter" << endl;
    cout << "Entrer une option : ";
    cin >> option;

    if (option == "4") {
      exita = false;
      break;
    }
    else if (option == "1") {
      string text = "changID";
      bool exitb = true;
      int oldId, newId;
      while (exitb == true) {
        cout << "Entrez 'exit' pour quitter" << std::endl;
        cout << "Entrez l'ancien ID du servomoteur : ";
        cin >> option;
        if (option == exits) {
          exitb = false;
          break;
        }
        else {
          try {
            oldId = stoi(option);
          }
          catch (const std::invalid_argument& e) {
            std::cerr << "L'ancien ID doit etre un entier ou tapez 'exit' pour quitter !" << endl;
            continue;
          }

          bool idExists = false;
          for (uint8_t id : servosOnBus) {
            if (id == oldId) {
              idExists = true;
              break;
            }
          }

          if (!idExists) {
            std::cerr << "L'ancien ID n'existe pas ! Veuillez essayer un autre ID." << endl;
            continue;
          }
        }
        cout << "Entrez le nouveau ID du servomoteur : ";
        cin >> newId;
```

```

        ServoH s(&VS, oldId);
        string idservo = to_string(oldId);
        // Changer l'ID
        servoh.changeID(oldId, newId, s);
        servoh.changeTXT(newId, text, idservo);
        break;
      }

    }
    else if (option == "2") {
      string text = "closeID";
      for (uint8_t id : servosOnBus) {
        ServoH servo(&VS, id);
        uint16_t pos = 0;
        int serID = static_cast<int>(id);

        if (servo.readPosition(pos)) {
          float degree = servoh.convertPositionToDegree(pos);
          cout << "Servo ID " << serID
            << " -> Position : " << pos
            << " | Degrée : " << degree << "°" << endl;
          string idservo =  to_string(serID);
          servoh.changeTXT(degree, text, idservo);
        }
        else {
          cerr << "Erreur lecture servo ID " << static_cast<int>(id)
            << " : " << servo.getLastError() << endl;
        }

      }
    }
    else if (option == "3") {
      int selectId;
      cout << "Entrez 'exit' pour quitter" << std::endl;
      cout << "Entrez l'ID du servomoteur a tester: ";
      cin >> option;
      if (option == exits) {
        exita = false;
        break;
      }
      else {
        try {

          selectId = stoi(option);
        }
        catch (const std::invalid_argument& e) {
          std::cerr << "L'ID doit etre un entier ou tapez 'exit' pour quitter !" << endl;
          continue;
```

```
        }

        bool idExists = false;
        for (uint8_t id : servosOnBus) {
          if (id == selectId) {
            idExists = true;
            break;
          }
        }

        if (!idExists) {
          std::cerr << "L'ID n'existe pas ! Veuillez essayer un autre ID." << endl;
          continue;
        }

        ServoH s(&VS, selectId);

        /*float voltMin = std::numeric_limits<float>::max();*/
        float voltLast = 0.0;
        float voltMin = 10.0;
        int servo_degree = 0;
        string text;
        string chaine;

        if (selectId == 0) {
          servo_degree = servopos.servo0;
          text = "voltID";
          chaine = "0";
        }
        else if (selectId == 1) {
          servo_degree = servopos.servo1;
          text = "voltID";
          chaine = "1";
        }
        else if (selectId == 2) {
          servo_degree = servopos.servo2;
          text = "voltID";
          chaine = "2";
        }
        else if (selectId == 3) {
          servo_degree = servopos.servo3;
          text = "voltID";
          chaine = "3";
        }
        else if (selectId == 4) {
          servo_degree = servopos.servo4;
          text = "voltID";
          chaine = "4";
        }
        else if (selectId == 5) {
```

```
          servo_degree = servopos.servo5;
          text = "voltID";
          chaine = "5";
        }

        cout << chaine << endl;

        servoh.openServoTestVolt(s, voltLast, voltMin, 0, 200);
        cout << "Tension minimale (open) : " << voltMin << endl;

        servoh.closeServoTestVolt(s, voltLast, voltMin, servo_degree, 200);
        cout << "Tension minimale (close) : " << voltMin << endl;

        servoh.changeTXT(voltMin, text, chaine);
      }
    }
    else {
      cout << "Entrer l'option 1, 2, 3 ou 4 !" << endl;
    }

  }

  servoh.closeConnection(VS);
  return 0;
}
```

1.3 Partie de Picard Alexandre

1.3.1 Base de données

Bdd.sql :

```
DROP TABLE IF EXISTS Utilisateur;
DROP TABLE IF EXISTS Affectation;
DROP TABLE IF EXISTS Casier;
DROP TABLE IF EXISTS Visiteur;
DROP TABLE IF EXISTS Tag;

CREATE TABLE Tag (
    tag VARCHAR(100) PRIMARY KEY NOT NULL,
    etat Enum('O','U','P') NOT NULL
) ENGINE = InnoDB;

# O = occuper
# U = utilisable
# P = perdu

CREATE TABLE Visiteur (
    id_Visiteur INT PRIMARY KEY AUTO_INCREMENT,
```

```
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    compagnie VARCHAR(100) NOT NULL,
    numPlaque VARCHAR(100) NOT NULL,
    pays VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE Casier (
    numeroCasier INT PRIMARY KEY NOT NULL
) ENGINE=InnoDB;

CREATE TABLE Affectation (
    id_Affectation INT PRIMARY KEY AUTO_INCREMENT,
    id_Casier INT NOT NULL,
    id_Visiteur INT NOT NULL,
    id_Tag VARCHAR(100) NOT NULL,
    dateDebut DATE NOT NULL,
    dateFin DATE NOT NULL,
    CONSTRAINT fk_id_Casier FOREIGN KEY (id_Casier) REFERENCES Casier(numeroCasier),
    CONSTRAINT fk_id_Visiteur FOREIGN KEY (id_Visiteur) REFERENCES Visiteur(id_Visiteur),
    CONSTRAINT fk_id_Tag FOREIGN KEY (id_Tag) REFERENCES Tag(tag)
) ENGINE=InnoDB;

CREATE TABLE Utilisateur (
    id_Administrateur INT PRIMARY KEY AUTO_INCREMENT,
    role Enum('admin', 'user') NOT NULL,
    login VARCHAR(100) NOT NULL,
    password VARCHAR(100) NOT NULL
) ENGINE=InnoDB;
```

requete.sql :

```
INSERT INTO `Utilisateur`(`role`, `login`, `password`) VALUES ("admin", "d2c72a3e81", "d2c72a3e81");
INSERT INTO `Utilisateur`(`role`, `login`, `password`) VALUES ("user", "c4ca2b3b861738", "c3c234249812");
INSERT INTO `Utilisateur`(`role`, `login`, `password`) VALUES ("user", "d2cd3338861830", "c3c234249812");

INSERT INTO `Casier`(`numeroCasier`) VALUES (1);
INSERT INTO `Casier`(`numeroCasier`) VALUES (2);
INSERT INTO `Casier`(`numeroCasier`) VALUES (3);
INSERT INTO `Casier`(`numeroCasier`) VALUES (4);

INSERT INTO `Tag`(`tag`,`etat`) VALUES ("010C2E14093E","U");
INSERT INTO `Tag`(`tag`,`etat`) VALUES ("2600B3271BA9","U");
INSERT INTO `Tag`(`tag`,`etat`) VALUES ("2600B43EFF53","U");
INSERT INTO `Tag`(`tag`,`etat`) VALUES ("2600B438C16B","U");
INSERT INTO `Tag`(`tag`,`etat`) VALUES ("2600B326FE4D","U");
```

```
INSERT INTO `Visiteur`(`nom`, `prenom`, `compagnie`, `numPlaque`) VALUES ( "e3ca2936831f30","e4ca2b3b861738", "ffe00b", "f2e66a62db4078d39a","f2cf2b32821732f9b8");
INSERT INTO `Visiteur`(`nom`, `prenom`, `compagnie`, `numPlaque`) VALUES ( "f0c2293e81","f2cd3338861830", "fbe108", "f7e46a66da4f78d396","e1cc3e369a1b30ba88d776");

INSERT INTO `Affectation`(`id_Casier`, `id_Visiteur`, `id_Tag`, `dateDebut`, `dateFin`) VALUES (1, 1, "010C2E14093E", "2020-01-01", "2020-01-02");
INSERT INTO `Affectation`(`id_Casier`, `id_Visiteur`, `id_Tag`, `dateDebut`, `dateFin`) VALUES (2, 2, "2600B3271BA9", "2020-01-01", "2020-01-02");
```

1.3.2 Application C++/CLI

MyForm.h :

```
#pragma once

#include "bdd.hpp"
#include <string>
#include <msclr\marshal_cppstd.h>

namespace InterfaceApp {

  using namespace System;
  using namespace System::ComponentModel;
  using namespace System::Collections;
  using namespace System::Windows::Forms;
  using namespace System::Data;
  using namespace System::Drawing;

  /// <summary>
  /// Description résumée de MyForm
  /// </summary>
  public ref class MyForm : public System::Windows::Forms::Form
  {
  public:

    String^ username;
    String^ password;


  private: System::Windows::Forms::Label^ login;
  public:

  private: System::Windows::Forms::Label^ label2;
  private: System::Windows::Forms::ListBox^ listeAffectation;
  private: System::Windows::Forms::Label^ textListAffectation;
```

```
private: System::Windows::Forms::Button^ Raffraichir;
private: System::Windows::Forms::Button^ Association;
private: System::Windows::Forms::ComboBox^ cbCasier;
private: System::Windows::Forms::ComboBox^ cbVisiteur;
private: System::Windows::Forms::ComboBox^ cdTag;
private: System::Windows::Forms::DateTimePicker^ dateDeb;
private: System::Windows::Forms::DateTimePicker^ dateFin;


public:

private: System::Windows::Forms::Button^ ajoutUtilisateur;
private: System::Windows::Forms::Button^ ajoutTag;
private: System::Windows::Forms::Button^ ajoutVisiteur;
private: System::Windows::Forms::Button^ suppressionVisiteur;
private: System::Windows::Forms::Button^ supprimeraffectation
;
private: System::Windows::Forms::Label^ label1;
private: System::Windows::Forms::TextBox^ tBNom;
private: System::Windows::Forms::TextBox^ tBPrenom;
private: System::Windows::Forms::TextBox^ tBPi;


private: System::Windows::Forms::TextBox^ tBCompagnie;

private: System::Windows::Forms::Label^ label3;
private: System::Windows::Forms::Label^ label4;
private: System::Windows::Forms::Label^ label5;
private: System::Windows::Forms::Label^ label6;
private: System::Windows::Forms::Button^ ajoutUtilisateu;
private: System::Windows::Forms::Button^ retour;


public:

  BDD* bdd = new BDD();
private: System::Windows::Forms::Button^ button1;
private: System::Windows::Forms::ComboBox^ cbRole;


private: System::Windows::Forms::Label^ label7;
private: System::Windows::Forms::Label^ label8;
private: System::Windows::Forms::Label^ label9;
private: System::Windows::Forms::Label^ label10;
private: System::Windows::Forms::Button^ button2;
private: System::Windows::Forms::TextBox^ tbUNom;
private: System::Windows::Forms::TextBox^ tbUPassword;
private: System::Windows::Forms::Button^ button3;
private: System::Windows::Forms::Button^ button4;
private: System::Windows::Forms::Label^ label11;
private: System::Windows::Forms::ComboBox^
```

```
cbChoixNumAfffectation;

  private: System::Windows::Forms::Label^ label12;
  private: System::Windows::Forms::Button^ button5;


  public:


    MyForm(void)
    {

      if (bdd->connecter() == false)
      {
        MessageBox::Show("Erreur de connexion é la base de donn
ées");
        Application::Exit();
      }

    Connexion();
    }

  protected:
    /// <summary>
    /// Nettoyage des ressources utilisées.
    /// </summary>
    ~MyForm()
    {
      if (components)
      {
        delete components;
      }
    }

  protected:



  private:
    /// <summary>
    /// Variable nécessaire au concepteur.
    /// </summary>
    System::ComponentModel::Container ^components;

#pragma region Windows Form Designer generated code

    // Page de connexion

    void Connexion(void)
    {
      // Définir les propriétés du formulaire
```

```
        this->Text = "Page de  Connexion";
        this->Width = 300;
        this->Height = 200;

        // Créer les labels
        Label^ labelUsername = gcnew Label();
        labelUsername->Text = "Login:";
        labelUsername->Left = 10;
        labelUsername->Top = 20;
        this->Controls->Add(labelUsername);

        Label^ labelPassword = gcnew Label();
        labelPassword->Text = "Mot de passe:";
        labelPassword->Left = 10;
        labelPassword->Top = 60;
        this->Controls->Add(labelPassword);

        // Créer les champs de texte
        TextBox^ textBoxUsername = gcnew TextBox();
        textBoxUsername->Left = 110;
        textBoxUsername->Top = 20;
        this->Controls->Add(textBoxUsername);

        TextBox^ textBoxPassword = gcnew TextBox();
        textBoxPassword->Left = 110;
        textBoxPassword->Top = 60;
        textBoxPassword->UseSystemPasswordChar = true; // Masquer
 le texte du mot de passe
        this->Controls->Add(textBoxPassword);

        // Bouton OK
        Button^ buttonOK = gcnew Button();
        buttonOK->Text = "OK";
        buttonOK->Left = 50;
        buttonOK->Top = 100;
        buttonOK->DialogResult = System::Windows::Forms::
DialogResult::OK;
        this->AcceptButton = buttonOK;
        this->Controls->Add(buttonOK);

        // Bouton Annuler
        Button^ buttonCancel = gcnew Button();
        buttonCancel->Text = "Annuler";
        buttonCancel->Left = 150;
        buttonCancel->Top = 100;
        buttonCancel->DialogResult = System::Windows::Forms::
DialogResult::Cancel;
        this->CancelButton = buttonCancel;
        this->Controls->Add(buttonCancel);

        // Gestionnaire d'événements pour le bouton OK
```

```
      buttonOK->Click += gcnew EventHandler(this, &MyForm::ConnexionClick);
      buttonCancel->Click += gcnew EventHandler(this, &MyForm::CancelClick);

    }

    void Admin()
    {
      this->login = (gcnew System::Windows::Forms::Label());
      this->listeAffectation = (gcnew System::Windows::Forms::ListBox());
      this->textListAffectation = (gcnew System::Windows::Forms::Label());
      this->Raffraichir = (gcnew System::Windows::Forms::Button());
      this->Association = (gcnew System::Windows::Forms::Button());
      this->cbCasier = (gcnew System::Windows::Forms::ComboBox());
      this->cbVisiteur = (gcnew System::Windows::Forms::ComboBox());
      this->cdTag = (gcnew System::Windows::Forms::ComboBox());
      this->dateDeb = (gcnew System::Windows::Forms::DateTimePicker());
      this->dateFin = (gcnew System::Windows::Forms::DateTimePicker());
      this->ajoutUtilisateur = (gcnew System::Windows::Forms::Button());
      this->ajoutTag = (gcnew System::Windows::Forms::Button());
      this->ajoutVisiteur = (gcnew System::Windows::Forms::Button());
      this->suppressionVisiteur = (gcnew System::Windows::Forms::Button());
      this->supprimeraffectation = (gcnew System::Windows::Forms::Button());
      this->SuspendLayout();
      //
      // login
      //
      this->login->AutoSize = true;
      this->login->Location = System::Drawing::Point(651, 0);
      this->login->Name = L"login";
      this->login->Size = System::Drawing::Size(53, 13);
      this->login->TabIndex = 0;
      this->login->Text = L"username";
      //
      // listeAffectation
      //
      this->listeAffectation->FormattingEnabled = true;
```

```
        this->listeAffectation->Location = System::Drawing::Point(60, 139);
        this->listeAffectation->Name = L"listeAffectation";
        this->listeAffectation->Size = System::Drawing::Size(126, 56);
        this->listeAffectation->TabIndex = 3;
        this->listeAffectation->SelectedIndexChanged += gcnew System::EventHandler(this, &MyForm::listeAffectation_SelectedIndexChanged);
        //
        // textListAffectation
        //
        this->textListAffectation->AutoSize = true;
        this->textListAffectation->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
        this->textListAffectation->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
          System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(0)));
        this->textListAffectation->Location = System::Drawing::Point(30, 100);
        this->textListAffectation->Name = L"textListAffectation";
        this->textListAffectation->Size = System::Drawing::Size(174, 15);
        this->textListAffectation->TabIndex = 2;
        this->textListAffectation->Text = L"Casiers en cours d\'utiilisation";
        //
        // Raffraichir
        //
        this->Raffraichir->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 9.75F, System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
          static_cast<System::Byte>(0)));
        this->Raffraichir->Location = System::Drawing::Point(74, 219);
        this->Raffraichir->Name = L"Raffraichir";
        this->Raffraichir->Size = System::Drawing::Size(97, 24);
        this->Raffraichir->TabIndex = 4;
        this->Raffraichir->Text = L"Raffraichir";
        this->Raffraichir->UseVisualStyleBackColor = true;
        this->Raffraichir->Click += gcnew System::EventHandler(this, &MyForm::Refresh_Click_1);
        //
        // Association
        //
        this->Association->Location = System::Drawing::Point(385, 203);
        this->Association->Name = L"Association";
        this->Association->Size = System::Drawing::Size(75, 23);
```

```
        this->Association->TabIndex = 5;
        this->Association->Text = L"Affectation";
        this->Association->UseVisualStyleBackColor = true;
        this->Association->Click += gcnew System::EventHandler(
  this, &MyForm::Association_Click);
        //
        // cbCasier
        //
        this->cbCasier->FormattingEnabled = true;
        this->cbCasier->Location = System::Drawing::Point(511,
  115);
        this->cbCasier->Name = L"cbCasier";
        this->cbCasier->Size = System::Drawing::Size(121, 21);
        this->cbCasier->TabIndex = 6;
        //
        // cbVisiteur
        //
        this->cbVisiteur->FormattingEnabled = true;
        this->cbVisiteur->Location = System::Drawing::Point(230,
  115);
        this->cbVisiteur->Name = L"cbVisiteur";
        this->cbVisiteur->Size = System::Drawing::Size(121, 21);
        this->cbVisiteur->TabIndex = 7;
        //
        // cdTag
        //
        this->cdTag->FormattingEnabled = true;
        this->cdTag->Location = System::Drawing::Point(371, 115);
        this->cdTag->Name = L"cdTag";
        this->cdTag->Size = System::Drawing::Size(121, 21);
        this->cdTag->TabIndex = 8;
        //
        // dateDeb
        //
        this->dateDeb->Location = System::Drawing::Point(230,
  166);
        this->dateDeb->Name = L"dateDeb";
        this->dateDeb->Size = System::Drawing::Size(184, 20);
        this->dateDeb->TabIndex = 9;
        //
        // dateFin
        //
        this->dateFin->Location = System::Drawing::Point(432,
  166);
        this->dateFin->Name = L"dateFin";
        this->dateFin->Size = System::Drawing::Size(200, 20);
        this->dateFin->TabIndex = 10;
        //
        // ajoutUtilisateur
        //
        this->ajoutUtilisateur->Font = (gcnew System::Drawing::
```

```
Font(L"Microsoft Sans Serif", 9.75F, System::Drawing::FontStyle
::Regular,
        System::Drawing::GraphicsUnit::Point, static_cast<
System::Byte>(0)));
      this->ajoutUtilisateur->Location = System::Drawing::Point
(25, 29);
      this->ajoutUtilisateur->Name = L"ajoutUtilisateur";
      this->ajoutUtilisateur->Size = System::Drawing::Size(97,
41);
      this->ajoutUtilisateur->TabIndex = 11;
      this->ajoutUtilisateur->Text = L"Créer utilisateur";
      this->ajoutUtilisateur->UseVisualStyleBackColor = true;
      this->ajoutUtilisateur->Click += gcnew System::
EventHandler(this, &MyForm::LienAjoutUtilisateur_Click);
      //
      // ajoutTag
      //
      this->ajoutTag->Font = (gcnew System::Drawing::Font(L"
Microsoft Sans Serif", 9.75F, System::Drawing::FontStyle::
Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(0)));
      this->ajoutTag->Location = System::Drawing::Point(570,
29);
      this->ajoutTag->Name = L"ajoutTag";
      this->ajoutTag->Size = System::Drawing::Size(97, 41);
      this->ajoutTag->TabIndex = 12;
      this->ajoutTag->Text = L"Ajout Tag";
      this->ajoutTag->UseVisualStyleBackColor = true;
      this->ajoutTag->Click += gcnew System::EventHandler(this,
 &MyForm::ajoutTag_Click);
      //
      // ajoutVisiteur
      //
      this->ajoutVisiteur->Font = (gcnew System::Drawing::Font(
L"Microsoft Sans Serif", 9.75F, System::Drawing::FontStyle::
Regular, System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(0)));
      this->ajoutVisiteur->Location = System::Drawing::Point
(157, 29);
      this->ajoutVisiteur->Name = L"ajoutVisiteur";
      this->ajoutVisiteur->Size = System::Drawing::Size(97, 41)
;
      this->ajoutVisiteur->TabIndex = 13;
      this->ajoutVisiteur->Text = L"Créer Visiteur";
      this->ajoutVisiteur->UseVisualStyleBackColor = true;
      this->ajoutVisiteur->Click += gcnew System::EventHandler(
this, &MyForm::LienAjoutVisiteur_Click);
      //
      // suppressionVisiteur
      //
      this->suppressionVisiteur->Font = (gcnew System::Drawing
```

```
::Font(L"Microsoft Sans Serif", 9.75F, System::Drawing::
FontStyle::Regular,
        System::Drawing::GraphicsUnit::Point, static_cast<
System::Byte>(0)));
      this->suppressionVisiteur->Location = System::Drawing::
Point(292, 29);
      this->suppressionVisiteur->Name = L"suppressionVisiteur";
      this->suppressionVisiteur->Size = System::Drawing::Size
(97, 41);
      this->suppressionVisiteur->TabIndex = 14;
      this->suppressionVisiteur->Text = L"Supprimer Visiteur";
      this->suppressionVisiteur->UseVisualStyleBackColor = true
;
      this->suppressionVisiteur->Click += gcnew System::
EventHandler(this, &MyForm::LienSuppressionVisiteur_Click);
      //
      // supprimeraffectation
      //
      this->supprimeraffectation->Font = (gcnew System::Drawing
::Font(L"Microsoft Sans Serif", 9.75F, System::Drawing::
FontStyle::Regular,
        System::Drawing::GraphicsUnit::Point, static_cast<
System::Byte>(0)));
      this->supprimeraffectation->Location = System::Drawing::
Point(437, 29);
      this->supprimeraffectation->Name = L"supprimeraffectation
";
      this->supprimeraffectation->Size = System::Drawing::Size
(97, 41);
      this->supprimeraffectation->TabIndex = 15;
      this->supprimeraffectation->Text = L"Supprimer
Affectation";
      this->supprimeraffectation->UseVisualStyleBackColor =
true;
      this->supprimeraffectation->Click += gcnew System::
EventHandler(this, &MyForm::LienSupprimeraffectation_Click);
      //
      // MyForm
      //
      this->AutoScaleDimensions = System::Drawing::SizeF(6, 13)
;
      this->AutoScaleMode = System::Windows::Forms::
AutoScaleMode::Font;
      this->BackColor = System::Drawing::SystemColors::
ScrollBar;
      this->ClientSize = System::Drawing::Size(705, 287);
      this->Controls->Add(this->supprimeraffectation);
      this->Controls->Add(this->suppressionVisiteur);
      this->Controls->Add(this->ajoutVisiteur);
      this->Controls->Add(this->ajoutTag);
      this->Controls->Add(this->ajoutUtilisateur);
```

```
        this->Controls->Add(this->dateFin);
        this->Controls->Add(this->dateDeb);
        this->Controls->Add(this->cdTag);
        this->Controls->Add(this->cbVisiteur);
        this->Controls->Add(this->cbCasier);
        this->Controls->Add(this->Association);
        this->Controls->Add(this->Raffraichir);
        this->Controls->Add(this->textListAffectation);
        this->Controls->Add(this->listeAffectation);
        this->Controls->Add(this->login);
        this->Load += gcnew System::EventHandler(this, &MyForm::
MyForm_Load);
        this->ResumeLayout(false);
        this->PerformLayout();
        this->Name = L"MyForm";
        this->Text = L"Interface";
    }

    void PageAjoutUtilisateur()
    {
        this->button1 = (gcnew System::Windows::Forms::Button());
        this->cbRole = (gcnew System::Windows::Forms::ComboBox())
;
        this->label7 = (gcnew System::Windows::Forms::Label());
        this->label8 = (gcnew System::Windows::Forms::Label());
        this->label9 = (gcnew System::Windows::Forms::Label());
        this->label10 = (gcnew System::Windows::Forms::Label());
        this->button2 = (gcnew System::Windows::Forms::Button());
        this->tbUNom = (gcnew System::Windows::Forms::TextBox());
        this->tbUPassword = (gcnew System::Windows::Forms::
TextBox());
        this->SuspendLayout();
        //
        // button1
        //
        this->button1->Location = System::Drawing::Point(89, 235)
;
        this->button1->Name = L"button1";
        this->button1->Size = System::Drawing::Size(75, 23);
        this->button1->TabIndex = 0;
        this->button1->Text = L"Créer";
        this->button1->UseVisualStyleBackColor = true;
        this->button1->Click += gcnew System::EventHandler(this,
&MyForm::ajoutUtilisateur_Click);
        //
        // cbRole
        //
        this->cbRole->FormattingEnabled = true;
        this->cbRole->Location = System::Drawing::Point(129, 181)
;
        this->cbRole->Name = L"cbRole";
```

```
      this->cbRole->Size = System::Drawing::Size(135, 21);
      this->cbRole->TabIndex = 1;
      //
      // label7
      //
      this->label7->AutoSize = true;
      this->label7->Font = (gcnew System::Drawing::Font(L"
Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
 System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(0)));
      this->label7->Location = System::Drawing::Point(12, 103);
      this->label7->Name = L"label7";
      this->label7->Size = System::Drawing::Size(46, 13);
      this->label7->TabIndex = 2;
      this->label7->Text = L"Login :";
      //
      // label8
      //
      this->label8->AutoSize = true;
      this->label8->Font = (gcnew System::Drawing::Font(L"
Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
 System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(0)));
      this->label8->Location = System::Drawing::Point(12, 143);
      this->label8->Name = L"label8";
      this->label8->Size = System::Drawing::Size(69, 13);
      this->label8->TabIndex = 3;
      this->label8->Text = L"Password :";
      //
      // label9
      //
      this->label9->AutoSize = true;
      this->label9->Font = (gcnew System::Drawing::Font(L"
Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
 System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(0)));
      this->label9->Location = System::Drawing::Point(12, 184);
      this->label9->Name = L"label9";
      this->label9->Size = System::Drawing::Size(41, 13);
      this->label9->TabIndex = 4;
      this->label9->Text = L"Réle :";
      //
      // label10
      //
      this->label10->AutoSize = true;
      this->label10->Font = (gcnew System::Drawing::Font(L"
Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
 System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(0)));
      this->label10->Location = System::Drawing::Point(64, 52);
      this->label10->Name = L"label10";
```

```
        this->label10->Size = System::Drawing::Size(133, 13);
        this->label10->TabIndex = 5;
        this->label10->Text = L"Ajout d\'un Utilisateur :";
        //
        // button2
        //
        this->button2->Location = System::Drawing::Point(-1, -1);
        this->button2->Name = L"button2";
        this->button2->Size = System::Drawing::Size(39, 29);
        this->button2->TabIndex = 6;
        this->button2->Text = L"<";
        this->button2->UseVisualStyleBackColor = true;
        this->button2->Click += gcnew System::EventHandler(this,
 &MyForm::retour_Click);
        //
        // tbUNom
        //
        this->tbUNom->Location = System::Drawing::Point(129, 103)
;
        this->tbUNom->Name = L"tbUNom";
        this->tbUNom->Size = System::Drawing::Size(135, 20);
        this->tbUNom->TabIndex = 7;
        //
        // tbUPassword
        //
        this->tbUPassword->Location = System::Drawing::Point(129,
 140);
        this->tbUPassword->Name = L"tbUPassword";
        this->tbUPassword->Size = System::Drawing::Size(135, 20);
        this->tbUPassword->TabIndex = 8;
        //
        // MyForm
        //
        this->ClientSize = System::Drawing::Size(381, 341);
        this->Controls->Add(this->tbUPassword);
        this->Controls->Add(this->tbUNom);
        this->Controls->Add(this->button2);
        this->Controls->Add(this->label10);
        this->Controls->Add(this->label9);
        this->Controls->Add(this->label8);
        this->Controls->Add(this->label7);
        this->Controls->Add(this->cbRole);
        this->Controls->Add(this->button1);
        this->Name = L"MyForm";
        this->Text = L"Creation d'utilisateur";
        this->ResumeLayout(false);
        this->PerformLayout();

    }

    void PageAjoutTag()
```

```
    {

    }

    void PageAjoutVisiteur()
    {
      this->label1 = (gcnew System::Windows::Forms::Label());
      this->tBNom = (gcnew System::Windows::Forms::TextBox());
      this->tBPrenom = (gcnew System::Windows::Forms::TextBox()
);
      this->tBPi = (gcnew System::Windows::Forms::TextBox());
      this->tBCompagnie = (gcnew System::Windows::Forms::
TextBox());
      this->label3 = (gcnew System::Windows::Forms::Label());
      this->label4 = (gcnew System::Windows::Forms::Label());
      this->label5 = (gcnew System::Windows::Forms::Label());
      this->label6 = (gcnew System::Windows::Forms::Label());
      this->ajoutUtilisateu = (gcnew System::Windows::Forms::
Button());
      this->retour = (gcnew System::Windows::Forms::Button());
      this->SuspendLayout();
      //
      // label1
      //
      this->label1->AutoSize = true;
      this->label1->Font = (gcnew System::Drawing::Font(L"
Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
 System::Drawing::GraphicsUnit::Point,
        static_cast<System::Byte>(0)));
      this->label1->Location = System::Drawing::Point(11, 38);
      this->label1->Name = L"label1";
      this->label1->Size = System::Drawing::Size(118, 13);
      this->label1->TabIndex = 0;
      this->label1->Text = L"Ajout d\'un Visiteur :";
      //
      // tBNom
      //
      this->tBNom->Location = System::Drawing::Point(169, 76);
      this->tBNom->Name = L"tBNom";
      this->tBNom->Size = System::Drawing::Size(157, 20);
      this->tBNom->TabIndex = 1;
      //
      // tBPrenom
      //
      this->tBPrenom->Location = System::Drawing::Point(169,
111);
      this->tBPrenom->Name = L"tBPrenom";
      this->tBPrenom->Size = System::Drawing::Size(157, 20);
      this->tBPrenom->TabIndex = 3;
      //
      // tBPi
```

```
        //
        this->tBPi->Location = System::Drawing::Point(169, 189);
        this->tBPi->Name = L"tBPi";
        this->tBPi->Size = System::Drawing::Size(157, 20);
        this->tBPi->TabIndex = 4;
        //
        // tBCompagnie
        //
        this->tBCompagnie->Location = System::Drawing::Point(169, 151);
        this->tBCompagnie->Name = L"tBCompagnie";
        this->tBCompagnie->Size = System::Drawing::Size(157, 20);
        this->tBCompagnie->TabIndex = 5;
        //
        // label3
        //
        this->label3->AutoSize = true;
        this->label3->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold, System::Drawing::GraphicsUnit::Point,
            static_cast<System::Byte>(0)));
        this->label3->Location = System::Drawing::Point(11, 79);
        this->label3->Name = L"label3";
        this->label3->Size = System::Drawing::Size(40, 13);
        this->label3->TabIndex = 6;
        this->label3->Text = L"Nom :";
        //
        // label4
        //
        this->label4->AutoSize = true;
        this->label4->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold, System::Drawing::GraphicsUnit::Point,
            static_cast<System::Byte>(0)));
        this->label4->Location = System::Drawing::Point(11, 114);
        this->label4->Name = L"label4";
        this->label4->Size = System::Drawing::Size(57, 13);
        this->label4->TabIndex = 7;
        this->label4->Text = L"Prénom :";
        //
        // label5
        //
        this->label5->AutoSize = true;
        this->label5->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold, System::Drawing::GraphicsUnit::Point,
            static_cast<System::Byte>(0)));
        this->label5->Location = System::Drawing::Point(11, 154);
        this->label5->Name = L"label5";
        this->label5->Size = System::Drawing::Size(77, 13);
        this->label5->TabIndex = 8;
```

```
			this->label5->Text = L"Compagnie :";
			//
			// label6
			//
			this->label6->AutoSize = true;
			this->label6->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold, System::Drawing::GraphicsUnit::Point,
				static_cast<System::Byte>(0)));
			this->label6->Location = System::Drawing::Point(11, 192);
			this->label6->Name = L"label6";
			this->label6->Size = System::Drawing::Size(152, 13);
			this->label6->TabIndex = 9;
			this->label6->Text = L"Plaque d\'immatriculation :";
			//
			// ajoutUtilisateu
			//
			this->ajoutUtilisateu->Location = System::Drawing::Point(379, 125);
			this->ajoutUtilisateu->Name = L"ajoutUtilisateu";
			this->ajoutUtilisateu->Size = System::Drawing::Size(75, 23);
			this->ajoutUtilisateu->TabIndex = 10;
			this->ajoutUtilisateu->Text = L"Ajouter";
			this->ajoutUtilisateu->UseVisualStyleBackColor = true;
			this->ajoutUtilisateu->Click += gcnew System::EventHandler(this, &MyForm::ajoutVisiteur_Click);
			//
			// retour
			//
			this->retour->Location = System::Drawing::Point(-1, -1);
			this->retour->Name = L"retour";
			this->retour->Size = System::Drawing::Size(34, 27);
			this->retour->TabIndex = 11;
			this->retour->Text = L"<";
			this->retour->UseVisualStyleBackColor = true;
			this->retour->Click += gcnew System::EventHandler(this, &MyForm::retour_Click);
			//
			// MyForm
			//
			this->ClientSize = System::Drawing::Size(504, 220);
			this->Controls->Add(this->retour);
			this->Controls->Add(this->ajoutUtilisateu);
			this->Controls->Add(this->label6);
			this->Controls->Add(this->label5);
			this->Controls->Add(this->label4);
			this->Controls->Add(this->label3);
			this->Controls->Add(this->tBCompagnie);
			this->Controls->Add(this->tBPi);
			this->Controls->Add(this->tBPrenom);
```

```
        this->Controls->Add(this->tBNom);
        this->Controls->Add(this->label1);
        this->Name = L"MyForm";
        this->Text = L"Ajout Visiteur";
        this->ResumeLayout(false);
        this->PerformLayout();
    }

    void PageSuppressionVisiteur()
    {

    }

    void PageSuppressionAffectation()
    {
        this->button3 = (gcnew System::Windows::Forms::Button());
        this->button4 = (gcnew System::Windows::Forms::Button());
        this->label11 = (gcnew System::Windows::Forms::Label());
        this->cbChoixNumAfffectation = (gcnew System::Windows::
Forms::ComboBox());
        this->label12 = (gcnew System::Windows::Forms::Label());
        this->button5 = (gcnew System::Windows::Forms::Button());
        this->SuspendLayout();
        //
        // button3
        //
        this->button3->Location = System::Drawing::Point(43, 142)
;
        this->button3->Name = L"button3";
        this->button3->Size = System::Drawing::Size(75, 23);
        this->button3->TabIndex = 0;
        this->button3->Text = L"Supprimer";
        this->button3->UseVisualStyleBackColor = true;
        this->button3->Click += gcnew System::EventHandler(this,
&MyForm::supprimeraffectation_Click);
        //
        // button4
        //
        this->button4->Location = System::Drawing::Point(-1, -1);
        this->button4->Name = L"button4";
        this->button4->Size = System::Drawing::Size(39, 29);
        this->button4->TabIndex = 6;
        this->button4->Text = L"<";
        this->button4->UseVisualStyleBackColor = true;
        this->button4->Click += gcnew System::EventHandler(this,
&MyForm::retour_Click);
        //
        // label11
        //
        this->label11->AutoSize = true;
        this->label11->Font = (gcnew System::Drawing::Font(L"
```

```
Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
 System::Drawing::GraphicsUnit::Point,
          static_cast<System::Byte>(0)));
        this->label11->Location = System::Drawing::Point(12, 48);
        this->label11->Name = L"label11";
        this->label11->Size = System::Drawing::Size(232, 13);
        this->label11->TabIndex = 7;
        this->label11->Text = L"Veuillez choisir l\'affectation é
 supprimer";
        //
        // cbChoixNumAfffectation
        //
        this->cbChoixNumAfffectation->FormattingEnabled = true;
        this->cbChoixNumAfffectation->Location = System::Drawing
::Point(165, 97);
        this->cbChoixNumAfffectation->Name = L"
cbChoixNumAfffectation";
        this->cbChoixNumAfffectation->Size = System::Drawing::
Size(121, 21);
        this->cbChoixNumAfffectation->TabIndex = 8;
        //
        // label12
        //
        this->label12->AutoSize = true;
        this->label12->Font = (gcnew System::Drawing::Font(L"
Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
 System::Drawing::GraphicsUnit::Point,
          static_cast<System::Byte>(0)));
        this->label12->Location = System::Drawing::Point(12, 100)
;
        this->label12->Name = L"label12";
        this->label12->Size = System::Drawing::Size(147, 13);
        this->label12->TabIndex = 9;
        this->label12->Text = L"Numéro de l\'affectation :";
        //
        // button5
        //
        this->button5->Location = System::Drawing::Point(308, 97)
;
        this->button5->Name = L"button5";
        this->button5->Size = System::Drawing::Size(36, 23);
        this->button5->TabIndex = 10;
        this->button5->Text = L"Info";
        this->button5->UseVisualStyleBackColor = true;
        this->button5->Click += gcnew System::EventHandler(this,
&MyForm::affichageInfo_Click);
        //
        // MyForm
        //
        this->ClientSize = System::Drawing::Size(356, 200);
        this->Controls->Add(this->button5);
```

```
        this->Controls->Add(this->label12);
        this->Controls->Add(this->cbChoixNumAfffectation);
        this->Controls->Add(this->label11);
        this->Controls->Add(this->button4);
        this->Controls->Add(this->button3);
        this->Text = L"Suppression d'affectation";
        this->Name = L"MyForm";
        this->ResumeLayout(false);
        this->PerformLayout();
    }

    void Utilisateur()
    {

        this->login = (gcnew System::Windows::Forms::Label());
        this->listeAffectation = (gcnew System::Windows::Forms::ListBox());
        this->textListAffectation = (gcnew System::Windows::Forms::Label());
        this->Raffraichir = (gcnew System::Windows::Forms::Button());
        this->Association = (gcnew System::Windows::Forms::Button());
        this->cbCasier = (gcnew System::Windows::Forms::ComboBox());
        this->cbVisiteur = (gcnew System::Windows::Forms::ComboBox());
        this->cdTag = (gcnew System::Windows::Forms::ComboBox());
        this->dateDeb = (gcnew System::Windows::Forms::DateTimePicker());
        this->dateFin = (gcnew System::Windows::Forms::DateTimePicker());
        this->SuspendLayout();
        //
        // login
        //
        this->login->AutoSize = true;
        this->login->Location = System::Drawing::Point(651, 0);
        this->login->Name = L"login";
        this->login->Size = System::Drawing::Size(53, 13);
        this->login->TabIndex = 0;
        this->login->Text = L"username";
        //
        // listeAffectation
        //
        this->listeAffectation->FormattingEnabled = true;
        this->listeAffectation->Location = System::Drawing::Point(59, 92);
        this->listeAffectation->Name = L"listeAffectation";
        this->listeAffectation->Size = System::Drawing::Size(110, 56);
```

```
        this->listeAffectation->TabIndex = 3;
        this->listeAffectation->SelectedIndexChanged += gcnew System::EventHandler(this, &MyForm::listeAffectation_SelectedIndexChanged);
        //
        // textListAffectation
        //
        this->textListAffectation->AutoSize = true;
        this->textListAffectation->BorderStyle = System::Windows::Forms::BorderStyle::FixedSingle;
        this->textListAffectation->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 8.25F, System::Drawing::FontStyle::Bold,
            System::Drawing::GraphicsUnit::Point, static_cast<System::Byte>(0)));
        this->textListAffectation->Location = System::Drawing::Point(29, 53);
        this->textListAffectation->Name = L"textListAffectation";
        this->textListAffectation->Size = System::Drawing::Size(174, 15);
        this->textListAffectation->TabIndex = 2;
        this->textListAffectation->Text = L"Casiers en cours d\'utiilisation";
        //
        // Raffraichir
        //
        this->Raffraichir->Font = (gcnew System::Drawing::Font(L"Microsoft Sans Serif", 9.75F, System::Drawing::FontStyle::Regular, System::Drawing::GraphicsUnit::Point,
            static_cast<System::Byte>(0)));
        this->Raffraichir->Location = System::Drawing::Point(305, 204);
        this->Raffraichir->Name = L"Raffraichir";
        this->Raffraichir->Size = System::Drawing::Size(97, 24);
        this->Raffraichir->TabIndex = 4;
        this->Raffraichir->Text = L"Raffraichir";
        this->Raffraichir->UseVisualStyleBackColor = true;
        this->Raffraichir->Click += gcnew System::EventHandler(this, &MyForm::Refresh_Click_1);
        //
        // Association
        //
        this->Association->Location = System::Drawing::Point(436, 156);
        this->Association->Name = L"Association";
        this->Association->Size = System::Drawing::Size(75, 23);
        this->Association->TabIndex = 5;
        this->Association->Text = L"Affectation";
        this->Association->UseVisualStyleBackColor = true;
        this->Association->Click += gcnew System::EventHandler(this, &MyForm::Association_Click);
```

```
        //
        // cbCasier
        //
        this->cbCasier->FormattingEnabled = true;
        this->cbCasier->Location = System::Drawing::Point(562,
68);
        this->cbCasier->Name = L"cbCasier";
        this->cbCasier->Size = System::Drawing::Size(121, 21);
        this->cbCasier->TabIndex = 6;
        //
        // cbVisiteur
        //
        this->cbVisiteur->FormattingEnabled = true;
        this->cbVisiteur->Location = System::Drawing::Point(281,
68);
        this->cbVisiteur->Name = L"cbVisiteur";
        this->cbVisiteur->Size = System::Drawing::Size(121, 21);
        this->cbVisiteur->TabIndex = 7;
        //
        // cdTag
        //
        this->cdTag->FormattingEnabled = true;
        this->cdTag->Location = System::Drawing::Point(422, 68);
        this->cdTag->Name = L"cdTag";
        this->cdTag->Size = System::Drawing::Size(121, 21);
        this->cdTag->TabIndex = 8;
        //
        // dateDeb
        //
        this->dateDeb->Location = System::Drawing::Point(281,
119);
        this->dateDeb->Name = L"dateDeb";
        this->dateDeb->Size = System::Drawing::Size(184, 20);
        this->dateDeb->TabIndex = 9;
        //
        // dateFin
        //
        this->dateFin->Location = System::Drawing::Point(483,
119);
        this->dateFin->Name = L"dateFin";
        this->dateFin->Size = System::Drawing::Size(200, 20);
        this->dateFin->TabIndex = 10;
        //
        // MyForm
        //
        this->AutoScaleDimensions = System::Drawing::SizeF(6, 13)
;
        this->AutoScaleMode = System::Windows::Forms::
AutoScaleMode::Font;
        this->BackColor = System::Drawing::SystemColors::
ScrollBar;
```

```
      this->ClientSize = System::Drawing::Size(705, 240);
      this->Controls->Add(this->dateFin);
      this->Controls->Add(this->dateDeb);
      this->Controls->Add(this->cdTag);
      this->Controls->Add(this->cbVisiteur);
      this->Controls->Add(this->cbCasier);
      this->Controls->Add(this->Association);
      this->Controls->Add(this->Raffraichir);
      this->Controls->Add(this->textListAffectation);
      this->Controls->Add(this->listeAffectation);
      this->Controls->Add(this->login);
      this->Name = L"MyForm";
      this->Text = L"Interface";
      this->Load += gcnew System::EventHandler(this, &MyForm::MyForm_Load);
      this->ResumeLayout(false);
      this->PerformLayout();
    }


//************************************************************************


    void InitializeComponent(void)
    {


    }


//************************************************************************

#pragma endregion
  private: System::Void MyForm_Load(System::Object^ sender, System::EventArgs^ e)
  {
  }

    //---------------------------- Code de la page de connexion ----------------------------//

  // Changement de l'username
  private: System::Void UseUsername()
  {
    this->login->Text = username;
    }

  private: System::Void ConnexionClick(Object^ sender, EventArgs^ e)
  {
    if (this->Controls[2]->Text == "" || this->Controls[3]->
```

```
Text == "" )
    {
      MessageBox::Show("Veuillez rentrez votre login ainsi que
votre mot de pase");
    }
    else
    {
      // Récupérer les informations de connexion
      msclr::interop::marshal_context context; // Permet la
conversion

      std::string log = context.marshal_as<std::string>(this->
Controls[2]->Text);
      std::string pass = context.marshal_as<std::string>(this->
Controls[3]->Text);

      // Verifier les logs de la connexion é la base de données

      if (bdd->estUnUtilisateurAutorise(log, pass) == false)
      {
        MessageBox::Show("Erreur de connexion é la base de donn
ées : Mot de passe ou Login faux");
      }
      else
      {
        if (bdd->estAdministrateur(log,pass) == true)
        {

          // Stocker les informations de connexion
          username = this->Controls[2]->Text;
          password = this->Controls[3]->Text;

          // Effacer le élements de la page de connexion
          this->Controls->Clear();

          // Créer les éléments de la page principale
          Admin();
          afficherAffectations();
          UseUsername();
          afficherVisiteur();
          afficherCasier();
          afficherTag();
        }
        else
        {
          // Stocker les informations de connexion
          username = this->Controls[2]->Text;
          password = this->Controls[3]->Text;

          // Effacer le élements de la page de connexion
          this->Controls->Clear();
```

```

            // Créer les éléments de la page principale
            Utilisateur();
            afficherAffectations();
            UseUsername();
            afficherVisiteur();
            afficherCasier();
            afficherTag();
          }


        }
      }
    }

    private: System::Void CancelClick(Object^ sender, EventArgs^
  e)
    {
      // Fermer le formulaire
      Application::Exit();
    }

    //--------------------------- Code de l'affichage des
  affectations ----------------------------//

    //Afficher les affectations
    private: System::Void afficherAffectations()
    {
      // Récupérer la liste des affectations
      std::vector<std::string> affectations = bdd->
  listeAffectation();

      //Ajouter "Casier né" devant le numéro de chaque
  affectation


      // Afficher les affectations
      for (int i = 0; i < affectations.size(); i++)
      {
        this->listeAffectation->Items->Add(gcnew String(
  affectations[i].c_str()));
      }
    }

    private: System::Void listeAffectation_SelectedIndexChanged(
  System::Object^ sender, System::EventArgs^ e)
    {
      // Récupérer l'index de l'élément sélectionné
      int index = this->listeAffectation->SelectedIndex;

      // Récupérer le nom de l'affectation
```

```
    msclr::interop::marshal_context context; // Permet la
conversion
    std::string affectation = context.marshal_as<std::string>(
this->listeAffectation->Items[index]->ToString());

    // Récupérer les informations de l'affectation
    std::vector<std::string> infos = bdd->infosAffectation(
affectation);

    // Afficher les informations de l'affectation
    String^ message = "Informations de l'affectation :\n\n";
    message += "Tag utilisé : " + gcnew String(infos[0].c_str()
) + "\n";
    message += "Prénom : " + gcnew String(infos[2].c_str()) + "
\t";
    message += "Nom : " + gcnew String(infos[1].c_str()) + "\n\
n";
    message += "Compagnie : " + gcnew String(infos[3].c_str())
+ "\n";
    message += "Numéro de la plaque : " + gcnew String(infos
[4].c_str()) + "\n\n";
    message += "Date de début : " + gcnew String(infos[6].c_str
()) + "\n";
    message += "Date de fin : " + gcnew String(infos[7].c_str()
) + "\n";
    MessageBox::Show(message);
  }

  private: System::Void Refresh_Click_1(System::Object^ sender,
 System::EventArgs^ e)
  {
    // Effacer les affectations actuelles
    this->listeAffectation->Items->Clear();
    afficherAffectations();

    // Effacer les éléments des ComboBox
    this->cbVisiteur->Items->Clear();
    this->cbCasier->Items->Clear();
    this->cdTag->Items->Clear();

    // Afficher les nouveaux éléments
    afficherVisiteur();
    afficherCasier();
    afficherTag();
  }
  //---------------------------- Code de la création d'
affectation ----------------------------//

  private: std::string extractDate(std:: string date)
  {
    std::string annee , mois , jour;
```

```
    annee = date.substr(6, 4);
    mois = date.substr(3, 2);
    jour = date.substr(0, 2);


    return annee + "-" + mois + "-" + jour;
  }

  private: System::Void afficherVisiteur()
  {
    // Récupérer la liste des Visiteurs non affectés
    std::vector<std::string> visiteurs = bdd->listeVisiteurNonAffecter();

    // Ajouter les visiteurs non affectés
    for (int i = 0; i < visiteurs.size(); i++)
    {
      this->cbVisiteur->Items->Add(gcnew String(visiteurs[i].c_str()));
    }
  }

  private: System::Void afficherCasier()
  {
    // Récupérer la liste des Casiers
    std::vector<std::string> casiers = bdd->listeCasierNonAffecter();

    // Ajouter les casiers
    for (int i = 0; i < casiers.size(); i++)
    {
      this->cbCasier->Items->Add(gcnew String(casiers[i].c_str()));
    }
  }

  private: System::Void afficherTag()
  {
    // Récupérer la liste des Tags
    std::vector<std::string> tags = bdd->listeTagNonAffecter();

    // Ajouter les tags
    for (int i = 0; i < tags.size(); i++)
    {

      this->cdTag->Items->Add(gcnew String(tags[i].c_str()));
    }
  }

```

```
  private: System::Void Association_Click(System::Object^ sender, System::EventArgs^ e)
  {

    // Récupérer les informations de l'affectation
    msclr::interop::marshal_context context; // Permet la conversion
    std::string visiteur = context.marshal_as<std::string>(this->cbVisiteur->Text);
    std::string casier = context.marshal_as<std::string>(this->cbCasier->Text);
    std::string tag = context.marshal_as<std::string>(this->cdTag->Text);
    std::string dateDebut = context.marshal_as<std::string>(this->dateDeb->Value.ToString());
    std::string dateFin = context.marshal_as<std::string>(this->dateFin->Value.ToString());

    dateDebut = extractDate(dateDebut);
    dateFin = extractDate(dateFin);

    int id = bdd->idVisiteur(visiteur);


    // Vérifier que les informations sont correctes
    if (visiteur == "" || casier == "" || tag == "" || dateDebut == "" || dateFin == "" || dateDebut == dateFin)
    {
      MessageBox::Show("Veuillez sélectionner un visiteur, un casier et un tag");
    }
    else
    {
      if (bdd->verificationAffectationExistente(visiteur, casier, tag) == false)
      {
        MessageBox::Show("Un élement est déja utilisé");
      }
      else
      {
        // Ajouter l'affectation
        bdd->ajouterAffectation(id, casier, tag, dateDebut, dateFin);

        // Actualiser les éléments
        this->listeAffectation->Items->Clear();
        afficherAffectations();
        this->cbVisiteur->Items->Clear();
        this->cbCasier->Items->Clear();
        this->cdTag->Items->Clear();
```

```
        afficherVisiteur();
        afficherCasier();
        afficherTag();
      }
    }
  }

  //--------------------------- Code de la partie Admin
---------------------------//

  private: System::Void LienAjoutUtilisateur_Click(System::
Object^ sender, System::EventArgs^ e)
  {
    // Effacer le formulaire actuel
    this->Controls->Clear();

    // Créer le formulaire d'ajout d'utilisateur
    PageAjoutUtilisateur();
    choixRole();
  }

  private: System::Void LienAjoutTag_Click(System::Object^
sender, System::EventArgs^ e)
  {
    // Effacer le formulaire actuel
    this->Controls->Clear();

    // Créer le formulaire d'ajout de tag
    PageAjoutTag();
  }

  private: System::Void LienAjoutVisiteur_Click(System::Object^
 sender, System::EventArgs^ e)
  {
    // Effacer le formulaire actuel
    this->Controls->Clear();

    // Créer le formulaire d'ajout de visiteur
    PageAjoutVisiteur();
  }

  private: System::Void LienSuppressionVisiteur_Click(System::
Object^ sender, System::EventArgs^ e)
  {
    // Effacer le formulaire actuel
    this->Controls->Clear();

    // Créer le formulaire de suppression de visiteur
    PageSuppressionVisiteur();
  }

```

```
  private: System::Void LienSupprimeraffectation_Click(System::Object^ sender, System::EventArgs^ e)
  {
    // Effacer le formulaire actuel
    this->Controls->Clear();

    // Créer le formulaire de suppression d'affectation
    PageSuppressionAffectation();
    affichageAffectationSuppr();
  }

  private: System::Void ajoutUtilisateur_Click(System::Object^ sender, System::EventArgs^ e)
  {
    String^ nom = this->tbUNom->Text;
    String^ password = this->tbUPassword->Text;
    String^ role = this->cbRole->Text;


    if (nom == "" || password == "" || role == "")
    {
      MessageBox::Show("Veuillez rentrer un nom, un mot de passe et un réle");
    }
    else
    {
      msclr::interop::marshal_context context; // Permet la conversion
      std::string nomS = context.marshal_as<std::string>(nom);
      std::string passwordS = context.marshal_as<std::string>(password);
      std::string roleS = context.marshal_as<std::string>(role);

      if (bdd->ajouterUtilisateur(nomS, passwordS, roleS) == false)
      {
        MessageBox::Show("Erreur lors de l'ajout de l'utilisateur");
      }
      else
      {
        MessageBox::Show("Utilisateur ajouté avec succés");
      }
    }

  }

  private: System::Void ajoutTag_Click(System::Object^ sender, System::EventArgs^ e)
  {
```

```
    }

    private: System::Void ajoutVisiteur_Click(System::Object^ sender, System::EventArgs^ e)
    {
        // Récupérer les informations de l'utilisateur
        String^ nom = this->tBNom->Text;
        String^ prenom = this->tBPrenom->Text;
        String^ compagnie = this->tBCompagnie->Text;
        String^ plaque = this->tBPi->Text;

        // Vérifier que les informations sont correctes
        if (nom == "" || prenom == "" || compagnie == "" || plaque == "")
        {
            MessageBox::Show("Veuillez rentrer un nom, un prénom, une compagnie et une plaque d'immatriculation");
        }
        else
        {
            // Ajouter l'utilisateur
            msclr::interop::marshal_context context; // Permet la conversion
            std::string nomS = context.marshal_as<std::string>(nom);
            std::string prenomS = context.marshal_as<std::string>(prenom);
            std::string compagnieS = context.marshal_as<std::string>(compagnie);
            std::string plaqueS = context.marshal_as<std::string>(plaque);

            if (bdd->ajouterVisiteur(nomS,prenomS,compagnieS,plaqueS) == false)
            {
                MessageBox::Show("Erreur lors de l'ajout de l'utilisateur");
            }
            else
            {
                MessageBox::Show("Utilisateur ajouté avec succés");
            }

        }
    }

    private: System::Void suppressionVisiteur_Click(System::Object^ sender, System::EventArgs^ e)
    {

    }
```

```

  private: System::Void supprimeraffectation_Click(System::Object^ sender, System::EventArgs^ e)
  {
    // Vérifier que l'affectation est vide
    if (this->cbChoixNumAfffectation->SelectedIndex == -1)
    {
      MessageBox::Show("Veuillez sélectionner une affectation");
      return;
    }
    else
    {
      // Supprimer l'affectation
      int index = this->cbChoixNumAfffectation->SelectedIndex;
      msclr::interop::marshal_context context; // Permet la conversion
      std::string affectation = context.marshal_as<std::string>(this->cbChoixNumAfffectation->Text);
      bdd->supprimerAffectation(affectation);

      // Actualiser les éléments
      this->listeAffectation->Items->Clear();
      afficherAffectations();
      this->cbChoixNumAfffectation->Items->Clear();
      affichageAffectationSuppr();
    }
  }

  private: System::Void affichageInfo_Click(System::Object^ sender, System::EventArgs^ e)
  {
    // Vérifier que l'affectation est vide
    if (this->cbChoixNumAfffectation->SelectedIndex == -1)
    {
      MessageBox::Show("Veuillez sélectionner une affectation");
      return;
    }
    else
    {

      // Affichage des informations de l'affectation sélectionnée
      int index = this->cbChoixNumAfffectation->SelectedIndex;
      msclr::interop::marshal_context context; // Permet la conversion
      std::string affectation = context.marshal_as<std::string>(this->cbChoixNumAfffectation->Text);
      std::vector<std::string> infos = bdd->infosAffectation(affectation);
```

```
        String^ message = "Informations de l'affectation :\n\n";
        message += "Tag utilisé : " + gcnew String(infos[0].c_str()) + "\n";
        message += "Prénom : " + gcnew String(infos[2].c_str()) + "\t";
        message += "Nom : " + gcnew String(infos[1].c_str()) + "\n\n";
        message += "Compagnie : " + gcnew String(infos[3].c_str()) + "\n";
        message += "Numéro de la plaque : " + gcnew String(infos[4].c_str()) + "\n\n";
        message += "Date de début : " + gcnew String(infos[6].c_str()) + "\n";
        message += "Date de fin : " + gcnew String(infos[7].c_str()) + "\n";
        MessageBox::Show(message);
      }
    }

    private: System::Void affichageAffectationSuppr()
    {
      std::vector<std::string> affectations = bdd->listeAffectation();

      // Afficher les affectations
      for (int i = 0; i < affectations.size(); i++)
      {
        this->cbChoixNumAfffectation->Items->Add(gcnew String(affectations[i].c_str()));
      }
    }

    private: System::Void retour_Click(System::Object^ sender, System::EventArgs^ e)
    {
      // Effacer le formulaire actuel
      this->Controls->Clear();

      // Créer le formulaire de connexion
      Admin();
      afficherAffectations();
      UseUsername();
      afficherVisiteur();
      afficherCasier();
      afficherTag();

    }

    private: System::Void choixRole()
    {
```

```
        // Afficher deux role disponible dansl acombo box

        this->cbRole->Items->Add("admin");
        this->cbRole->Items->Add("user");

    }

    //---------------------------- Code de l'interface graphique ----------------------------//
};
}
```

Crypt.hpp :

```
#ifndef _CRYPT_HPP_
#define _CRYPT_HPP_

#include <string>
#include <vector>

class Crypt {

  private:
        std::vector<unsigned char> m_key; // Clé secréte

        // Génére une clé à partir d'un mot de passe
        void generateKey(const std::string& password) {
            m_key.resize(32); // 256 bits
            for (size_t i = 0; i < 32; ++i) {
                m_key[i] = static_cast<unsigned char>(password[i % password.size()] + i);
            }
        }

    public:
        // Constructeur avec mot de passe
        Crypt(const std::string& password) {
            generateKey(password);
        }

        // Fonction de chiffrement
        std::string crypter(const std::string input) {
            std::string output = input;

            for (size_t i = 0; i < input.size(); ++i) {
                output[i] = input[i] ^ m_key[i % m_key.size()] ^ (i % 256); // XOR + Décalage
            }

            return output;
        }

```

```
        // Fonction de déchiffrement (identique au chiffrement)
        std::string decrypt(const std::string input) {
            return crypter(input); // XOR étant réversible, on
applique la même fonction
        }

};

#endif // !_CRYPT_HPP_
```

bdd.hpp :

```
#ifndef _BDD_
#define _BDD_

#include "mysql-connector-c++-noinstall-1.1.9-win32/include/
cppconn/driver.h"
#include "mysql-connector-c++-noinstall-1.1.9-win32/include/
cppconn/exception.h"
#include "mysql-connector-c++-noinstall-1.1.9-win32/include/
cppconn/resultset.h"
#include "mysql-connector-c++-noinstall-1.1.9-win32/include/
cppconn/statement.h"
#include "mysql-connector-c++-noinstall-1.1.9-win32/include/
cppconn/connection.h"
#include "mysql-connector-c++-noinstall-1.1.9-win32/include/
cppconn/prepared_statement.h"

#include <string>
#include <vector>
#include "Crypt.hpp"

Crypt* crypt = new Crypt("CryptageApplication");

class BDD {

  private:

    std::string m_database = "m_Casier";
    std::string m_host = "tcp://10.187.52.123:3306";

    //std::string m_host = "tcp://10.187.52.4:3306";
    //std::string m_database = "casier_b";
    std::string m_login = "casier";
    std::string m_password = "casier";



    sql::Driver* m_driver;
    sql::Connection* m_con;
    sql::Statement* m_stmt;
    sql::ResultSet* m_res;  // A SUPPRIMER ET A METTRE DANS LES
```

```
 METHODES EN LOCAL

    public:
        BDD::BDD()
            : m_driver(nullptr), m_con(nullptr), m_stmt(nullptr), m_res(nullptr)
        {}

        BDD::~BDD()
        {}

        bool BDD::connecter()
        {
            // Premier test du connecteur C++ Mysql


            try {
                m_driver = get_driver_instance();

                m_con = m_driver->connect(m_host, m_login, m_password);

                m_con->setSchema(m_database);

                m_stmt = m_con->createStatement();

            }
            catch (sql::SQLException& e) {
                // Gestion des execeptions
        return false;
            }
            return true;
        }

        bool BDD::connecter(std::string serveur, std::string login, std::string password, std::string baseDeDonnee)
        {

            try {

                sql::Driver* driver;
                sql::Connection* con;
                sql::Statement* stmt;
                sql::ResultSet* res;

                driver = get_driver_instance();

                con = driver->connect(serveur, login, password);

                con->setSchema(baseDeDonnee);
```

```


                m_stmt = m_con->createStatement();

            }
            catch (sql::SQLException& e) {
                // Gestion des execeptions
                return false;
            }
            return true;
        }

        void BDD::deconnecter()
        {
            delete m_res;
            delete m_stmt;
            delete m_con;

        }

        bool BDD::estUnUtilisateurAutorise(std::string log, std::string pass)
        {

            std::string verification = "SELECT * FROM Utilisateur" ;

            m_res = m_stmt->executeQuery(verification);

            while (m_res->next())
            {
        std::string temp1 = crypt->decrypt(m_res->getString("login"));
        std::string temp2 = crypt->decrypt(m_res->getString("password"));

                if (log == crypt->decrypt(m_res->getString("login")) && pass == crypt->decrypt(m_res->getString("password")))
                {
                    return true;
                }

            }
      return false;
        }

    bool BDD::estAdministrateur(std::string log, std::string pass)
    {
      std::string verification = "SELECT * FROM Utilisateur
```

```
WHERE role = 'admin' ";

      m_res = m_stmt->executeQuery(verification);

      while (m_res->next())
      {
        if (log == crypt->decrypt(m_res->getString("login")) &&
 pass == crypt->decrypt(m_res->getString("password")))
        {
          return true;
        }
      }
      return false;
    }

    std::vector<std::string> BDD::listeAffectation()
    {
      std::vector<std::string> liste;

      std::string affectation = "SELECT * FROM Affectation";

      m_res = m_stmt->executeQuery(affectation);

      while (m_res->next())
      {
        liste.push_back(m_res->getString("id_Casier"));
      }

      return liste;
    }

    std::vector<std::string> BDD::infosAffectation(std::string
casier)
    {
      std::vector<std::string> infos;

      std::string affectationInfos = "SELECT t.tag , v.nom , v.
prenom , v.compagnie , v.numPlaque , c.numeroCasier , a.
dateDebut , a.dateFin FROM Affectation a , Visiteur v , Tag t ,
 Casier c WHERE a.id_Visiteur = v.id_Visiteur AND a.id_Tag = t.
tag AND a.id_Casier = c.numeroCasier AND c.numeroCasier = '" +
casier + "'";

      m_res = m_stmt->executeQuery(affectationInfos);

      while (m_res->next())
      {
        infos.push_back(crypt->decrypt(m_res->getString("tag"))
);
        infos.push_back(crypt->decrypt(m_res->getString("nom"))
);
```

```
      infos.push_back(crypt->decrypt(m_res->getString("prenom")));
      infos.push_back(crypt->decrypt(m_res->getString("compagnie")));
      infos.push_back(crypt->decrypt(m_res->getString("numPlaque")));
      infos.push_back(m_res->getString("numeroCasier"));
      infos.push_back(m_res->getString("dateDebut"));
      infos.push_back(m_res->getString("dateFin"));
    }


    return infos;
  }

  std::vector<std::string> BDD::listeVisiteurNonAffecter()
  {
    std::vector<std::string> liste;

    std::string affectation = "SELECT * FROM Visiteur WHERE id_Visiteur NOT IN (SELECT id_Visiteur FROM Affectation)";

    m_res = m_stmt->executeQuery(affectation);

    while (m_res->next())
    {
      liste.push_back(crypt->decrypt(m_res->getString("nom")));
    }
    return liste;
  }

  std::vector<std::string> BDD::listeCasierNonAffecter()
  {
    std::vector<std::string> liste;

    std::string affectation = "SELECT * FROM Casier WHERE numeroCasier NOT IN (SELECT id_Casier FROM Affectation)";

    m_res = m_stmt->executeQuery(affectation);

    while (m_res->next())
    {
      liste.push_back(m_res->getString("numeroCasier"));
    }
    return liste;
  }

      std::vector<std::string> BDD::listeTagNonAffecter()
  {
    std::vector<std::string> liste;
```

```

    std::string affectation = "SELECT * FROM Tag WHERE tag NOT IN (SELECT id_Tag FROM Affectation)";

    m_res = m_stmt->executeQuery(affectation);

    while (m_res->next())
    {
      liste.push_back(m_res->getString("tag"));
    }
    return liste;
  }

  int BDD::idVisiteur(std::string nom)
  {
    std::string affectation = "SELECT id_Visiteur FROM Visiteur ";

    m_res = m_stmt->executeQuery(affectation);

    while (m_res->next())
    {
      if (nom == crypt->decrypt(m_res->getString("nom")))
      {
        return m_res->getInt("id_Visiteur");
      }
    }
  }

  bool BDD::ajouterAffectation(int id, std::string casier, std::string tag,std::string dateDebut, std::string dateFin)
  {
    std::string idString = std::to_string(id);

    tag = crypt->crypter(tag);

    std::string affectation = "INSERT INTO Affectation (id_Visiteur, id_Casier, id_Tag, dateDebut, dateFin) VALUES ('" + idString + "', '" + casier + "', '" + tag + "', '" + dateDebut + "', '" + dateFin + "')";

    m_stmt->execute(affectation);

    return true;
  }

  bool BDD::verificationAffectationExistente(std::string nom, std::string casier, std::string tag)
  {
    // Verifie si un Visiteur est deja affecter
    std::string affectation = "SELECT * FROM Affectation ";
```

```

    m_res = m_stmt->executeQuery(affectation);

    while (m_res->next())
    {
      if (nom == crypt->decrypt(m_res->getString("nom")) || casier == m_res->getString("id_Casier") || tag == crypt->decrypt(m_res->getString("id_Tag")))
      {
        return false;
      }
      else
      {
        return true;
      }
    }

    ////// A voir
  }

  bool BDD::ajouterVisiteur(std::string nom, std::string prenom, std::string compagnie, std::string plaque)
  {
    nom = crypt->crypter(nom);
    prenom = crypt->crypter(prenom);
    compagnie = crypt->crypter(compagnie);
    plaque = crypt->crypter(plaque);

    std::string affectation = "INSERT INTO Visiteur (nom, prenom, compagnie, numPlaque) VALUES ('" + nom + "', '" + prenom + "', '" + compagnie + "', '" + plaque + "')";

    m_stmt->execute(affectation);

    return true;
  }

  bool BDD::ajouterUtilisateur(std::string login, std::string password, std::string role)
  {
    login = crypt->crypter(login);
    password = crypt->crypter(password);

    std::string affectation = "INSERT INTO Utilisateur (login, password, role) VALUES ('" + login + "', '" + password + "', '" + role + "')";

    m_stmt->execute(affectation);

    return true;
  }
```

```
    bool BDD::ajouterCasier(std::string casier)
    {
      std::string affectation = "INSERT INTO Casier (numeroCasier) VALUES ('" + casier + "')";

      m_stmt->execute(affectation);

      return true;
    }

    bool BDD::ajouterTag(std::string tag)
    {
      tag = crypt->crypter(tag);

      std::string affectation = "INSERT INTO Tag (tag) VALUES ('" + tag + "')";

      m_stmt->execute(affectation);

      return true;
    }

    bool BDD::supprimerVisiteur(std::string nom)
    {
      std::string affectation = "SELECT * FROM Visiteur";

      m_stmt->execute(affectation);

      while (m_res->next())
      {
        if (nom == crypt->decrypt(m_res->getString("nom")))
        {
          std::string affectation = "DELETE FROM Visiteur WHERE nom = '" + crypt->crypter(nom) + "'";

          m_stmt->execute(affectation);

          return true;
        }
      }

      return false;

    }

    bool BDD::supprimerAffectation(std::string casier)
    {
      std::string affectation = "SELECT * FROM Affectation ";

      m_res = m_stmt->executeQuery(affectation);
```

```

        while (m_res->next())
        {
          if (casier == m_res->getString("id_Casier"))
          {
            std::string affectation = "DELETE FROM Affectation WHERE id_Casier = '" + casier + "'";

            m_stmt->execute(affectation);

            return true;
          }
        }
        return false;
      }
};
#endif // !_BDD_
```

MyForm.cpp :

```
#include "MyForm.h"

using namespace System;
using namespace System::Windows::Forms;
[STAThread]
void Main(array<String^>^ args)
{
  Application::EnableVisualStyles();
  Application::SetCompatibleTextRenderingDefault(false);
  InterfaceApp::MyForm form; // MonProjet
  Application::Run(% form);
}
```

1.3.3 Application de C#

Chiffrage.cs :

```
using System;
using System.Text;

namespace AppCasier
{
    public class ChiffrageXOR
    {
        private byte[] _key; // Clé générée à partir du mot de passe

        public ChiffrageXOR(string password)
        {
            _key = GenerateKey(password, 256); // Génération de la clé à partir du mot de passe
        }
```

```
        // Génère une clé pseudo-aléatoire basée sur un mot de passe (PRNG)
        private byte[] GenerateKey(string password, int length)
        {
            byte[] key = new byte[length];
            byte[] passwordBytes = Encoding.UTF8.GetBytes(password);
            int seed = 0;

            // Création d'un "seed" en XORant les bytes du mot de passe
            foreach (byte b in passwordBytes)
                seed ^= b;

            Random random = new Random(seed); // Initialisation du PRNG

            for (int i = 0; i < length; i++)
                key[i] = (byte)random.Next(0, 256); // Génération de bytes pseudo-aléatoires

            return key;
        }

        // Fonction de chiffrement XOR avancé
        public string Encrypt(string input)
        {
            byte[] inputBytes = Encoding.UTF8.GetBytes(input);
            byte[] outputBytes = new byte[inputBytes.Length];

            for (int i = 0; i < inputBytes.Length; i++)
            {
                outputBytes[i] = (byte)(inputBytes[i] ^ _key[i % _key.Length]); // XOR avec la clé
            }

            return BytesToHex(outputBytes); // Encodage en Hexadécimal
        }

        // Fonction de déchiffrement
        public string Decrypt(string input)
        {
            byte[] inputBytes = HexToBytes(input);
            byte[] outputBytes = new byte[inputBytes.Length];

            for (int i = 0; i < inputBytes.Length; i++)
            {
                outputBytes[i] = (byte)(inputBytes[i] ^ _key[i % _key.Length]); // XOR avec la clé
```

```
            }

            return Encoding.UTF8.GetString(outputBytes);
        }

        // Encodage en Hexadécimal
        private string BytesToHex(byte[] bytes)
        {
            StringBuilder hex = new StringBuilder(bytes.Length * 2);
            foreach (byte b in bytes)
                hex.AppendFormat("{0:x2}", b);
            return hex.ToString();
        }

        // Décodage Hexadécimal vers byte[]
        private byte[] HexToBytes(string hex)
        {
            byte[] bytes = new byte[hex.Length / 2];
            for (int i = 0; i < bytes.Length; i++)
                bytes[i] = Convert.ToByte(hex.Substring(i * 2, 2), 16);
            return bytes;
        }
    }
}
```

Connexion.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{

    public partial class Connexion : Form
    {
        private MainForm mainForm; // Référence vers MainForm
        DatabaseConnection db = new DatabaseConnection(); // Instance de la classe DatabaseConnection
        public Connexion(MainForm form)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
```

```
            mainForm = form;
            if (db.OpenConnection() == false)
            {
                MessageBox.Show("Connexion échouée !");
            }
        }

        private void btConnexion_Click(object sender, EventArgs e)
        {
            if (db.estAdministateur(tbLogin.Text))
            {
                if (db.estUtilisateur(tbLogin.Text, tbPasswd.Text))
                {
                    mainForm.SetLogin(tbLogin.Text); // Met à jour les données
                    mainForm.SetRole("admin"); // Met à jour les données
                    this.Close(); // Ferme la page actuelle pour revenir à ‘MainForm‘

                }
                else
                {
                    MessageBox.Show("Connexion échouée !");
                }
            }
            else if (db.estUtilisateur(tbLogin.Text, tbPasswd.Text))
            {
                mainForm.SetLogin(tbLogin.Text); // Met à jour les données
                mainForm.SetRole("user"); // Met à jour les données
                this.Close(); // Ferme la page actuelle pour revenir à ‘MainForm‘
            }
            else
            {
                MessageBox.Show("Connexion échouée !");
            }


        }

        private void btCancel_Click(object sender, EventArgs e)
        {
            this.Close(); // Ferme la page actuelle pour revenir à ‘MainForm‘
        }
```

```

        private void Connexion_FormClosing(object sender,
FormClosingEventArgs e)
        {
            // Verifier si un utilisateur a été choisi
            if (mainForm.GetLogin() == "")
            {
                // Si non, fermer l'application
                Application.Exit();
            }
        }
    }
}
```

Connexion.Designer.cs :

```
namespace AppCasier
{
    partial class Connexion : System.Windows.Forms.Form
    {
        /// <summary>
        /// Variable nécessaire au concepteur.
        /// </summary>
        private System.ComponentModel.IContainer components =
null;

        /// <summary>
        /// Nettoyage des ressources utilisées.
        /// </summary>
        /// <param name="disposing">true si les ressources
managées doivent être supprimées ; sinon, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Code généré par le Concepteur Windows Form

        /// <summary>
        /// Méthode requise pour la prise en charge du
concepteur - ne modifiez pas
        /// le contenu de cette méthode avec l'éditeur de code.
        /// </summary>
        private void InitializeComponent()
        {
            this.lbConnexion = new System.Windows.Forms.Label()
;
            this.tbLogin = new System.Windows.Forms.TextBox();
```

```
            this.tbPasswd = new System.Windows.Forms.TextBox();
            this.lbLogin = new System.Windows.Forms.Label();
            this.lbPasswd = new System.Windows.Forms.Label();
            this.btConnexion = new System.Windows.Forms.Button();
            this.btCancel = new System.Windows.Forms.Button();
            this.SuspendLayout();
            //
            // lbConnexion
            //
            this.lbConnexion.AutoSize = true;
            this.lbConnexion.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbConnexion.Location = new System.Drawing.Point(12, 9);
            this.lbConnexion.Name = "lbConnexion";
            this.lbConnexion.Size = new System.Drawing.Size(151, 13);
            this.lbConnexion.TabIndex = 0;
            this.lbConnexion.Text = "Veuillez vous connecter :";
            //
            // tbLogin
            //
            this.tbLogin.Location = new System.Drawing.Point(109, 60);
            this.tbLogin.Name = "tbLogin";
            this.tbLogin.Size = new System.Drawing.Size(130, 20);
            this.tbLogin.TabIndex = 1;
            //
            // tbPasswd
            //
            this.tbPasswd.Location = new System.Drawing.Point(109, 101);
            this.tbPasswd.Name = "tbPasswd";
            this.tbPasswd.Size = new System.Drawing.Size(130, 20);
            this.tbPasswd.TabIndex = 2;
            this.tbPasswd.UseSystemPasswordChar = true;
            //
            // lbLogin
            //
            this.lbLogin.AutoSize = true;
            this.lbLogin.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbLogin.Location = new System.Drawing.Point(12, 60);
            this.lbLogin.Name = "lbLogin";
```

```
            this.lbLogin.Size = new System.Drawing.Size(46, 13);
            this.lbLogin.TabIndex = 4;
            this.lbLogin.Text = "Login :";
            //
            // lbPasswd
            //
            this.lbPasswd.AutoSize = true;
            this.lbPasswd.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbPasswd.Location = new System.Drawing.Point(12, 104);
            this.lbPasswd.Name = "lbPasswd";
            this.lbPasswd.Size = new System.Drawing.Size(91, 13);
            this.lbPasswd.TabIndex = 5;
            this.lbPasswd.Text = "Mot de passe :";
            //
            // btConnexion
            //
            this.btConnexion.Location = new System.Drawing.Point(28, 157);
            this.btConnexion.Name = "btConnexion";
            this.btConnexion.Size = new System.Drawing.Size(75, 23);
            this.btConnexion.TabIndex = 6;
            this.btConnexion.Text = "Connexion";
            this.btConnexion.UseVisualStyleBackColor = true;
            this.btConnexion.Click += new System.EventHandler(this.btConnexion_Click);
            //
            // btCancel
            //
            this.btCancel.Location = new System.Drawing.Point(154, 157);
            this.btCancel.Name = "btCancel";
            this.btCancel.Size = new System.Drawing.Size(75, 23);
            this.btCancel.TabIndex = 7;
            this.btCancel.Text = "Annuler";
            this.btCancel.UseVisualStyleBackColor = true;
            this.btCancel.Click += new System.EventHandler(this.btCancel_Click);
            //
            // Connexion
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
```

```
            this.ClientSize = new System.Drawing.Size(275, 206)
;
            this.Controls.Add(this.btCancel);
            this.Controls.Add(this.btConnexion);
            this.Controls.Add(this.lbPasswd);
            this.Controls.Add(this.lbLogin);
            this.Controls.Add(this.tbPasswd);
            this.Controls.Add(this.tbLogin);
            this.Controls.Add(this.lbConnexion);
            this.Name = "Connexion";
            this.Text = "Connexion";
            this.FormClosing += new System.Windows.Forms.
FormClosingEventHandler(this.Connexion_FormClosing);
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label lbConnexion;
        private System.Windows.Forms.TextBox tbLogin;
        private System.Windows.Forms.TextBox tbPasswd;
        private System.Windows.Forms.Label lbLogin;
        private System.Windows.Forms.Label lbPasswd;
        private System.Windows.Forms.Button btConnexion;
        private System.Windows.Forms.Button btCancel;
    }
}
```

DatabaseConnection.cs :

```
using System;
using System.Collections.Generic;
using System.Windows.Forms;
using MySqlConnector; // Assurez-vous d'importer cette biblioth
èque
using static System.Windows.Forms.VisualStyles.
VisualStyleElement;

//d2c72a3e81

namespace AppCasier
{
    internal class DatabaseConnection
    {
        private MySqlConnection connection;

        private ChiffrageXOR chiffrage = new ChiffrageXOR("
ChiffrementXORApplication"); // Chiffrement du mot de passe

        // Chaîne de connexion pour MySQL/MariaDB
```

```
        private string connectionString = "server=10.187.52.4;userid=casier;password=casier;database=casier_b;";
        //private string connectionString = "server=10.187.52.123;userid=casier;password=casier;database=m_Casier;";
        //private string connectionString = "server=10.187.52.123;userid=casier;password=casier;database=m_Casier;";


        public DatabaseConnection()
        {
            connection = new MySqlConnection(connectionString); // Initialisation de la connexion
        }

        // Ouvrir la connexion à la base de données

        public bool OpenConnection()
        {
            try
            {
                connection.Open();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la connexion à la base de données : " + ex.Message);
                return false;
            }
        }

        public bool CloseConnection()
        {
            try
            {
                connection.Close();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la fermeture de la connexion à la base de données : " + ex.Message);
                return false;
            }
        }


        // Exécution d'une requête
        public bool ExecuteQuery(string query)
```

```
        {
            try
            {
                MySqlCommand cmd = new MySqlCommand(query, connection);
                int result = cmd.ExecuteNonQuery();  // Utilisé pour les requêtes qui ne retournent pas de données
                return true;

            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de l'exécution de la requête : " + ex.Message);
                return false;

            }
        }

        public bool estUtilisateur(string login, string pass)
        {
            try
            {
                string requete = "SELECT * FROM Utilisateur";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                while (reader.Read())
                {
                    if (login == chiffrage.Decrypt(reader[2].ToString()) && pass == chiffrage.Decrypt(reader[3].ToString()))
                    {
                        reader.Close();
                        return true;
                    }
                }
                reader.Close();
                return false;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la vérification de l'utilisateur : " + ex.Message);
                return false;
            }
        }

        public bool estAdministateur(string login)
        {
```

```
            try
            {


                string requete = "SELECT * FROM Utilisateur WHERE role = ’admin’ ";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                login = chiffrage.Encrypt(login);

                while (reader.Read())
                {
                    if (reader[2].ToString() == login)
                    {
                        reader.Close();
                        return true;
                    }
                }
                reader.Close();
                return false;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la vérification de l’administrateur : " + ex.Message);
                return false;
            }
        }

        public string[] listeAffectation()
        {
            try
            {
                string requete = "SELECT * FROM Affectation";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                List<string> liste = new List<string>();

                while (reader.Read())
                {
                    liste.Add(reader[1].ToString());
                }
                reader.Close();
                return liste.ToArray();
            }
```

```
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des affectations : " + ex.Message);
                return null;
            }
        }

        public string[] detailsAffectation(string casier)
        {
            try
            {
                string requete = "SELECT t.tag , v.nom , v.prenom , v.compagnie , v.numPlaque , c.numeroCasier , a.dateDebut , a.dateFin , v.pays FROM Affectation a, Visiteur v , Tag t, Casier c WHERE a.id_Visiteur = v.id_Visiteur AND a.id_Tag = t.tag AND a.id_Casier = c.numeroCasier AND c.numeroCasier = '" + casier + "'";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                reader.Read();

                string[] details = new string[9];
                details[0] = reader[0].ToString();
                details[1] = reader[1].ToString();
                details[2] = reader[2].ToString();
                details[3] = reader[3].ToString();
                details[4] = reader[4].ToString();
                details[5] = reader[5].ToString();
                details[6] = reader[6].ToString();
                details[7] = reader[7].ToString();
                details[8] = reader[8].ToString();

                reader.Close();
                return details;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des détails de l'affectation : " + ex.Message);
                return null;
            }
        }

        public bool ajouterAffectation(string tag, string nom, string prenom ,string casier, string dateDeb, string dateFin)
        {
            try
```

```
            {
                nom = chiffrage.Encrypt(nom);
                prenom = chiffrage.Encrypt(prenom);

                if (!updateEtatTagO(tag)) throw new Exception("
Erreur lors de la mise à jour du tag");

                string requete = "INSERT INTO Affectation (id_
Tag, id_Visiteur, id_Casier, dateDebut, dateFin) VALUES ('" +
tag + "', (SELECT id_Visiteur FROM Visiteur WHERE nom = '" +
nom + "' AND prenom = '" + prenom + "' ) , '" + casier + "' , '
" + dateDeb + "' , '" + dateFin + "')";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de l'ajout de l'
affectation : " + ex.Message);
                return false;
            }
        }

        public void supprimerAffectation(string casier)
        {
            try
            {
                if (!updateEtatTagU(casier)) throw new
Exception("Erreur lors de la mise à jour du tag");

                string requete = "DELETE FROM Affectation WHERE
 id_Casier = '" + casier + "'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                cmd.ExecuteNonQuery();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la suppression
de l'affectation : " + ex.Message);
            }
        }

        public string[] recupNomVisiteurNonAffecté()
        {
            try
            {
```

```
                string requete = "SELECT * FROM Visiteur WHERE
id_Visiteur NOT IN (SELECT id_Visiteur FROM Affectation)";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                List<string> liste = new List<string>();

                while (reader.Read())
                {
                    liste.Add(chiffrage.Decrypt(reader[1].
ToString()));
                }
                reader.Close();
                return liste.ToArray();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération
 des visiteurs : " + ex.Message);
                return null;
            }
        }

        public string[] recupNumCasierNonAffecté()
        {
            try
            {
                string requete = "SELECT * FROM Casier WHERE
numeroCasier NOT IN (SELECT id_Casier FROM Affectation)";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                List<string> liste = new List<string>();

                while (reader.Read())
                {
                    liste.Add(reader[0].ToString());
                }
                reader.Close();
                return liste.ToArray();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération
 des casiers : " + ex.Message);
                return null;
            }
```

```
        }

        public string[] recupNumTagNonAffecté()
        {
            try
            {
                string requete = "SELECT * FROM Tag WHERE tag NOT IN (SELECT id_Tag FROM Affectation)";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                List<string> liste = new List<string>();

                while (reader.Read())
                {
                    liste.Add(reader[0].ToString());
                }
                reader.Close();
                return liste.ToArray();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des tags : " + ex.Message);
                return null;
            }
        }

        public bool ajouterUtilisateur(string login, string pass, string role)
        {
            try
            {
                // Chiffrage du login et du mot de passe
                login = chiffrage.Encrypt(login);
                pass = chiffrage.Encrypt(pass);

                string requete = "SELECT * FROM Utilisateur WHERE login = '" + login + "'";

                // Vérifier si l'utilisateur existe déjà

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                if (reader.Read())
                {
                    reader.Close();
```

```
                    return false;
                }
                else
                {
                    reader.Close();

                    requete = "INSERT INTO Utilisateur (login, password, role) VALUES (’" + login + "’, ’" + pass + "’, ’" + role + "’)";

                    MySqlCommand cmd1 = new MySqlCommand(requete, connection);
                    cmd1.ExecuteNonQuery();
                    return true;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de l’ajout de l’utilisateur : " + ex.Message);
                return false;
            }
        }

        public bool ajouterVisiteur(string nom, string prenom, string plaque, string compagnie,string pays)
        {
            try
            {

                // Chiffrage des informations
                nom = chiffrage.Encrypt(nom);
                prenom = chiffrage.Encrypt(prenom);
                plaque = chiffrage.Encrypt(plaque);
                compagnie = chiffrage.Encrypt(compagnie);
                pays = chiffrage.Encrypt(pays);

                string requete = "INSERT INTO Visiteur (nom, prenom, numPlaque, compagnie,pays) VALUES (’" + nom + "’, ’" + prenom + "’, ’" + plaque + "’, ’" + compagnie + "’ ,’" + pays + "’)";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de l’ajout du visiteur : " + ex.Message);
```

```
                return false;
            }
        }

        public string[] recupNomPrenomVisiteurNonAffecté()
        {
            try
            {
                string requete = "SELECT * FROM Visiteur WHERE id_Visiteur NOT IN (SELECT id_Visiteur FROM Affectation)";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                List<string> liste = new List<string>();

                while (reader.Read())
                {
                    liste.Add(chiffrage.Decrypt(reader[1].ToString()) + " " + chiffrage.Decrypt(reader[2].ToString()));
                }
                reader.Close();
                return liste.ToArray();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des visiteurs : " + ex.Message);
                return null;
            }
        }

        public string[] recupNomUtilisateur()
        {
            try
            {
                string requete = "SELECT * FROM Utilisateur";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                List<string> liste = new List<string>();

                while (reader.Read())
                {
                    liste.Add(chiffrage.Decrypt(reader[2].ToString()));
                }
                reader.Close();
```

```
                return liste.ToArray();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des utilisateurs : " + ex.Message);
                return null;
            }
        }


        public bool supprimerVisiteur(string nom, string prenom)
        {
            try
            {
                nom = chiffrage.Encrypt(nom);
                prenom = chiffrage.Encrypt(prenom);

                string requete = "DELETE FROM Visiteur WHERE nom = '" + nom + "' AND prenom = '" + prenom + "'";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la suppression du visiteur : " + ex.Message);
                return false;
            }
        }

        public bool supprimerUtilisateur(string login)
        {
            try
            {
                login = chiffrage.Encrypt(login);

                string requete = "DELETE FROM Utilisateur WHERE login = '" + login + "'";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
```

```
                MessageBox.Show("Erreur lors de la suppression
de l'utilisateur : " + ex.Message);
                return false;
            }
        }

        public bool supprimerTag(string tag)
        {
            try
            {
                string requete = "DELETE FROM Tag WHERE tag = '
" + tag + "' AND etat = 'U'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la suppression
du tag : " + ex.Message);
                return false;
            }
        }

        public bool verifyVisiteur(string nom, string prenom)
        {
            try
            {
                nom = chiffrage.Encrypt(nom);
                prenom = chiffrage.Encrypt(prenom);

                string requete = "SELECT * FROM Visiteur WHERE
nom = '" + nom + "' AND prenom = '" + prenom + "'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                if (reader.Read())
                {
                    reader.Close();
                    return true;
                }
                else
                {
                    reader.Close();
                    return false;
                }
            }
```

```
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la vérification
 du visiteur : " + ex.Message);
                return false;
            }
        }

        public bool verifyUtilisateur(string login)
        {
            try
            {
                login = chiffrage.Encrypt(login);
                string requete = "SELECT * FROM Utilisateur
WHERE login = '" + login + "'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                if (reader.Read())
                {
                    reader.Close();
                    return true;
                }
                else
                {
                    reader.Close();
                    return false;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la vérification
 de l'utilisateur : " + ex.Message);
                return false;
            }
        }

        public string[] affectationDateLimite()
        {
            try
            {
                string requete = "SELECT * FROM Affectation a,
Tag t WHERE a.id_Tag = t.tag AND a.dateFin < NOW() AND t.etat =
 '0'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                MySqlDataReader reader = cmd.ExecuteReader();

```

```
                List<string> liste = new List<string>();

                while (reader.Read())
                {
                    liste.Add(reader[0].ToString());
                }
                reader.Close();
                return liste.ToArray();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des affectations : " + ex.Message);
                return null;
            }
        }

        public string[] infoAffectation(string id)
        {
            try
            {
                string requete = "SELECT t.tag , v.nom , v.prenom , v.compagnie , v.numPlaque , c.numeroCasier , a.dateDebut , a.dateFin , v.pays FROM Affectation a, Visiteur v , Tag t, Casier c WHERE a.id_Visiteur = v.id_Visiteur AND a.id_Tag = t.tag AND a.id_Casier = c.numeroCasier AND a.id_Affectation = '" + id + "'";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                reader.Read();

                string[] details = new string[9];
                details[0] = reader[0].ToString();
                details[1] = reader[1].ToString();
                details[2] = reader[2].ToString();
                details[3] = reader[3].ToString();
                details[4] = reader[4].ToString();
                details[5] = reader[5].ToString();
                details[6] = reader[6].ToString();
                details[7] = reader[7].ToString();
                details[8] = reader[8].ToString();




                reader.Close();
                return details;
            }
```

```
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des info de l’affectation : " + ex.Message);
                return null;
            }
        }

        public bool verifTag(string tag)
        {
            try
            {
                string requete = "SELECT * FROM Tag WHERE tag = ’" + tag + "’";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                if (reader.Read())
                {
                    reader.Close();
                    return true;
                }
                else
                {
                    reader.Close();
                    return false;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la vérification du tag : " + ex.Message);
                return false;
            }
        }

        public bool ajouterTag(string tag)
        {
            try
            {
                string requete = "INSERT INTO Tag (tag,etat) VALUES (’" + tag + "’,’U’)";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
```

```
            {
                MessageBox.Show("Erreur lors de l'ajout du tag
 : " + ex.Message);
                return false;
            }
        }

        public bool rendrePerdu(string tag)
        {
            try
            {
                string requete = "UPDATE Tag SET etat = 'P'
WHERE tag = '" + tag + "'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la mise à jour
du tag : " + ex.Message);
                return false;
            }
        }

        public bool rendreUtilisable(string tag)
        {
            try
            {
                string requete = "UPDATE Tag SET etat = 'U'
WHERE tag = '" + tag + "'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la mise à jour
du tag : " + ex.Message);
                return false;
            }
        }

        public bool updateEtatTagO(string tag)
        {
            try
            {
```

```
                string requete = "UPDATE Tag SET etat = 'O'
WHERE tag = '" + tag + "'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la mise à jour
du tag : " + ex.Message);
                return false;
            }
        }

        public bool updateEtatTagU(string casier)
        {
            try
            {
                string requete = "UPDATE Tag SET etat = 'U'
WHERE tag = (SELECT id_Tag FROM Affectation WHERE id_Casier = '
" + casier + "')";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                cmd.ExecuteNonQuery();
                return true;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la mise à jour
du tag : " + ex.Message);
                return false;
            }
        }

        public string[] recupTagPO()
        {
            try
            {
                string requete = "SELECT * FROM Tag WHERE etat
= 'P' OR etat = 'O'";

                MySqlCommand cmd = new MySqlCommand(requete,
connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                List<string> liste = new List<string>();

                while (reader.Read())
```

```
                {
                    liste.Add(reader[0].ToString());
                }
                reader.Close();
                return liste.ToArray();
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération des tags : " + ex.Message);
                return null;
            }

        }

        public char getEtatTag(string tag)
        {
            try
            {
                string requete = "SELECT * FROM Tag WHERE tag = '" + tag + "'";

                MySqlCommand cmd = new MySqlCommand(requete, connection);
                MySqlDataReader reader = cmd.ExecuteReader();

                reader.Read();
                char etat = Convert.ToChar(reader[1].ToString());
                reader.Close();
                return etat;
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la récupération de l'état du tag : " + ex.Message);
                return '?';
            }
        }

        public bool updateTag(string tag, string etat)
        {
            try
            {
                if (etat == "O")
                {
                    string requete = "UPDATE Tag SET etat = 'P' WHERE tag = '" + tag + "'";
                    MySqlCommand cmd = new MySqlCommand(requete, connection);
                    cmd.ExecuteNonQuery();
```

```
                    return true;
                }
                else if (etat == "P")
                {
                    string requete = "UPDATE Tag SET etat = 'O' WHERE tag = '" + tag + "'";
                    MySqlCommand cmd = new MySqlCommand(requete, connection);
                    cmd.ExecuteNonQuery();
                    return true;
                }
                else
                {
                    MessageBox.Show("Erreur lors de la mise à jour du tag : état invalide");
                    return false;
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur lors de la mise à jour du tag : " + ex.Message);
                return false;
            }
        }
    }
}
```

Lecteur.cs :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.IO.Ports;

namespace AppCasier
{
    public class Lecteur
    {

        private string m_tag = ""; // Tag lu par le lecteur
        private string m_data = "";
        private string m_port;
        private int m_baud;
        private SerialPort m_SPort;

        public Lecteur(string port, int baud)
        {
            m_port = port;
            m_baud = baud;
```

```
            m_SPort = new SerialPort(m_port, m_baud, Parity.None, 8, StopBits.One);
        }

        public void OpenPort()
        {
            m_SPort.Open();
        }
        public void ClosePort()
        {
            m_SPort.DataReceived -= eventDeLecture; // Détacher l'événement de lecture
            m_SPort.Dispose(); // Libérer les ressources du port série
            m_SPort.Close();
        }

        private void eventDeLecture(object sender, SerialDataReceivedEventArgs e)
        {
            Task.Delay(2000).Wait(); // Attendre 2
            m_data = m_SPort.ReadExisting();

            if (m_data.Length == 14)
            {
                m_tag = ""; // Réinitialiser le tag
                for (int i = 1; i < m_data.Length - 1; i++)
                {
                    m_tag += m_data[i]; // ajouter le caractère au tag
                }
            }
        }

        public string GetTag()
        {
            return m_tag;
        }

        public void SetTag(string tag)
        {
            m_tag = tag;
        }
        public bool lireTag()
        {
            int time = 0;
            while (m_tag == "")
            {
                m_SPort.DataReceived += eventDeLecture;
                Task.Delay(50).Wait(); // Attendre 50ms
                time += 50;
```

```
                if (time >= 5000) // Si le temps d'attente dépasse 5 secondes
                {
                    m_SPort.DataReceived -= eventDeLecture; // Détacher l'événement de lecture
                    return false; // Retourner false si aucun tag n'est lu
                }
            }
            return true; // Retourner true si un tag est lu
        }

        public bool IsConnected()
        {
            try
            {
                m_SPort.Open();
                m_SPort.Close();
                return true;
            }
            catch (Exception ex)
            {
                Console.WriteLine("Erreur de connexion au lecteur : " + ex.Message);
                return false;
            }
        }
    }
}
```

MainForm.cs :

```
using System;
using System.Runtime.CompilerServices;
using System.Windows.Forms;


namespace AppCasier
{

    public partial class MainForm : Form
    {

        private string login = ""; // Variable conservée
        private string role = ""; // Variable conservée
        DatabaseConnection db = new DatabaseConnection(); // Instance de la classe DatabaseConnection
        MenuHamburger menu = new MenuHamburger(); // Instance de la classe Menu

        public MainForm()
        {
```

```
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            OpenConnexion(); // Ouvrir Page de Connexion
            if (login == "") // Si l'utilisateur n'est pas connecté
            {
                // Fermer l'application
                Application.Exit();
            }
            else
            {
                db.OpenConnection(); // Ouvrir la connexion à la base de données
                affichageGeneral();
            }
        }

        // Ouvrir Page de Connexion

        public void affichageGeneral()
        {
            InitializeComponent();
            afficherAffectation(); // Afficher les affectations
            affichageSelection();
            menu.InitializeHamburgerMenu(this); // Initialiser le menu
            menu.SetMainForm(this); // Mettre à jour le formulaire principal
            affichageDatePasser();

        }
        private void OpenConnexion()
        {
            Connexion connexion = new Connexion(this);
            this.Hide();  // Cacher MainForm
            connexion.ShowDialog();  // Afficher Connexion et attendre sa fermeture
            this.Show();  // Rendre MainForm visible après la fermeture de Connexion
        }

        // Accéder aux données partagées
        public string GetLogin()
        {
            return login;
        }

        public void SetLogin(string newData)
        {
            login = newData;
        }
```

```
        public string GetRole()
        {
            return role;
        }

        public void SetRole(string newData)
        {
            role = newData;
        }

        public void afficherAffectation()
        {

            listBoxAffectation.Items.Clear(); // Effacer les affectations

            // Afficher les affectations
            string[] listAffectation = db.listeAffectation();

            for (int i = 0; i < listAffectation.Length; i++)
            {
                listBoxAffectation.Items.Add("Casier n°" + listAffectation[i]);
            }

            affichageSelection();

        }

        public void Maj_Affichage()
        {
            afficherAffectation(); // Rafraîchir les affectations
            affichageSelection(); // Rafraîchir les sélections
        }

        private void clickListeAffectation(object sender, EventArgs e)
        {
            // Gerer si il nèy a pas d'affectation
            if (listBoxAffectation.SelectedItem == null)
            {
                return;
            }

            // Selectionner le numero du casier
            string selectedAffectation = listBoxAffectation.SelectedItem.ToString().Substring(9);
            string[] detailsAffectation = db.detailsAffectation(selectedAffectation);
```

```
            // Afficher les détails de l'affectation
            PageAffichageInterface pageAffichage = new PageAffichageInterface(detailsAffectation,this);
            pageAffichage.ShowDialog();

        }

        private void affichageSelection()
        {
            // Supprimer les éléments de la liste
            cbAffectationNom.Items.Clear();
            cbAffectationCasier.Items.Clear();
            cbAffectationTag.Items.Clear();

            cbAffectationCasier.Text = "";
            cbAffectationNom.Text = "";
            cbAffectationTag.Text = "";

            // Récupérer les détails de l'affectation sélectionnée
            string[] Nom = db.recupNomPrenomVisiteurNonAffecté();
            string[] Casier = db.recupNumCasierNonAffecté();
            string[] Tag = db.recupNumTagNonAffecté();

            // Afficher les détails de l'affectation
            for (int i = 0; i < Nom.Length; i++)
            {
                cbAffectationNom.Items.Add(Nom[i]);
            }

            for (int i = 0; i < Casier.Length; i++)
            {
                cbAffectationCasier.Items.Add(Casier[i]);
            }

            for (int i = 0; i < Tag.Length; i++)
            {
                cbAffectationTag.Items.Add(Tag[i]);
            }

        }

        private void btAffectation_Click(object sender, EventArgs e)
        {

            if (cbAffectationTag.SelectedItem == null || cbAffectationNom.SelectedItem == null || cbAffectationCasier.SelectedItem == null)
```

```
            {
                MessageBox.Show("Veuillez remplir tous les champs !");
                return;
            }
            else
            {
                if (dtpDateDeb.Value >= dtpDateFin.Value)
                {
                    MessageBox.Show("La date de début doit être inférieure à la date de fin !");
                    return;
                }

                else
                {
                    if (dtpDateDeb.Value < DateTime.Now)
                    {
                        // Recuperer les informations de l'affectation
                        string tag = cbAffectationTag.SelectedItem.ToString();
                        // Recuperer le nom du visiteur
                        string[] nomPrenom = cbAffectationNom.SelectedItem.ToString().Split(' ');
                        string nom = nomPrenom[0];
                        string prenom = nomPrenom[1];
                        string casier = cbAffectationCasier.SelectedItem.ToString();
                        string dateDeb = dtpDateDeb.Value.ToString("yyyy-MM-dd");
                        string dateFin = dtpDateFin.Value.ToString("yyyy-MM-dd");

                        // Ajouter l'affectation
                        if (db.ajouterAffectation(tag, nom, prenom, casier, dateDeb, dateFin))
                        {
                            MessageBox.Show("Affectation ajoutée !");
                            // Mettre à jour l'affichage
                            afficherAffectation();
                            affichageSelection();
                        }
                        else
                        {
                            MessageBox.Show("Erreur lors de l'ajout de l'affectation !");
                        }
                    }
        else
```

```
        {
      MessageBox.Show("La date de début doit être celle d'
aujourd'hui");
        }
                }
            }
        }

        private void FermetureMenuHamburger(object sender,
EventArgs e)
        {
            menu.menuOpen = false;
            menu.menuTimer.Start();
        }

        public void FermetureMenu()
        {
            menu.menuOpen = false;
            menu.menuTimer.Start();
        }

        private void pbLogo_Click(object sender, EventArgs e)
        {
            logout logout = new logout(this);
            logout.ShowDialog();
        }

        private void affichageDatePasser()
        {
            // Recuperer les affectations
            string[] affectation = db.affectationDateLimite();

            // Parcourir les affectations
            for (int i = 0; i < affectation.Length; i++)
            {
                // Recuperer les informations de l'affectation
                string[] detailsAffectation = db.
infoAffectation(affectation[i]);

                // Afficher les informations de l'affectation
                verifDate verifDate = new verifDate(
detailsAffectation, this);
                verifDate.ShowDialog();

            }
        }
    }
}
```

MainForm.Designer.cs :

```
using System;
```

```
using System.Windows.Forms;

namespace AppCasier
{
    public partial class MainForm : Form
    {
        /// <summary>
        /// Variable nécessaire au concepteur.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Nettoyage des ressources utilisées.
        /// </summary>
        /// <param name="disposing">true si les ressources managées doivent être supprimées ; sinon, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        private void InitializeComponent()
        {
            System.ComponentModel.ComponentResourceManager resources = new System.ComponentModel.ComponentResourceManager(typeof(MainForm));
            this.cbAffectationTag = new System.Windows.Forms.ComboBox();
            this.cbAffectationNom = new System.Windows.Forms.ComboBox();
            this.cbAffectationCasier = new System.Windows.Forms.ComboBox();
            this.gpbAffectation = new System.Windows.Forms.GroupBox();
            this.label3 = new System.Windows.Forms.Label();
            this.label2 = new System.Windows.Forms.Label();
            this.label1 = new System.Windows.Forms.Label();
            this.lbAffectation = new System.Windows.Forms.Label();
            this.btAffectation = new System.Windows.Forms.Button();
            this.dtpDateFin = new System.Windows.Forms.DateTimePicker();
            this.dtpDateDeb = new System.Windows.Forms.DateTimePicker();
            this.lbListeAffectation = new System.Windows.Forms.
```

```
Label();
            this.listBoxAffectation = new System.Windows.Forms.
ListBox();
            this.pbLogo = new System.Windows.Forms.PictureBox()
;
            this.gpbAffectation.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.
pbLogo)).BeginInit();
            this.SuspendLayout();
            //
            // cbAffectationTag
            //
            this.cbAffectationTag.FormattingEnabled = true;
            this.cbAffectationTag.Location = new System.Drawing
.Point(319, 61);
            this.cbAffectationTag.Name = "cbAffectationTag";
            this.cbAffectationTag.Size = new System.Drawing.
Size(153, 21);
            this.cbAffectationTag.TabIndex = 6;
            //
            // cbAffectationNom
            //
            this.cbAffectationNom.FormattingEnabled = true;
            this.cbAffectationNom.Location = new System.Drawing
.Point(23, 61);
            this.cbAffectationNom.Name = "cbAffectationNom";
            this.cbAffectationNom.Size = new System.Drawing.
Size(121, 21);
            this.cbAffectationNom.TabIndex = 7;
            //
            // cbAffectationCasier
            //
            this.cbAffectationCasier.FormattingEnabled = true;
            this.cbAffectationCasier.Location = new System.
Drawing.Point(172, 61);
            this.cbAffectationCasier.Name = "
cbAffectationCasier";
            this.cbAffectationCasier.Size = new System.Drawing.
Size(121, 21);
            this.cbAffectationCasier.TabIndex = 8;
            //
            // gpbAffectation
            //
            this.gpbAffectation.BackColor = System.Drawing.
Color.DarkGoldenrod;
            this.gpbAffectation.Controls.Add(this.label3);
            this.gpbAffectation.Controls.Add(this.label2);
            this.gpbAffectation.Controls.Add(this.label1);
            this.gpbAffectation.Controls.Add(this.lbAffectation
);
            this.gpbAffectation.Controls.Add(this.btAffectation
```

```
);
            this.gpbAffectation.Controls.Add(this.dtpDateFin);
            this.gpbAffectation.Controls.Add(this.dtpDateDeb);
            this.gpbAffectation.Controls.Add(this.
cbAffectationNom);
            this.gpbAffectation.Controls.Add(this.
cbAffectationCasier);
            this.gpbAffectation.Controls.Add(this.
cbAffectationTag);
            this.gpbAffectation.Location = new System.Drawing.
Point(12, 183);
            this.gpbAffectation.Name = "gpbAffectation";
            this.gpbAffectation.Size = new System.Drawing.Size
(494, 224);
            this.gpbAffectation.TabIndex = 12;
            this.gpbAffectation.TabStop = false;
            this.gpbAffectation.Enter += new System.
EventHandler(this.FermetureMenuHamburger);
            //
            // label3
            //
            this.label3.AutoSize = true;
            this.label3.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.label3.Location = new System.Drawing.Point(20,
 45);
            this.label3.Name = "label3";
            this.label3.Size = new System.Drawing.Size(57, 13);
            this.label3.TabIndex = 14;
            this.label3.Text = "Visiteur :";
            //
            // label2
            //
            this.label2.AutoSize = true;
            this.label2.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.label2.Location = new System.Drawing.Point
(169, 45);
            this.label2.Name = "label2";
            this.label2.Size = new System.Drawing.Size(50, 13);
            this.label2.TabIndex = 13;
            this.label2.Text = "Casier :";
            //
            // label1
            //
            this.label1.AutoSize = true;
            this.label1.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
```

```
            this.label1.Location = new System.Drawing.Point(316, 45);
            this.label1.Name = "label1";
            this.label1.Size = new System.Drawing.Size(37, 13);
            this.label1.TabIndex = 12;
            this.label1.Text = "Tag :";
            //
            // lbAffectation
            //
            this.lbAffectation.AutoSize = true;
            this.lbAffectation.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbAffectation.Location = new System.Drawing.Point(11, 16);
            this.lbAffectation.Name = "lbAffectation";
            this.lbAffectation.Size = new System.Drawing.Size(318, 13);
            this.lbAffectation.TabIndex = 11;
            this.lbAffectation.Text = "Selectionnez les éléments à ajouter dans l\'affectation :";
            this.lbAffectation.Click += new System.EventHandler(this.FermetureMenuHamburger);
            //
            // btAffectation
            //
            this.btAffectation.Location = new System.Drawing.Point(213, 177);
            this.btAffectation.Name = "btAffectation";
            this.btAffectation.Size = new System.Drawing.Size(80, 27);
            this.btAffectation.TabIndex = 11;
            this.btAffectation.Text = "Affectation";
            this.btAffectation.UseVisualStyleBackColor = true;
            this.btAffectation.Click += new System.EventHandler(this.btAffectation_Click);
            //
            // dtpDateFin
            //
            this.dtpDateFin.Location = new System.Drawing.Point(288, 120);
            this.dtpDateFin.Name = "dtpDateFin";
            this.dtpDateFin.Size = new System.Drawing.Size(184, 20);
            this.dtpDateFin.TabIndex = 10;
            //
            // dtpDateDeb
            //
            this.dtpDateDeb.Location = new System.Drawing.Point(26, 120);
            this.dtpDateDeb.Name = "dtpDateDeb";
```

```
            this.dtpDateDeb.Size = new System.Drawing.Size(193, 20);
            this.dtpDateDeb.TabIndex = 9;
            //
            // lbListeAffectation
            //
            this.lbListeAffectation.AutoSize = true;
            this.lbListeAffectation.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbListeAffectation.Location = new System.Drawing.Point(21, 96);
            this.lbListeAffectation.Name = "lbListeAffectation";
            this.lbListeAffectation.Size = new System.Drawing.Size(273, 13);
            this.lbListeAffectation.TabIndex = 10;
            this.lbListeAffectation.Text = "Liste des casiers occupé par une Affecttation :";
            this.lbListeAffectation.Click += new System.EventHandler(this.FermetureMenuHamburger);
            //
            // listBoxAffectation
            //
            this.listBoxAffectation.FormattingEnabled = true;
            this.listBoxAffectation.Location = new System.Drawing.Point(300, 47);
            this.listBoxAffectation.Name = "listBoxAffectation";
            this.listBoxAffectation.Size = new System.Drawing.Size(119, 121);
            this.listBoxAffectation.TabIndex = 9;
            this.listBoxAffectation.Click += new System.EventHandler(this.clickListeAffectation);
            //
            // pbLogo
            //
            this.pbLogo.BorderStyle = System.Windows.Forms.BorderStyle.FixedSingle;
            this.pbLogo.Image = ((System.Drawing.Image)(resources.GetObject("pbLogo.Image")));
            this.pbLogo.Location = new System.Drawing.Point(461, 2);
            this.pbLogo.Name = "pbLogo";
            this.pbLogo.Size = new System.Drawing.Size(55, 58);
            this.pbLogo.SizeMode = System.Windows.Forms.PictureBoxSizeMode.StretchImage;
            this.pbLogo.TabIndex = 15;
            this.pbLogo.TabStop = false;
            this.pbLogo.Click += new System.EventHandler(this.pbLogo_Click);
```

```
            //
            // MainForm
            //
            this.BackColor = System.Drawing.SystemColors.Info;
            this.ClientSize = new System.Drawing.Size(518, 416)
;
            this.Controls.Add(this.pbLogo);
            this.Controls.Add(this.lbListeAffectation);
            this.Controls.Add(this.listBoxAffectation);
            this.Controls.Add(this.gpbAffectation);
            this.Name = "MainForm";
            this.Text = "AppCasier";
            this.Click += new System.EventHandler(this.
FermetureMenuHamburger);
            this.gpbAffectation.ResumeLayout(false);
            this.gpbAffectation.PerformLayout();
            ((System.ComponentModel.ISupportInitialize)(this.
pbLogo)).EndInit();
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        private ComboBox cbAffectationTag;
        private ComboBox cbAffectationNom;
        private ComboBox cbAffectationCasier;
        private GroupBox gpbAffectation;
        private Label lbAffectation;
        private Button btAffectation;
        private DateTimePicker dtpDateFin;
        private DateTimePicker dtpDateDeb;
        private Label label3;
        private Label label2;
        private Label label1;
        private Label lbListeAffectation;
        private ListBox listBoxAffectation;
        private PictureBox pbLogo;
    }
}
```

MenuHamburger.cs :

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
```

```
    internal class MenuHamburger
    {
        public bool menuOpen = false;
        public Timer menuTimer;
        private Panel menuPanel;

        string[] menuItemsAdmin = { "Ajouter Utilisateur", "
Ajouter Visiteur", "Ajout Tag", "Supprimer Utilisateur", "
Supprimer Visiteur", "Supprimer Tag","Changement d'état" };
        string[] menuItemsUser = { "Ajouter Visiteur", "Ajout
Tag", "Supprimer Visiteur", "Supprimer Tag", "
PageChangementEtat" };

        MainForm mainform;
        Lecteur lecteur = new Lecteur("COM8", 9600); // Port et
 vitesse de communication0

        public MenuHamburger()
        {
            // Test pour voir si le lecteur est connecté
            if (!lecteur.IsConnected())
            {
                MessageBox.Show("Le lecteur de carte n'est pas
connecté");
                return;
            }
        }

        public void InitializeHamburgerMenu(MainForm form)
        {
            // Création du panel latéral (menu)
            menuPanel = new Panel
            {
                Size = new Size(200, form.Height),
                BackColor = Color.FromArgb(50, 50, 50),
                Location = new Point(-200, 0) // Caché au dé
part
            };
            form.Controls.Add(menuPanel);

            // Bouton Hamburger ([menu])
            Button btnHamburger = new Button
            {
                Text = "[menu]",
                Font = new Font("Arial", 14, FontStyle.Bold),
                Size = new Size(50, 40),
                Location = new Point(10, 10),
                BackColor = Color.Gray,
                ForeColor = Color.White,
                FlatStyle = FlatStyle.Flat
            };
```

```
            btnHamburger.Click += (s, e) => ToggleMenu();
            form.Controls.Add(btnHamburger);

            // Boutons du menu
            int yOffset = 10;

            string[] menuItems;

            if (form.GetRole() == "admin")
            {
                menuItems = menuItemsAdmin;
            }
            else
            {
                menuItems = menuItemsUser;
            }

            foreach (string item in menuItems)
            {
                Button menuButton = new Button
                {
                    Text = item,
                    Size = new Size(180, 40),
                    Location = new Point(10, yOffset),
                    BackColor = Color.DarkGray,
                    ForeColor = Color.White,
                    FlatStyle = FlatStyle.Flat
                };

                menuButton.Click += (s, e) => MenuButtonClick(item);
                menuPanel.Controls.Add(menuButton);
                yOffset += 50;
            }

            // Timer pour l'animation
            menuTimer = new Timer();
            menuTimer.Interval = 10;
            menuTimer.Tick += AnimateMenu;
        }
        private void ToggleMenu()
        {
            menuPanel.BringToFront();
            menuOpen = !menuOpen;
            menuTimer.Start();
        }

        private void AnimateMenu(object sender, EventArgs e)
        {
            int targetX = menuOpen ? 0 : -200;
            int step = menuOpen ? 20 : -20;
```

```
            if ((menuOpen && menuPanel.Left < targetX) || (!menuOpen && menuPanel.Left > targetX))
            {
                menuPanel.Left += step;
            }
            else
            {
                menuPanel.Left = targetX;
                menuTimer.Stop();
            }
        }

        private void MenuButtonClick(string menuItem)
        {

            if (menuItem == "Ajouter Utilisateur")
            {
                // Ouvrir la page d'ajout d'utilisateur
                PageAjoutUtilisateur page = new PageAjoutUtilisateur();
                page.ShowDialog();
            }
            else if (menuItem == "Ajouter Visiteur")
            {
                // Ouvrir la page d'ajout de visiteur
                PageAjoutVisiteur page = new PageAjoutVisiteur(mainform);
                page.ShowDialog();
            }
            else if (menuItem == "Ajout Tag")
            {
                // Ouvrir la page d'ajout de tag
                // Vérifier si le lecteur est connecté
                if (!lecteur.IsConnected())
                {
                    MessageBox.Show("Le lecteur de carte n'est pas connecté");
                    return;
                }
                else
                {
                    PageAjoutTag page = new PageAjoutTag(mainform, lecteur);
                    page.ShowDialog();
                }
            }
            else if (menuItem == "Supprimer Utilisateur")
            {
                // Ouvrir la page de suppression d'utilisateur
                pageSupprUtilisateur page = new
```

```
pageSupprUtilisateur();
                page.ShowDialog();
            }
            else if (menuItem == "Supprimer Visiteur")
            {
                // Ouvrir la page de suppression de visiteur
                pageSupprVisiteur page = new pageSupprVisiteur(
mainform);
                page.ShowDialog();
            }
            else if (menuItem == "Supprimer Tag")
            {
                // Ouvrir la page de suppression de tag
                pageSupprTag page = new pageSupprTag(mainform);
                page.ShowDialog();
            }
            else if (menuItem == "Changement d'état")
            {
                // Ouvrir la page de changement d'état
                PageChangementEtat page = new
PageChangementEtat(mainform);
                page.ShowDialog();
            }
            else if (menuItem == "Déconnexion")
            {
                // Ouvrir la page de déconnexion
                logout logout = new logout(mainform);
                logout.ShowDialog();
            }
            else
            {
                // Ferme automatiquement le menu après un clic
                menuOpen = false;
                menuTimer.Start();
            }
        }

        public void SetMainForm(MainForm form)
        {
            mainform = form;
        }
    }
}
```

PageAjoutTag.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
```

```
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class PageAjoutTag : Form
    {
        DatabaseConnection db = new DatabaseConnection();
        Lecteur lecteur;
        MainForm mainform;
        public PageAjoutTag(MainForm form, Lecteur m_lecteur)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            db.OpenConnection();
            lecteur = m_lecteur; // Récupérer l’instance de Lecteur
            mainform = form; // Récupérer l’instance de MainForm

        }

        private void btAjoutTag_Click(object sender, EventArgs e)
        {
            if (tbTag.Text == "")
            {
                MessageBox.Show("Veuillez entrer un tag valide.");
                return;
            }
            else
            {

                if (db.verifTag(tbTag.Text))
                {
                    MessageBox.Show("Le tag existe déjà.");
                    return;
                }
                else
                {
                    // Ajouter le tag à la base de données
                    if (db.ajouterTag(tbTag.Text))
                    {
                        MessageBox.Show("Tag ajouté avec succès.");
                        this.Close(); // Fermer la fenêtre après l’ajout
                        mainform.Maj_Affichage();
```

```
                    }
                    else
                    {
                        MessageBox.Show("Erreur lors de l'ajout du tag.");
                    }
                }
            }
        }

        private void btSelectTag_Click(object sender, EventArgs e)
        {
            lecteur.SetTag(""); // Réinitialiser le tag
            lectureTag lectureTag = new lectureTag(lecteur);
            lectureTag.ShowDialog(); // Afficher la fenêtre de lecture de tag
            if (lecteur.GetTag() != "")
            {
                tbTag.Text = lecteur.GetTag(); // Récupérer le tag lu
            }
        }
    }
}
```

PageAjoutTag.Designer.cs :

```
namespace AppCasier
{
    partial class PageAjoutTag
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

```

```
        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.lbTitre = new System.Windows.Forms.Label();
            this.btSelectTag = new System.Windows.Forms.Button();
            this.tbTag = new System.Windows.Forms.TextBox();
            this.btAjoutTag = new System.Windows.Forms.Button();
            this.SuspendLayout();
            //
            // lbTitre
            //
            this.lbTitre.AutoSize = true;
            this.lbTitre.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbTitre.Location = new System.Drawing.Point(12, 14);
            this.lbTitre.Name = "lbTitre";
            this.lbTitre.Size = new System.Drawing.Size(144, 13);
            this.lbTitre.TabIndex = 0;
            this.lbTitre.Text = "Veuillez choisir un tag : ";
            //
            // btSelectTag
            //
            this.btSelectTag.Location = new System.Drawing.Point(190, 9);
            this.btSelectTag.Name = "btSelectTag";
            this.btSelectTag.Size = new System.Drawing.Size(75, 23);
            this.btSelectTag.TabIndex = 1;
            this.btSelectTag.Text = "Selectionner";
            this.btSelectTag.UseVisualStyleBackColor = true;
            this.btSelectTag.Click += new System.EventHandler(this.btSelectTag_Click);
            //
            // tbTag
            //
            this.tbTag.Location = new System.Drawing.Point(15, 54);
            this.tbTag.Name = "tbTag";
            this.tbTag.ReadOnly = true;
            this.tbTag.Size = new System.Drawing.Size(131, 20);
```

```
            this.tbTag.TabIndex = 2;
            //
            // btAjoutTag
            //
            this.btAjoutTag.Location = new System.Drawing.Point(190, 52);
            this.btAjoutTag.Name = "btAjoutTag";
            this.btAjoutTag.Size = new System.Drawing.Size(75, 23);
            this.btAjoutTag.TabIndex = 3;
            this.btAjoutTag.Text = "Ajouter";
            this.btAjoutTag.UseVisualStyleBackColor = true;
            this.btAjoutTag.Click += new System.EventHandler(this.btAjoutTag_Click);
            //
            // PageAjoutTag
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(291, 102);
            this.Controls.Add(this.btAjoutTag);
            this.Controls.Add(this.tbTag);
            this.Controls.Add(this.btSelectTag);
            this.Controls.Add(this.lbTitre);
            this.Name = "PageAjoutTag";
            this.Text = "PageAjoutTag";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label lbTitre;
        private System.Windows.Forms.Button btSelectTag;
        private System.Windows.Forms.TextBox tbTag;
        private System.Windows.Forms.Button btAjoutTag;
    }
}
```

PageAjoutUtilisateur.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
```

```
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class PageAjoutUtilisateur : Form
    {
        DatabaseConnection db = new DatabaseConnection();

        public PageAjoutUtilisateur()
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            db.OpenConnection();
        }

        private void btAccepter_Click(object sender, EventArgs e)
        {
            if ( tbLogin.Text == "" || tbPassword.Text == "" || cbRole.Text == "" )
            {
                MessageBox.Show("Veuillez remplir tous les champs");
            }
            else if (db.verifyUtilisateur(tbLogin.Text))
            {
                MessageBox.Show("L'Utilisateur existe déjà !");
                return;
            }
            else
            {

                if (db.ajouterUtilisateur(tbLogin.Text, tbPassword.Text, cbRole.Text))
                {
                    MessageBox.Show("Utilisateur ajouté avec succès");
                    this.Close();
                }
                else
                {
                    MessageBox.Show("Cet utilisateur existe déja");
                }
            }
        }
    }
}
```

PageAjoutUtilisateur.Designer.cs :

```
namespace AppCasier
{
    partial class PageAjoutUtilisateur
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.btAccepter = new System.Windows.Forms.Button();
            this.lbTitreAjoutUtilisateur = new System.Windows.Forms.Label();
            this.lbLogin = new System.Windows.Forms.Label();
            this.lbPassword = new System.Windows.Forms.Label();
            this.lbRole = new System.Windows.Forms.Label();
            this.tbLogin = new System.Windows.Forms.TextBox();
            this.tbPassword = new System.Windows.Forms.TextBox();
            this.cbRole = new System.Windows.Forms.ComboBox();
            this.SuspendLayout();
            //
            // btAccepter
            //
            this.btAccepter.Location = new System.Drawing.Point(85, 204);
```

```
            this.btAccepter.Name = "btAccepter";
            this.btAccepter.Size = new System.Drawing.Size(75, 23);
            this.btAccepter.TabIndex = 0;
            this.btAccepter.Text = "Confirmer";
            this.btAccepter.UseVisualStyleBackColor = true;
            this.btAccepter.Click += new System.EventHandler(this.btAccepter_Click);
            //
            // lbTitreAjoutUtilisateur
            //
            this.lbTitreAjoutUtilisateur.AutoSize = true;
            this.lbTitreAjoutUtilisateur.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbTitreAjoutUtilisateur.Location = new System.Drawing.Point(13, 13);
            this.lbTitreAjoutUtilisateur.Name = "lbTitreAjoutUtilisateur";
            this.lbTitreAjoutUtilisateur.Size = new System.Drawing.Size(136, 13);
            this.lbTitreAjoutUtilisateur.TabIndex = 1;
            this.lbTitreAjoutUtilisateur.Text = "Ajoutez un Utilisateur :";
            //
            // lbLogin
            //
            this.lbLogin.AutoSize = true;
            this.lbLogin.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbLogin.Location = new System.Drawing.Point(13, 64);
            this.lbLogin.Name = "lbLogin";
            this.lbLogin.Size = new System.Drawing.Size(46, 13);
            this.lbLogin.TabIndex = 2;
            this.lbLogin.Text = "Login :";
            //
            // lbPassword
            //
            this.lbPassword.AutoSize = true;
            this.lbPassword.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbPassword.Location = new System.Drawing.Point(13, 104);
            this.lbPassword.Name = "lbPassword";
            this.lbPassword.Size = new System.Drawing.Size(91, 13);
```

```
            this.lbPassword.TabIndex = 3;
            this.lbPassword.Text = "Mot de passe :";
            //
            // lbRole
            //
            this.lbRole.AutoSize = true;
            this.lbRole.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbRole.Location = new System.Drawing.Point(13,
 149);
            this.lbRole.Name = "lbRole";
            this.lbRole.Size = new System.Drawing.Size(41, 13);
            this.lbRole.TabIndex = 4;
            this.lbRole.Text = "Role :";
            //
            // tbLogin
            //
            this.tbLogin.Location = new System.Drawing.Point
(110, 64);
            this.tbLogin.Name = "tbLogin";
            this.tbLogin.Size = new System.Drawing.Size(115,
20);
            this.tbLogin.TabIndex = 5;
            //
            // tbPassword
            //
            this.tbPassword.Location = new System.Drawing.Point
(110, 104);
            this.tbPassword.Name = "tbPassword";
            this.tbPassword.Size = new System.Drawing.Size(115,
 20);
            this.tbPassword.TabIndex = 6;
            //
            // cbRole
            //
            this.cbRole.FormattingEnabled = true;
            this.cbRole.Items.AddRange(new object[] {
            "admin",
            "user"});
            this.cbRole.Location = new System.Drawing.Point
(110, 146);
            this.cbRole.Name = "cbRole";
            this.cbRole.DropDownStyle = System.Windows.Forms.
ComboBoxStyle.DropDownList;
            this.cbRole.Size = new System.Drawing.Size(115, 21)
;
            this.cbRole.TabIndex = 7;
            //
            // PageAjoutUtilisateur
            //
```

```
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(248, 236);
            this.Controls.Add(this.cbRole);
            this.Controls.Add(this.tbPassword);
            this.Controls.Add(this.tbLogin);
            this.Controls.Add(this.lbRole);
            this.Controls.Add(this.lbPassword);
            this.Controls.Add(this.lbLogin);
            this.Controls.Add(this.lbTitreAjoutUtilisateur);
            this.Controls.Add(this.btAccepter);
            this.Name = "PageAjoutUtilisateur";
            this.Text = "Ajout d\'utilisateur";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Button btAccepter;
        private System.Windows.Forms.Label lbTitreAjoutUtilisateur;
        private System.Windows.Forms.Label lbPassword;
        private System.Windows.Forms.Label lbRole;
        private System.Windows.Forms.TextBox tbLogin;
        private System.Windows.Forms.TextBox tbPassword;
        private System.Windows.Forms.Label lbLogin;
        private System.Windows.Forms.ComboBox cbRole;
    }
}
```

PageAjoutVisiteur.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Numerics;
using System.Text;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class PageAjoutVisiteur : Form
```

```
    {
        MainForm mainform;
        DatabaseConnection db = new DatabaseConnection();
        PlaqueInfo pays ;
        public PageAjoutVisiteur(MainForm form)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            mainform = form;
            InitializeComponent();
            afficherPays(); // Afficher les pays
            db.OpenConnection();
        }

        private void btCreerVisiteur_Click(object sender, EventArgs e)
        {
            // Verifier si les champs sont remplis
            if (tbNom.Text == "" || tbPrenom.Text == "" || tbCompagnie.Text == "" || tbPlaque.Text == "" || cbPays.Text == "")
            {
                MessageBox.Show("Veuillez remplir tous les champs comme demandé !");
            }
            else
            {
                // Verifier si la plaque est valide encomparant avec le regex
                if(!Regex.IsMatch(tbPlaque.Text, pays.Regex))
                {
                    MessageBox.Show("La plaque doit être sous la forme demandée !");
                    return;
                }
                else if (db.verifyVisiteur(tbNom.Text, tbPrenom.Text))
                {
                    MessageBox.Show("Le visiteur existe déjà !");
                    return;
                }
                else
                {


                    if (db.ajouterVisiteur(tbNom.Text, tbPrenom.Text, tbPlaque.Text, tbCompagnie.Text,cbPays.Text))
                    {
                        MessageBox.Show("Visiteur ajouté avec succès !");
```

```
                        this.Close();
                        // Mettre à jour l'affichage
                        mainform.Maj_Affichage();
                    }
                    else
                    {
                        MessageBox.Show("Erreur lors de l'ajout
 du visiteur !");
                        return;
                    }
                }
            }
        }

        private void afficherPays()
        {
            // Ajouter les pays à la liste
            cbPays.Items.Add("France");
            cbPays.Items.Add("Allemagne");
            cbPays.Items.Add("Belgique");
            cbPays.Items.Add("Suisse");
            cbPays.Items.Add("Luxembourg");
            cbPays.Items.Add("Espagne");
            cbPays.Items.Add("Italie");
            cbPays.Items.Add("Portugal");
            cbPays.Items.Add("Royaume-Uni");
            cbPays.Items.Add("Irlande");
            cbPays.Items.Add("Pays-Bas");
            cbPays.Items.Add("Danemark");
            cbPays.Items.Add("Suède");
            cbPays.Items.Add("Finlande");
            cbPays.Items.Add("Autriche");
            cbPays.Items.Add("République Tchèque");
            cbPays.Items.Add("Slovénie");
            cbPays.Items.Add("Slovaquie");
            cbPays.Items.Add("Hongrie");
            cbPays.Items.Add("Roumanie");
            cbPays.Items.Add("Bulgarie");
            cbPays.Items.Add("Croatie");
            cbPays.Items.Add("Estonie");
            cbPays.Items.Add("Lettonie");
            cbPays.Items.Add("Lituanie");
            cbPays.Items.Add("Chypre");
            cbPays.Items.Add("Malte");
            cbPays.Items.Add("Autre");
        }

        private void ChangementPays(object sender, EventArgs e)
        {
            // Verifier le pays selectionné
            switch (cbPays.Text)
```

```
            {
                case "France":
                    pbPays.Image = Properties.Resources.france;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.France].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.France].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.France].Exemple;
                    break;
                case "Allemagne":
                    pbPays.Image = Properties.Resources.allemagne;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Allemagne].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Allemagne].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Allemagne].Exemple;
                    break;
                case "Belgique":
                    pbPays.Image = Properties.Resources.belgique;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Belgique].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Belgique].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Belgique].Exemple;
                    break;
                case "Suisse":
                    pbPays.Image = Properties.Resources.suisse;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Suisse].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Suisse].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Suisse].Exemple;
                    break;
                case "Luxembourg":
                    pbPays.Image = Properties.Resources.luxembourg;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Luxembourg].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Luxembourg].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Luxembourg].Exemple;
                    break;
                case "Espagne":
                    pbPays.Image = Properties.Resources.espagne;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Espagne].Regex, PlaquesEuropeennes.
```

```
InfosPlaques[PaysEuropeen.Espagne].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Espagne].Exemple;
                    break;
                case "Italie":
                    pbPays.Image = Properties.Resources.italie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Italie].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Italie].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Italie].Exemple;
                    break;
                case "Portugal":
                    pbPays.Image = Properties.Resources.
portugal;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Portugal].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Portugal].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Portugal].Exemple;
                    MessageBox.Show(cbPays.Text);
                    break;
                case "Royaume-Uni":
                    pbPays.Image = Properties.Resources.uk;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.RoyaumeUni].Regex, PlaquesEuropeennes
.InfosPlaques[PaysEuropeen.RoyaumeUni].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.RoyaumeUni].
Exemple;
                    break;
                case "Irlande":
                    pbPays.Image = Properties.Resources.irlande
;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Irlande].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Irlande].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Irlande].Exemple;
                    break;
                case "Pays-Bas":
                    pbPays.Image = Properties.Resources.paysbas
;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.PaysBas].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.PaysBas].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.PaysBas].Exemple;
                    break;
                case "Danemark":
                    pbPays.Image = Properties.Resources.
```

```
danemark;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Danemark].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Danemark].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Danemark].Exemple;
                    break;
                case "Suède":
                    pbPays.Image = Properties.Resources.suede;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Suede].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Suede].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Suede].Exemple;
                    break;
                case "Finlande":
                    pbPays.Image = Properties.Resources.
finlande;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Finlande].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Finlande].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Finlande].Exemple;
                    break;
                case "Autriche":
                    pbPays.Image = Properties.Resources.
autriche;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Autriche].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Autriche].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Autriche].Exemple;
                    break;
                case "République Tchèque":
                    pbPays.Image = Properties.Resources.
tchequie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Tchequie].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Tchequie].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Tchequie].Exemple;
                    break;
                case "Slovénie":
                    pbPays.Image = Properties.Resources.
slovenie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Slovenie].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Slovenie].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Slovenie].Exemple;
                    break;
```

```
                case "Slovaquie":
                    pbPays.Image = Properties.Resources.slovaquie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Slovaquie].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Slovaquie].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Slovaquie].Exemple;
                    break;
                case "Hongrie":
                    pbPays.Image = Properties.Resources.hongrie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Hongrie].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Hongrie].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Hongrie].Exemple;
                    break;
                case "Roumanie":
                    pbPays.Image = Properties.Resources.roumanie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Roumanie].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Roumanie].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Roumanie].Exemple;
                    break;
                case "Bulgarie":
                    pbPays.Image = Properties.Resources.bulgarie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Bulgarie].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Bulgarie].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Bulgarie].Exemple;
                    break;
                case "Croatie":
                    pbPays.Image = Properties.Resources.croatie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Croatie].Regex, PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Croatie].Exemple);
                    lbPlaqueEx.Text = "Ex : " + PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Croatie].Exemple;
                    break;
                case "Estonie":
                    pbPays.Image = Properties.Resources.estonie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Estonie].Regex, PlaquesEuropeennes.
```

```
InfosPlaques[PaysEuropeen.Estonie].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Estonie].Exemple;
                    break;
                case "Lettonie":
                    pbPays.Image = Properties.Resources.
lettonie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Lettonie].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Lettonie].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Lettonie].Exemple;
                    break;
                case "Lituanie":
                    pbPays.Image = Properties.Resources.
lituanie;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Lituanie].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Lituanie].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Lituanie].Exemple;
                    break;
                case "Chypre":
                    pbPays.Image = Properties.Resources.chypre;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Chypre].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Chypre].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Chypre].Exemple;
                    break;
                case "Malte":
                    pbPays.Image = Properties.Resources.malte;
                    pays = new PlaqueInfo(PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Malte].Regex, PlaquesEuropeennes.
InfosPlaques[PaysEuropeen.Malte].Exemple);
                    lbPlaqueEx.Text = "Ex : " +
PlaquesEuropeennes.InfosPlaques[PaysEuropeen.Malte].Exemple;
                    break;
                case "Autre":
                    pbPays.Image = Properties.Resources.autre;
                    lbPlaqueEx.Text = "";
                    break;

            }
        }
    }
}
```

PageAjoutVisiteur.Designer.cs :

```
namespace AppCasier
{
```

```
    partial class PageAjoutVisiteur
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base. Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            System.ComponentModel.ComponentResourceManager resources = new System.ComponentModel.ComponentResourceManager(typeof(PageAjoutVisiteur));
            this.lbTitre = new System.Windows.Forms.Label();
            this.lbPrenom = new System.Windows.Forms.Label();
            this.lbPlaque = new System.Windows.Forms.Label();
            this.lbCompagnie = new System.Windows.Forms.Label();
            this.lbNom = new System.Windows.Forms.Label();
            this.tbNom = new System.Windows.Forms.TextBox();
            this.tbPlaque = new System.Windows.Forms.TextBox();
            this.tbCompagnie = new System.Windows.Forms.TextBox();
            this.tbPrenom = new System.Windows.Forms.TextBox();
            this.lbPlaqueEx = new System.Windows.Forms.Label();
            this.btCreerVisiteur = new System.Windows.Forms.Button();
            this.lbPays = new System.Windows.Forms.Label();
            this.cbPays = new System.Windows.Forms.ComboBox();
            this.pbPays = new System.Windows.Forms.PictureBox()
```

```
;
            ((System.ComponentModel.ISupportInitialize)(this.
pbPays)).BeginInit();
            this.SuspendLayout();
            //
            // lbTitre
            //
            this.lbTitre.AutoSize = true;
            this.lbTitre.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbTitre.Location = new System.Drawing.Point
(12, 9);
            this.lbTitre.Name = "lbTitre";
            this.lbTitre.Size = new System.Drawing.Size(290,
13);
            this.lbTitre.TabIndex = 0;
            this.lbTitre.Text = "Veuillez rentrer les
informations correspondantes :";
            //
            // lbPrenom
            //
            this.lbPrenom.AutoSize = true;
            this.lbPrenom.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbPrenom.Location = new System.Drawing.Point
(12, 103);
            this.lbPrenom.Name = "lbPrenom";
            this.lbPrenom.Size = new System.Drawing.Size(57,
13);
            this.lbPrenom.TabIndex = 1;
            this.lbPrenom.Text = "Prenom :";
            //
            // lbPlaque
            //
            this.lbPlaque.AutoSize = true;
            this.lbPlaque.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbPlaque.Location = new System.Drawing.Point
(12, 231);
            this.lbPlaque.Name = "lbPlaque";
            this.lbPlaque.Size = new System.Drawing.Size(54,
13);
            this.lbPlaque.TabIndex = 3;
            this.lbPlaque.Text = "Plaque :";
            //
            // lbCompagnie
            //
            this.lbCompagnie.AutoSize = true;
```

```
            this.lbCompagnie.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbCompagnie.Location = new System.Drawing.Point(12, 143);
            this.lbCompagnie.Name = "lbCompagnie";
            this.lbCompagnie.Size = new System.Drawing.Size(77, 13);
            this.lbCompagnie.TabIndex = 4;
            this.lbCompagnie.Text = "Compagnie :";
            //
            // lbNom
            //
            this.lbNom.AutoSize = true;
            this.lbNom.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbNom.Location = new System.Drawing.Point(12, 63);
            this.lbNom.Name = "lbNom";
            this.lbNom.Size = new System.Drawing.Size(40, 13);
            this.lbNom.TabIndex = 5;
            this.lbNom.Text = "Nom :";
            //
            // tbNom
            //
            this.tbNom.Location = new System.Drawing.Point(102, 60);
            this.tbNom.Name = "tbNom";
            this.tbNom.Size = new System.Drawing.Size(119, 20);
            this.tbNom.TabIndex = 6;
            //
            // tbPlaque
            //
            this.tbPlaque.Location = new System.Drawing.Point(102, 228);
            this.tbPlaque.Name = "tbPlaque";
            this.tbPlaque.Size = new System.Drawing.Size(119, 20);
            this.tbPlaque.TabIndex = 7;
            //
            // tbCompagnie
            //
            this.tbCompagnie.Location = new System.Drawing.Point(102, 143);
            this.tbCompagnie.Name = "tbCompagnie";
            this.tbCompagnie.Size = new System.Drawing.Size(119, 20);
            this.tbCompagnie.TabIndex = 8;
            //
            // tbPrenom
```

```
            //
            this.tbPrenom.Location = new System.Drawing.Point(102, 103);
            this.tbPrenom.Name = "tbPrenom";
            this.tbPrenom.Size = new System.Drawing.Size(119, 20);
            this.tbPrenom.TabIndex = 9;
            //
            // lbPlaqueEx
            //
            this.lbPlaqueEx.AutoSize = true;
            this.lbPlaqueEx.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbPlaqueEx.Location = new System.Drawing.Point(110, 251);
            this.lbPlaqueEx.Name = "lbPlaqueEx";
            this.lbPlaqueEx.Size = new System.Drawing.Size(25, 13);
            this.lbPlaqueEx.TabIndex = 10;
            this.lbPlaqueEx.Text = "Ex :";
            //
            // btCreerVisiteur
            //
            this.btCreerVisiteur.Location = new System.Drawing.Point(113, 298);
            this.btCreerVisiteur.Name = "btCreerVisiteur";
            this.btCreerVisiteur.Size = new System.Drawing.Size(75, 23);
            this.btCreerVisiteur.TabIndex = 11;
            this.btCreerVisiteur.Text = "Confirmer";
            this.btCreerVisiteur.UseVisualStyleBackColor = true;
            this.btCreerVisiteur.Click += new System.EventHandler(this.btCreerVisiteur_Click);
            //
            // lbPays
            //
            this.lbPays.AutoSize = true;
            this.lbPays.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbPays.Location = new System.Drawing.Point(12, 187);
            this.lbPays.Name = "lbPays";
            this.lbPays.Size = new System.Drawing.Size(42, 13);
            this.lbPays.TabIndex = 12;
            this.lbPays.Text = "Pays :";
            //
            // cbPays
            //
```

```
            this.cbPays.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.cbPays.FormattingEnabled = true;
            this.cbPays.Location = new System.Drawing.Point(102, 187);
            this.cbPays.Name = "cbPays";
            this.cbPays.Size = new System.Drawing.Size(119, 21);
            this.cbPays.TabIndex = 13;
            this.cbPays.SelectedIndexChanged += new System.EventHandler(this.ChangementPays);
            //
            // pbPays
            //
            this.pbPays.Image = ((System.Drawing.Image)(resources.GetObject("pbPays.Image")));
            this.pbPays.Location = new System.Drawing.Point(236, 187);
            this.pbPays.Name = "pbPays";
            this.pbPays.Size = new System.Drawing.Size(28, 21);
            this.pbPays.SizeMode = System.Windows.Forms.PictureBoxSizeMode.StretchImage;
            this.pbPays.TabIndex = 14;
            this.pbPays.TabStop = false;
            //
            // PageAjoutVisiteur
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(313, 334);
            this.Controls.Add(this.pbPays);
            this.Controls.Add(this.cbPays);
            this.Controls.Add(this.lbPays);
            this.Controls.Add(this.btCreerVisiteur);
            this.Controls.Add(this.lbPlaqueEx);
            this.Controls.Add(this.tbPrenom);
            this.Controls.Add(this.tbCompagnie);
            this.Controls.Add(this.tbPlaque);
            this.Controls.Add(this.tbNom);
            this.Controls.Add(this.lbNom);
            this.Controls.Add(this.lbCompagnie);
            this.Controls.Add(this.lbPlaque);
            this.Controls.Add(this.lbPrenom);
            this.Controls.Add(this.lbTitre);
            this.Name = "PageAjoutVisiteur";
            this.Text = "PageAjoutVisiteur";
            ((System.ComponentModel.ISupportInitialize)(this.pbPays)).EndInit();
```

```
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label lbTitre;
        private System.Windows.Forms.Label lbPrenom;
        private System.Windows.Forms.Label lbPlaque;
        private System.Windows.Forms.Label lbCompagnie;
        private System.Windows.Forms.Label lbNom;
        private System.Windows.Forms.TextBox tbNom;
        private System.Windows.Forms.TextBox tbPlaque;
        private System.Windows.Forms.TextBox tbCompagnie;
        private System.Windows.Forms.TextBox tbPrenom;
        private System.Windows.Forms.Label lbPlaqueEx;
        private System.Windows.Forms.Button btCreerVisiteur;
        private System.Windows.Forms.Label lbPays;
        private System.Windows.Forms.ComboBox cbPays;
        private System.Windows.Forms.PictureBox pbPays;
    }
}
```

PageChangementEtat.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class PageChangementEtat : Form
    {
        DatabaseConnection db = new DatabaseConnection();
        MainForm mainform;
        public PageChangementEtat(MainForm form)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            db.OpenConnection();
            mainform = form;
            affichageTag();

        }
```

```

        private void affichageTag()
        {

            string[] liste = db.recupTagPO();

            for (int i = 0; i < liste.Length; i++)
            {
                cbTag.Items.Add(liste[i]);
            }
        }

        private void cbTag_SelectedIndexChanged(object sender,
EventArgs e)
        {
            lbEtat.Text = "";
            lbEtat.Text += db.getEtatTag(cbTag.Text);
        }

        private void btChange_Click(object sender, EventArgs e)
        {
            db.updateTag(cbTag.Text, lbEtat.Text);
            MessageBox.Show("Changement d'état effectué !");
            lbEtat.Text = "";
            lbEtat.Text += db.getEtatTag(cbTag.Text);
        }


    }
}
```

PageChangementEtat.Designer.cs :

```
namespace AppCasier
{
    partial class PageChangementEtat
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components =
null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources
should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
```

```
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not
modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.lbTag = new System.Windows.Forms.Label();
            this.cbTag = new System.Windows.Forms.ComboBox();
            this.lbTitre = new System.Windows.Forms.Label();
            this.btChange = new System.Windows.Forms.Button();
            this.lbEtat = new System.Windows.Forms.Label();
            this.SuspendLayout();
            //
            // lbTag
            //
            this.lbTag.AutoSize = true;
            this.lbTag.Location = new System.Drawing.Point(12,
65);
            this.lbTag.Name = "lbTag";
            this.lbTag.Size = new System.Drawing.Size(32, 13);
            this.lbTag.TabIndex = 11;
            this.lbTag.Text = "Tag :";
            //
            // cbTag
            //
            this.cbTag.FormattingEnabled = true;
            this.cbTag.Location = new System.Drawing.Point(62,
62);
            this.cbTag.Name = "cbTag";
            this.cbTag.Size = new System.Drawing.Size(172, 21);
            this.cbTag.TabIndex = 10;
            this.cbTag.SelectedIndexChanged += new System.
EventHandler(this.cbTag_SelectedIndexChanged);
            //
            // lbTitre
            //
            this.lbTitre.AutoSize = true;
            this.lbTitre.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbTitre.Location = new System.Drawing.Point
(12, 9);
            this.lbTitre.Name = "lbTitre";
```

```
            this.lbTitre.Size = new System.Drawing.Size(225, 13);
            this.lbTitre.TabIndex = 9;
            this.lbTitre.Text = "Veuillez selectionnez le tag à modifier:";
            //
            // btChange
            //
            this.btChange.Location = new System.Drawing.Point(301, 62);
            this.btChange.Name = "btChange";
            this.btChange.Size = new System.Drawing.Size(83, 23);
            this.btChange.TabIndex = 8;
            this.btChange.Text = "Changement";
            this.btChange.UseVisualStyleBackColor = true;
            this.btChange.Click += new System.EventHandler(this.btChange_Click);
            //
            // lbEtat
            //
            this.lbEtat.AutoSize = true;
            this.lbEtat.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbEtat.Location = new System.Drawing.Point(251, 65);
            this.lbEtat.Name = "lbEtat";
            this.lbEtat.Size = new System.Drawing.Size(29, 13);
            this.lbEtat.TabIndex = 12;
            this.lbEtat.Text = "etat";
            //
            // PageChangementEtat
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(407, 119);
            this.Controls.Add(this.lbEtat);
            this.Controls.Add(this.lbTag);
            this.Controls.Add(this.cbTag);
            this.Controls.Add(this.lbTitre);
            this.Controls.Add(this.btChange);
            this.Name = "PageChangementEtat";
            this.Text = "PageChangementEtat";
            this.ResumeLayout(false);
            this.PerformLayout();

        }
```

```

        #endregion

        private System.Windows.Forms.Label lbTag;
        private System.Windows.Forms.ComboBox cbTag;
        private System.Windows.Forms.Label lbTitre;
        private System.Windows.Forms.Button btChange;
        private System.Windows.Forms.Label lbEtat;
    }
}
```

Program.cs :

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{

    internal static class Program
    {

        /// <summary>
        /// Point d’entrée principal de l’application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false
);
            MainForm main = new MainForm();

            if (main.GetLogin() == "")
            {
                Application.Exit();
            }
            else
            {
                Application.Run(main);

            }
        }
    }
}
```

RegexPays.cs :

```
using System;
```

```
using System.Collections.Generic;

public enum PaysEuropeen
{
    France,
    Allemagne,
    Italie,
    Espagne,
    Portugal,
    Belgique,
    PaysBas,
    Suisse,
    Autriche,
    Suede,
    Norvege,
    Finlande,
    Danemark,
    RoyaumeUni,
    Irlande,
    Pologne,
    Tchequie,
    Slovaquie,
    Hongrie,
    Roumanie,
    Bulgarie,
    Slovenie,
    Croatie,
    Serbie,
    MacedoineNord,
    Montenegro,
    Albanie,
    Grece,
    Luxembourg,
    Estonie,
    Lettonie,
    Lituanie,
    Chypre,
    Malte
}

public class PlaqueInfo
{
    public string Regex { get; set; }
    public string Exemple { get; set; }

    public PlaqueInfo(string regex, string exemple)
    {
        Regex = regex;
        Exemple = exemple;
    }
}
```

```
public static class PlaquesEuropeennes
{
    public static readonly Dictionary<PaysEuropeen, PlaqueInfo> InfosPlaques = new Dictionary<PaysEuropeen, PlaqueInfo>()
    {
        { PaysEuropeen.France, new PlaqueInfo("^[A-Z]{2}-\\d{3}-[A-Z]{2}$", "AB-123-CD") },
        { PaysEuropeen.Allemagne, new PlaqueInfo("^[A-Z]{1,3} [A-Z]{1,2} \\d{1,4}$", "B AB 1234") },
        { PaysEuropeen.Italie, new PlaqueInfo("^[A-Z]{2} \\d{3} [A-Z]{2}$", "AA 123 BB") },
        { PaysEuropeen.Espagne, new PlaqueInfo("^\\d{4} [A-Z]{3}$", "1234 ABC") },
        { PaysEuropeen.Portugal, new PlaqueInfo("^\\d{2}-[A-Z]{2}-\\d{2}$", "12-AB-34") },
        { PaysEuropeen.Belgique, new PlaqueInfo("^\\d-[A-Z]{3}-\\d{3}$", "1-ABC-123") },
        { PaysEuropeen.PaysBas, new PlaqueInfo("^[A-Z0-9]{2}-[A-Z0-9]{2}-[A-Z0-9]{2}$", "AB-12-CD") },
        { PaysEuropeen.Suisse, new PlaqueInfo("^[A-Z]{2} \\d{1,6}$", "GE 123456") },
        { PaysEuropeen.Autriche, new PlaqueInfo("^[A-Z]{1,3} \\d{1,5} [A-Z]{1,2}$", "W 12345 AB") },
        { PaysEuropeen.Suede, new PlaqueInfo("^[A-Z]{3} \\d{3}$", "ABC 123") },
        { PaysEuropeen.Norvege, new PlaqueInfo("^[A-Z]{2} \\d{5}$", "AB 12345") },
        { PaysEuropeen.Finlande, new PlaqueInfo("^[A-Z]{3}-\\d{3}$", "ABC-123") },
        { PaysEuropeen.Danemark, new PlaqueInfo("^[A-Z]{2} \\d{2} \\d{3}$", "AB 12 345") },
        { PaysEuropeen.RoyaumeUni, new PlaqueInfo("^[A-Z]{2}\\d{2} [A-Z]{3}$", "AB12 CDE") },
        { PaysEuropeen.Irlande, new PlaqueInfo("^\\d{2,3}-[A-Z]{1,2}-\\d{1,5}$", "12-D-12345") },
        { PaysEuropeen.Pologne, new PlaqueInfo("^[A-Z]{2} \\d{4,5}$", "AB 12345") },
        { PaysEuropeen.Tchequie, new PlaqueInfo("^\\d[A-Z]\\d \\d{4}$", "1A2 3456") },
        { PaysEuropeen.Slovaquie, new PlaqueInfo("^[A-Z]{2} \\d{3}[A-Z]{2}$", "BA 123AB") },
        { PaysEuropeen.Hongrie, new PlaqueInfo("^[A-Z]{3}-\\d{3}$", "ABC-123") },
        { PaysEuropeen.Roumanie, new PlaqueInfo("^[A-Z]{1,2} \\d{2,3} [A-Z]{3}$", "B 123 ABC") },
        { PaysEuropeen.Bulgarie, new PlaqueInfo("^[A-Z]{1,2} \\d{4} [A-Z]{1,2}$", "CB 1234 AB") },
        { PaysEuropeen.Slovenie, new PlaqueInfo("^[A-Z]{2} [A-Z]{1,2}-\\d{3}$", "LJ AB-123") },
        { PaysEuropeen.Croatie, new PlaqueInfo("^[A-Z]{2} \\d
```

```
{3}-[A-Z]{1,2}$", "ZG 123-AA") },
        { PaysEuropeen.Serbie, new PlaqueInfo("^[A-Z]{2} \\d{3}-[A-Z]{2}$", "NS 123-AA") },
        { PaysEuropeen.MacedoineNord, new PlaqueInfo("^[A-Z]{2} \\d{4} [A-Z]{2}$", "SK 1234 AB") },
        { PaysEuropeen.Montenegro, new PlaqueInfo("^[A-Z]{2} [A-Z]{1,2}\\d{3}$", "PG AB123") },
        { PaysEuropeen.Albanie, new PlaqueInfo("^[A-Z]{2} \\d{3} [A-Z]{2}$", "AA 123 BB") },
        { PaysEuropeen.Grece, new PlaqueInfo("^[A-Z]{3} \\d{4}$", "ABC 1234") },
        { PaysEuropeen.Luxembourg, new PlaqueInfo("^\\d{2,6}$", "123456") },
        { PaysEuropeen.Estonie, new PlaqueInfo("^[0-9]{3}[A-Z]{3}$", "123ABC") },
        { PaysEuropeen.Lettonie, new PlaqueInfo("^[A-Z]{2}-\\d{4}$", "AB-1234") },
        { PaysEuropeen.Lituanie, new PlaqueInfo("^[A-Z]{3} \\d{3}$", "ABC 123") },
        { PaysEuropeen.Chypre, new PlaqueInfo("^[A-Z]{3}-\\d{3}$", "ABC-123") },
        { PaysEuropeen.Malte, new PlaqueInfo("^[A-Z]{3} \\d{3}$", "MLT 123") }
    };
}
```

lectureTag.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class lectureTag : Form
    {
        Lecteur m_lecteur;

        public lectureTag(Lecteur lecteur)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            m_lecteur = lecteur;

        }
```

```

        private void lectureTagValide(object sender, EventArgs e)
        {

            if (!m_lecteur.lireTag())
            {
                lbConfirmation.Visible = false; // Cacher le picto de tag
                lbRefus.Text = "Tag non valide"; // Afficher le message d'erreur
            }
            else
            {
                if (m_lecteur.GetTag() != "")
                {
                    lbTag.Text = m_lecteur.GetTag(); // Afficher le tag lu dans le TextBox
                    lbConfirmation.Visible = true; // Afficher le picto de tag
                }
            }

        }

    }
}
```

lectureTag.Designer.cs :

```
namespace AppCasier
{
    partial class lectureTag
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
```

```
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.lbTitre = new System.Windows.Forms.Label();
            this.lbTag = new System.Windows.Forms.Label();
            this.lbConfirmation = new System.Windows.Forms.Label();
            this.lbRefus = new System.Windows.Forms.Label();
            this.SuspendLayout();
            //
            // lbTitre
            //
            this.lbTitre.AutoSize = true;
            this.lbTitre.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbTitre.Location = new System.Drawing.Point(40, 9);
            this.lbTitre.Name = "lbTitre";
            this.lbTitre.Size = new System.Drawing.Size(143, 13);
            this.lbTitre.TabIndex = 1;
            this.lbTitre.Text = "Tag en cours de lecture";
            //
            // lbTag
            //
            this.lbTag.AutoSize = true;
            this.lbTag.Location = new System.Drawing.Point(70, 67);
            this.lbTag.Name = "lbTag";
            this.lbTag.Size = new System.Drawing.Size(70, 13);
            this.lbTag.TabIndex = 3;
            this.lbTag.Text = "                    ";
            //
            // lbConfirmation
            //
            this.lbConfirmation.AutoSize = true;
            this.lbConfirmation.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbConfirmation.Location = new System.Drawing.Point(70, 31);
            this.lbConfirmation.Name = "lbConfirmation";
```

```
            this.lbConfirmation.Size = new System.Drawing.Size(77, 13);
            this.lbConfirmation.TabIndex = 4;
            this.lbConfirmation.Text = "Tag trouvé :";
            this.lbConfirmation.Visible = false;
            //
            // lbRefus
            //
            this.lbRefus.AutoSize = true;
            this.lbRefus.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbRefus.Location = new System.Drawing.Point(67, 54);
            this.lbRefus.Name = "lbRefus";
            this.lbRefus.Size = new System.Drawing.Size(91, 13);
            this.lbRefus.TabIndex = 5;
            this.lbRefus.Text = "                     ";
            //
            // lectureTag
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(231, 102);
            this.Controls.Add(this.lbRefus);
            this.Controls.Add(this.lbConfirmation);
            this.Controls.Add(this.lbTag);
            this.Controls.Add(this.lbTitre);
            this.Name = "lectureTag";
            this.Text = "lectureTag";
            this.Load += new System.EventHandler(this.lectureTagValide);
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label lbTitre;
        private System.Windows.Forms.Label lbTag;
        private System.Windows.Forms.Label lbConfirmation;
        private System.Windows.Forms.Label lbRefus;
    }
}
```

logout.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class logout : Form
    {
        MainForm mainform;
        public logout(MainForm form)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
 // Empêcher le redimensionnement de la fenêtre
            mainform = form;
            mainform.FermetureMenu();
            InitializeComponent();
            affichageInfo();
        }
        private void affichageInfo()
        {
            lbLogin.Text = mainform.GetLogin();

            if (mainform.GetRole() == "admin")
            {
                lbRole.Text = "Administrateur";
            }
            else
            {
                lbRole.Text = "Utilisateur";
            }
        }

        private void btLogout_Click(object sender, EventArgs e)
        {
            Connexion connexion = new Connexion(mainform);
            connexion.ShowDialog();
            this.Close();
            mainform.affichageGeneral();
        }
    }
}
```

logout.Designer.cs :

```
namespace AppCasier
```

```
{
    partial class logout
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            System.ComponentModel.ComponentResourceManager resources = new System.ComponentModel.ComponentResourceManager(typeof(logout));
            this.pictureBox1 = new System.Windows.Forms.PictureBox();
            this.lbLogin = new System.Windows.Forms.Label();
            this.lbRole = new System.Windows.Forms.Label();
            this.btLogout = new System.Windows.Forms.Button();
            ((System.ComponentModel.ISupportInitialize)(this.pictureBox1)).BeginInit();
            this.SuspendLayout();
            //
            // pictureBox1
            //
            this.pictureBox1.Image = ((System.Drawing.Image)(resources.GetObject("pictureBox1.Image")));
            this.pictureBox1.Location = new System.Drawing.Point(12, 12);
            this.pictureBox1.Name = "pictureBox1";
```

```
            this.pictureBox1.Size = new System.Drawing.Size(75, 81);
            this.pictureBox1.SizeMode = System.Windows.Forms.PictureBoxSizeMode.StretchImage;
            this.pictureBox1.TabIndex = 0;
            this.pictureBox1.TabStop = false;
            //
            // lbLogin
            //
            this.lbLogin.AutoSize = true;
            this.lbLogin.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbLogin.ForeColor = System.Drawing.SystemColors.Control;
            this.lbLogin.Location = new System.Drawing.Point(103, 27);
            this.lbLogin.Name = "lbLogin";
            this.lbLogin.Size = new System.Drawing.Size(34, 13);
            this.lbLogin.TabIndex = 1;
            this.lbLogin.Text = "login";
            //
            // lbRole
            //
            this.lbRole.AutoSize = true;
            this.lbRole.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbRole.ForeColor = System.Drawing.Color.MidnightBlue;
            this.lbRole.Location = new System.Drawing.Point(103, 63);
            this.lbRole.Name = "lbRole";
            this.lbRole.Size = new System.Drawing.Size(28, 13);
            this.lbRole.TabIndex = 2;
            this.lbRole.Text = "role";
            //
            // btLogout
            //
            this.btLogout.Location = new System.Drawing.Point(230, 37);
            this.btLogout.Name = "btLogout";
            this.btLogout.Size = new System.Drawing.Size(108, 39);
            this.btLogout.TabIndex = 3;
            this.btLogout.Text = "Se déconnecter";
            this.btLogout.UseVisualStyleBackColor = true;
            this.btLogout.Click += new System.EventHandler(this.btLogout_Click);
            //
```

```
            // logout
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackColor = System.Drawing.Color.Peru;
            this.BackgroundImageLayout = System.Windows.Forms.ImageLayout.Stretch;
            this.ClientSize = new System.Drawing.Size(361, 121);
            this.Controls.Add(this.btLogout);
            this.Controls.Add(this.lbRole);
            this.Controls.Add(this.lbLogin);
            this.Controls.Add(this.pictureBox1);
            this.DoubleBuffered = true;
            this.Name = "logout";
            this.Text = "Information Personnelle";
            ((System.ComponentModel.ISupportInitialize)(this.pictureBox1)).EndInit();
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.PictureBox pictureBox1;
        private System.Windows.Forms.Label lbLogin;
        private System.Windows.Forms.Label lbRole;
        private System.Windows.Forms.Button btLogout;
    }
}
```

pageAffichageInterface.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class PageAffichageInterface : Form
    {
        ChiffrageXOR ch = new ChiffrageXOR("
```

```
ChiffrementXORApplication");
        DatabaseConnection db = new DatabaseConnection(); // Instance de la classe DatabaseConnection
        MainForm mainForm;
        public PageAffichageInterface(string[] detailsAffectation,MainForm form)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            AffichageInfo(detailsAffectation);
            mainForm = form;

        }

        private void btInfoAffectationFermer_Click(object sender, EventArgs e)
        {
            this.Close();
        }

        private void btInfoAffectationSupprimer_Click(object sender, EventArgs e)
        {
            // Recuperer les informations de l'affectation
            string casier = lbAffectationCasierINFO.Text;


            db.OpenConnection();
            db.supprimerAffectation(casier);
            this.Close();

            // Mettre à jour l'affichage
            mainForm.Maj_Affichage();
        }

        private void AffichageInfo(string[] detailsAffectation)
        {

            lbAffectationTagINFO.Text = detailsAffectation[0];
            lbAffectationNomINFO.Text = ch.Decrypt(detailsAffectation[1]);
            lbAffectationPrenomINFO.Text = ch.Decrypt(detailsAffectation[2]);
            lbAffectationCompagnieINFO.Text = ch.Decrypt(detailsAffectation[3]);
            lbAffectationPlaqueINFO.Text = ch.Decrypt(detailsAffectation[4]);
            lbAffectationCasierINFO.Text = detailsAffectation[5];
            lbAffectationDateDebINFO.Text = detailsAffectation
```

```
[6];
            lbAffectationDateFinINFO.Text = detailsAffectation[7];
            lbAffectationPaysINFO.Text = ch.Decrypt(detailsAffectation[8]);

        }

        private void btPerdu_Click(object sender, EventArgs e)
        {
            if (db.rendrePerdu(lbAffectationTagINFO.Text))
            {
                MessageBox.Show("Le tag a été marqué comme perdu.");
                this.Close();
            }
            else
            {
                MessageBox.Show("Erreur lors de la mise à jour du tag.");
            }
        }
    }
}
```

pageAffichageInterface.Designer.cs :

```
namespace AppCasier
{
    partial class PageAffichageInterface
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code
```

```

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.btInfoAffectationFermer = new System.Windows.Forms.Button();
            this.btInfoAffectationSupprimer = new System.Windows.Forms.Button();
            this.lbTitreInfoAffectation = new System.Windows.Forms.Label();
            this.lbAffectationPrenom = new System.Windows.Forms.Label();
            this.lbAffectationNom = new System.Windows.Forms.Label();
            this.lbAffectationCompagnie = new System.Windows.Forms.Label();
            this.lbAffectationPlaque = new System.Windows.Forms.Label();
            this.lbAffectationTag = new System.Windows.Forms.Label();
            this.lbAffectationDateDeb = new System.Windows.Forms.Label();
            this.lbAffectationDateFin = new System.Windows.Forms.Label();
            this.lbAffectationPrenomINFO = new System.Windows.Forms.Label();
            this.lbAffectationNomINFO = new System.Windows.Forms.Label();
            this.lbAffectationCompagnieINFO = new System.Windows.Forms.Label();
            this.lbAffectationPlaqueINFO = new System.Windows.Forms.Label();
            this.lbAffectationTagINFO = new System.Windows.Forms.Label();
            this.lbAffectationDateDebINFO = new System.Windows.Forms.Label();
            this.lbAffectationDateFinINFO = new System.Windows.Forms.Label();
            this.lbAffectationCasier = new System.Windows.Forms.Label();
            this.lbAffectationCasierINFO = new System.Windows.Forms.Label();
            this.btPerdu = new System.Windows.Forms.Button();
            this.lbAffectationPaysINFO = new System.Windows.Forms.Label();
            this.lbAffectationPays = new System.Windows.Forms.Label();
```

```
            this.SuspendLayout();
            //
            // btInfoAffectationFermer
            //
            this.btInfoAffectationFermer.Location = new System.Drawing.Point(22, 294);
            this.btInfoAffectationFermer.Name = "btInfoAffectationFermer";
            this.btInfoAffectationFermer.Size = new System.Drawing.Size(75, 23);
            this.btInfoAffectationFermer.TabIndex = 0;
            this.btInfoAffectationFermer.Text = "Fermer";
            this.btInfoAffectationFermer.UseVisualStyleBackColor = true;
            this.btInfoAffectationFermer.Click += new System.EventHandler(this.btInfoAffectationFermer_Click);
            //
            // btInfoAffectationSupprimer
            //
            this.btInfoAffectationSupprimer.Location = new System.Drawing.Point(217, 294);
            this.btInfoAffectationSupprimer.Name = "btInfoAffectationSupprimer";
            this.btInfoAffectationSupprimer.Size = new System.Drawing.Size(75, 23);
            this.btInfoAffectationSupprimer.TabIndex = 1;
            this.btInfoAffectationSupprimer.Text = "Supprimer";
            this.btInfoAffectationSupprimer.UseVisualStyleBackColor = true;
            this.btInfoAffectationSupprimer.Click += new System.EventHandler(this.btInfoAffectationSupprimer_Click);
            //
            // lbTitreInfoAffectation
            //
            this.lbTitreInfoAffectation.AutoSize = true;
            this.lbTitreInfoAffectation.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.lbTitreInfoAffectation.Location = new System.Drawing.Point(19, 29);
            this.lbTitreInfoAffectation.Name = "lbTitreInfoAffectation";
            this.lbTitreInfoAffectation.Size = new System.Drawing.Size(170, 13);
            this.lbTitreInfoAffectation.TabIndex = 2;
            this.lbTitreInfoAffectation.Text = "Information sur l\'affectation :";
            //
            // lbAffectationPrenom
            //
```

```
            this.lbAffectationPrenom.AutoSize = true;
            this.lbAffectationPrenom.Location = new System.Drawing.Point(22, 64);
            this.lbAffectationPrenom.Name = "lbAffectationPrenom";
            this.lbAffectationPrenom.Size = new System.Drawing.Size(49, 13);
            this.lbAffectationPrenom.TabIndex = 3;
            this.lbAffectationPrenom.Text = "Prénom :";
            //
            // lbAffectationNom
            //
            this.lbAffectationNom.AutoSize = true;
            this.lbAffectationNom.Location = new System.Drawing.Point(22, 88);
            this.lbAffectationNom.Name = "lbAffectationNom";
            this.lbAffectationNom.Size = new System.Drawing.Size(35, 13);
            this.lbAffectationNom.TabIndex = 4;
            this.lbAffectationNom.Text = "Nom :";
            //
            // lbAffectationCompagnie
            //
            this.lbAffectationCompagnie.AutoSize = true;
            this.lbAffectationCompagnie.Location = new System.Drawing.Point(22, 141);
            this.lbAffectationCompagnie.Name = "lbAffectationCompagnie";
            this.lbAffectationCompagnie.Size = new System.Drawing.Size(66, 13);
            this.lbAffectationCompagnie.TabIndex = 5;
            this.lbAffectationCompagnie.Text = "Compagnie :";
            //
            // lbAffectationPlaque
            //
            this.lbAffectationPlaque.AutoSize = true;
            this.lbAffectationPlaque.Location = new System.Drawing.Point(22, 165);
            this.lbAffectationPlaque.Name = "lbAffectationPlaque";
            this.lbAffectationPlaque.Size = new System.Drawing.Size(46, 13);
            this.lbAffectationPlaque.TabIndex = 6;
            this.lbAffectationPlaque.Text = "Plaque :";
            //
            // lbAffectationTag
            //
            this.lbAffectationTag.AutoSize = true;
            this.lbAffectationTag.Location = new System.Drawing.Point(22, 190);
            this.lbAffectationTag.Name = "lbAffectationTag";
```

```
            this.lbAffectationTag.Size = new System.Drawing.Size(32, 13);
            this.lbAffectationTag.TabIndex = 7;
            this.lbAffectationTag.Text = "Tag :";
            //
            // lbAffectationDateDeb
            //
            this.lbAffectationDateDeb.AutoSize = true;
            this.lbAffectationDateDeb.Location = new System.Drawing.Point(22, 242);
            this.lbAffectationDateDeb.Name = "lbAffectationDateDeb";
            this.lbAffectationDateDeb.Size = new System.Drawing.Size(81, 13);
            this.lbAffectationDateDeb.TabIndex = 8;
            this.lbAffectationDateDeb.Text = "Date de début :";
            //
            // lbAffectationDateFin
            //
            this.lbAffectationDateFin.AutoSize = true;
            this.lbAffectationDateFin.Location = new System.Drawing.Point(22, 265);
            this.lbAffectationDateFin.Name = "lbAffectationDateFin";
            this.lbAffectationDateFin.Size = new System.Drawing.Size(65, 13);
            this.lbAffectationDateFin.TabIndex = 9;
            this.lbAffectationDateFin.Text = "Date de fin :";
            //
            // lbAffectationPrenomINFO
            //
            this.lbAffectationPrenomINFO.AutoSize = true;
            this.lbAffectationPrenomINFO.Location = new System.Drawing.Point(127, 64);
            this.lbAffectationPrenomINFO.Name = "lbAffectationPrenomINFO";
            this.lbAffectationPrenomINFO.Size = new System.Drawing.Size(127, 13);
            this.lbAffectationPrenomINFO.TabIndex = 10;
            this.lbAffectationPrenomINFO.Text = "lbAffectationPrenomINFO";
            //
            // lbAffectationNomINFO
            //
            this.lbAffectationNomINFO.AutoSize = true;
            this.lbAffectationNomINFO.Location = new System.Drawing.Point(127, 88);
            this.lbAffectationNomINFO.Name = "lbAffectationNomINFO";
            this.lbAffectationNomINFO.Size = new System.Drawing.Size(113, 13);
```

```
            this.lbAffectationNomINFO.TabIndex = 11;
            this.lbAffectationNomINFO.Text = "
lbAffectationNomINFO";
            //
            // lbAffectationCompagnieINFO
            //
            this.lbAffectationCompagnieINFO.AutoSize = true;
            this.lbAffectationCompagnieINFO.Location = new
System.Drawing.Point(127, 141);
            this.lbAffectationCompagnieINFO.Name = "
lbAffectationCompagnieINFO";
            this.lbAffectationCompagnieINFO.Size = new System.
Drawing.Size(144, 13);
            this.lbAffectationCompagnieINFO.TabIndex = 12;
            this.lbAffectationCompagnieINFO.Text = "
lbAffectationCompagnieINFO";
            //
            // lbAffectationPlaqueINFO
            //
            this.lbAffectationPlaqueINFO.AutoSize = true;
            this.lbAffectationPlaqueINFO.Location = new System.
Drawing.Point(127, 165);
            this.lbAffectationPlaqueINFO.Name = "
lbAffectationPlaqueINFO";
            this.lbAffectationPlaqueINFO.Size = new System.
Drawing.Size(124, 13);
            this.lbAffectationPlaqueINFO.TabIndex = 13;
            this.lbAffectationPlaqueINFO.Text = "
lbAffectationPlaqueINFO";
            //
            // lbAffectationTagINFO
            //
            this.lbAffectationTagINFO.AutoSize = true;
            this.lbAffectationTagINFO.Location = new System.
Drawing.Point(127, 190);
            this.lbAffectationTagINFO.Name = "
lbAffectationTagINFO";
            this.lbAffectationTagINFO.Size = new System.Drawing
.Size(110, 13);
            this.lbAffectationTagINFO.TabIndex = 14;
            this.lbAffectationTagINFO.Text = "
lbAffectationTagINFO";
            //
            // lbAffectationDateDebINFO
            //
            this.lbAffectationDateDebINFO.AutoSize = true;
            this.lbAffectationDateDebINFO.Location = new System
.Drawing.Point(127, 242);
            this.lbAffectationDateDebINFO.Name = "
lbAffectationDateDebINFO";
            this.lbAffectationDateDebINFO.Size = new System.
```

```
Drawing.Size(134, 13);
            this.lbAffectationDateDebINFO.TabIndex = 15;
            this.lbAffectationDateDebINFO.Text = "
lbAffectationDateDebINFO";
            //
            // lbAffectationDateFinINFO
            //
            this.lbAffectationDateFinINFO.AutoSize = true;
            this.lbAffectationDateFinINFO.Location = new System
.Drawing.Point(127, 265);
            this.lbAffectationDateFinINFO.Name = "
lbAffectationDateFinINFO";
            this.lbAffectationDateFinINFO.Size = new System.
Drawing.Size(128, 13);
            this.lbAffectationDateFinINFO.TabIndex = 16;
            this.lbAffectationDateFinINFO.Text = "
lbAffectationDateFinINFO";
            //
            // lbAffectationCasier
            //
            this.lbAffectationCasier.AutoSize = true;
            this.lbAffectationCasier.Location = new System.
Drawing.Point(22, 213);
            this.lbAffectationCasier.Name = "
lbAffectationCasier";
            this.lbAffectationCasier.Size = new System.Drawing.
Size(42, 13);
            this.lbAffectationCasier.TabIndex = 17;
            this.lbAffectationCasier.Text = "Casier :";
            //
            // lbAffectationCasierINFO
            //
            this.lbAffectationCasierINFO.AutoSize = true;
            this.lbAffectationCasierINFO.Location = new System.
Drawing.Point(127, 213);
            this.lbAffectationCasierINFO.Name = "
lbAffectationCasierINFO";
            this.lbAffectationCasierINFO.Size = new System.
Drawing.Size(120, 13);
            this.lbAffectationCasierINFO.TabIndex = 18;
            this.lbAffectationCasierINFO.Text = "
lbAffectationCasierINFO";
            //
            // btPerdu
            //
            this.btPerdu.Location = new System.Drawing.Point
(114, 294);
            this.btPerdu.Name = "btPerdu";
            this.btPerdu.Size = new System.Drawing.Size(86, 23)
;
            this.btPerdu.TabIndex = 19;
```

```
            this.btPerdu.Text = "Rendre perdu";
            this.btPerdu.UseVisualStyleBackColor = true;
            this.btPerdu.Click += new System.EventHandler(this.btPerdu_Click);
            //
            // lbAffectationPaysINFO
            //
            this.lbAffectationPaysINFO.AutoSize = true;
            this.lbAffectationPaysINFO.Location = new System.Drawing.Point(127, 113);
            this.lbAffectationPaysINFO.Name = "lbAffectationPaysINFO";
            this.lbAffectationPaysINFO.Size = new System.Drawing.Size(114, 13);
            this.lbAffectationPaysINFO.TabIndex = 21;
            this.lbAffectationPaysINFO.Text = "lbAffectationPaysINFO";
            //
            // lbAffectationPays
            //
            this.lbAffectationPays.AutoSize = true;
            this.lbAffectationPays.Location = new System.Drawing.Point(22, 113);
            this.lbAffectationPays.Name = "lbAffectationPays";
            this.lbAffectationPays.Size = new System.Drawing.Size(36, 13);
            this.lbAffectationPays.TabIndex = 20;
            this.lbAffectationPays.Text = "Pays :";
            //
            // PageAffichageInterface
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(320, 345);
            this.Controls.Add(this.lbAffectationPaysINFO);
            this.Controls.Add(this.lbAffectationPays);
            this.Controls.Add(this.btPerdu);
            this.Controls.Add(this.lbAffectationCasierINFO);
            this.Controls.Add(this.lbAffectationCasier);
            this.Controls.Add(this.lbAffectationDateFinINFO);
            this.Controls.Add(this.lbAffectationDateDebINFO);
            this.Controls.Add(this.lbAffectationTagINFO);
            this.Controls.Add(this.lbAffectationPlaqueINFO);
            this.Controls.Add(this.lbAffectationCompagnieINFO);
            this.Controls.Add(this.lbAffectationNomINFO);
            this.Controls.Add(this.lbAffectationPrenomINFO);
            this.Controls.Add(this.lbAffectationDateFin);
            this.Controls.Add(this.lbAffectationDateDeb);
```

```
            this.Controls.Add(this.lbAffectationTag);
            this.Controls.Add(this.lbAffectationPlaque);
            this.Controls.Add(this.lbAffectationCompagnie);
            this.Controls.Add(this.lbAffectationNom);
            this.Controls.Add(this.lbAffectationPrenom);
            this.Controls.Add(this.lbTitreInfoAffectation);
            this.Controls.Add(this.btInfoAffectationSupprimer);
            this.Controls.Add(this.btInfoAffectationFermer);
            this.Name = "PageAffichageInterface";
            this.Text = "Information";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Button btInfoAffectationFermer;
        private System.Windows.Forms.Button btInfoAffectationSupprimer;
        private System.Windows.Forms.Label lbTitreInfoAffectation;
        private System.Windows.Forms.Label lbAffectationPrenom;
        private System.Windows.Forms.Label lbAffectationNom;
        private System.Windows.Forms.Label lbAffectationCompagnie;
        private System.Windows.Forms.Label lbAffectationPlaque;
        private System.Windows.Forms.Label lbAffectationTag;
        private System.Windows.Forms.Label lbAffectationDateDeb;
        private System.Windows.Forms.Label lbAffectationDateFin;
        private System.Windows.Forms.Label lbAffectationPrenomINFO;
        private System.Windows.Forms.Label lbAffectationNomINFO;
        private System.Windows.Forms.Label lbAffectationCompagnieINFO;
        private System.Windows.Forms.Label lbAffectationPlaqueINFO;
        private System.Windows.Forms.Label lbAffectationTagINFO;
        private System.Windows.Forms.Label lbAffectationDateDebINFO;
        private System.Windows.Forms.Label lbAffectationDateFinINFO;
        private System.Windows.Forms.Label lbAffectationCasier;
        private System.Windows.Forms.Label lbAffectationCasierINFO;
        private System.Windows.Forms.Button btPerdu;
```

```
        private System.Windows.Forms.Label
lbAffectationPaysINFO;
        private System.Windows.Forms.Label lbAffectationPays;
    }
}
```

pageSupprTag.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class pageSupprTag : Form
    {
        DatabaseConnection db = new DatabaseConnection();
        MainForm mainform;
        public pageSupprTag(MainForm form)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
 // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            db.OpenConnection();
            mainform = form;
            affichageTag();
        }

        private void affichageTag()
        {

            string[] liste = db.recupNumTagNonAffecté();

            for (int i = 0; i < liste.Length; i++)
            {
                cbTag.Items.Add(liste[i]);
            }
        }

        private void btSuppr_Click(object sender, EventArgs e)
        {
            if (cbTag.Text == "")
            {
                MessageBox.Show("Veuillez sélectionner un tag
valide.");
                return;
```

```
            }
            else
            {
                if (db.supprimerTag(cbTag.Text))
                {
                    MessageBox.Show("Tag supprimé !");
                    this.Close();
                    mainform.Maj_Affichage();
                }
                else
                {
                    MessageBox.Show("Erreur lors de la suppression du tag !");
                }
            }

        }
    }
}
```

pageSupprTag.Designer.cs :

```
namespace AppCasier
{
    partial class pageSupprTag
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
```

```
        /// </summary>
        private void InitializeComponent()
        {
            this.lbTag = new System.Windows.Forms.Label();
            this.cbTag = new System.Windows.Forms.ComboBox();
            this.lbTitre = new System.Windows.Forms.Label();
            this.btSuppr = new System.Windows.Forms.Button();
            this.SuspendLayout();
            //
            // lbTag
            //
            this.lbTag.AutoSize = true;
            this.lbTag.Location = new System.Drawing.Point(12, 65);
            this.lbTag.Name = "lbTag";
            this.lbTag.Size = new System.Drawing.Size(32, 13);
            this.lbTag.TabIndex = 7;
            this.lbTag.Text = "Tag :";
            //
            // cbTag
            //
            this.cbTag.FormattingEnabled = true;
            this.cbTag.Location = new System.Drawing.Point(62, 62);
            this.cbTag.Name = "cbTag";
            this.cbTag.Size = new System.Drawing.Size(172, 21);
            this.cbTag.TabIndex = 6;
            //
            // lbTitre
            //
            this.lbTitre.AutoSize = true;
            this.lbTitre.Location = new System.Drawing.Point(12, 9);
            this.lbTitre.Name = "lbTitre";
            this.lbTitre.Size = new System.Drawing.Size(194, 13);
            this.lbTitre.TabIndex = 5;
            this.lbTitre.Text = "Veuillez selectionnez le tag à supprimer:";
            //
            // btSuppr
            //
            this.btSuppr.Location = new System.Drawing.Point(261, 62);
            this.btSuppr.Name = "btSuppr";
            this.btSuppr.Size = new System.Drawing.Size(75, 23);
            this.btSuppr.TabIndex = 4;
            this.btSuppr.Text = "Supprimer";
            this.btSuppr.UseVisualStyleBackColor = true;
            this.btSuppr.Click += new System.EventHandler(this.
```

```
btSuppr_Click);
            //
            // pageSupprTag
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF
(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.
AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(383, 155)
;
            this.Controls.Add(this.lbTag);
            this.Controls.Add(this.cbTag);
            this.Controls.Add(this.lbTitre);
            this.Controls.Add(this.btSuppr);
            this.Name = "pageSupprTag";
            this.Text = "pageSupprTag";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label lbTag;
        private System.Windows.Forms.ComboBox cbTag;
        private System.Windows.Forms.Label lbTitre;
        private System.Windows.Forms.Button btSuppr;
    }
}
```

pageSupprUtilisateur.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class pageSupprUtilisateur : Form
    {

        DatabaseConnection db = new DatabaseConnection();
        public pageSupprUtilisateur()
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
 // Empêcher le redimensionnement de la fenêtre
```

```
            InitializeComponent();
            db.OpenConnection();
            affichageUtiisateur();
        }
        private void affichageUtiisateur()
        {

            string[] liste = db.recupNomUtilisateur();

            for (int i = 0; i < liste.Length; i++)
            {
                cbUtilisateur.Items.Add(liste[i]);
            }
        }

        private void btSuppr_Click(object sender, EventArgs e)
        {
            string utilisateur = cbUtilisateur.Text;
            if (db.supprimerUtilisateur(utilisateur))
            {
                MessageBox.Show("Utilisateur supprimé !");
                this.Close();
            }
            else
            {
                MessageBox.Show("Erreur lors de la suppression de l'utilisateur !");
            }

        }
    }
}
```

pageSupprUtilisateur.Designer.cs :

```
namespace AppCasier
{
    partial class pageSupprUtilisateur
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
```

```
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.lbUtilisateur = new System.Windows.Forms.Label();
            this.cbUtilisateur = new System.Windows.Forms.ComboBox();
            this.lbTitre = new System.Windows.Forms.Label();
            this.btSuppr = new System.Windows.Forms.Button();
            this.SuspendLayout();
            //
            // lbUtilisateur
            //
            this.lbUtilisateur.AutoSize = true;
            this.lbUtilisateur.Location = new System.Drawing.Point(12, 65);
            this.lbUtilisateur.Name = "lbUtilisateur";
            this.lbUtilisateur.Size = new System.Drawing.Size(59, 13);
            this.lbUtilisateur.TabIndex = 11;
            this.lbUtilisateur.Text = "Utilisateur :";
            //
            // cbUtilisateur
            //
            this.cbUtilisateur.FormattingEnabled = true;
            this.cbUtilisateur.Location = new System.Drawing.Point(82, 62);
            this.cbUtilisateur.Name = "cbUtilisateur";
            this.cbUtilisateur.Size = new System.Drawing.Size(172, 21);
            this.cbUtilisateur.TabIndex = 10;
            //
            // lbTitre
            //
            this.lbTitre.AutoSize = true;
            this.lbTitre.Location = new System.Drawing.Point(12, 9);
            this.lbTitre.Name = "lbTitre";
```

```
            this.lbTitre.Size = new System.Drawing.Size(216, 13);
            this.lbTitre.TabIndex = 9;
            this.lbTitre.Text = "Veuillez selectionnez l\'utilisateur à supprimer:";
            //
            // btSuppr
            //
            this.btSuppr.Location = new System.Drawing.Point(278, 62);
            this.btSuppr.Name = "btSuppr";
            this.btSuppr.Size = new System.Drawing.Size(75, 23);
            this.btSuppr.TabIndex = 8;
            this.btSuppr.Text = "Supprimer";
            this.btSuppr.UseVisualStyleBackColor = true;
            this.btSuppr.Click += new System.EventHandler(this.btSuppr_Click);
            //
            // pageSupprUtilisateur
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(388, 157);
            this.Controls.Add(this.lbUtilisateur);
            this.Controls.Add(this.cbUtilisateur);
            this.Controls.Add(this.lbTitre);
            this.Controls.Add(this.btSuppr);
            this.Name = "pageSupprUtilisateur";
            this.Text = "pageSupprUtilisateur";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label lbUtilisateur;
        private System.Windows.Forms.ComboBox cbUtilisateur;
        private System.Windows.Forms.Label lbTitre;
        private System.Windows.Forms.Button btSuppr;
    }
}
```

pageSupprVisiteur.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
```

```
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class pageSupprVisiteur : Form
    {
        MainForm mainform;
        DatabaseConnection db = new DatabaseConnection(); //
Instance de la classe DatabaseConnection
        public pageSupprVisiteur(MainForm form)
        {
            this.FormBorderStyle = FormBorderStyle.FixedSingle;
 // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            db.OpenConnection();
            affichageVisiteur();
            mainform = form;
        }

        private void affichageVisiteur()
        {

            // Récupération des visiteurs
            string[] visiteurs = db.
recupNomPrenomVisiteurNonAffecté();
            // Ajout des visiteurs dans la liste

            if (visiteurs != null)
            {
                for (int i = 0; i < visiteurs.Length; i++)
                {
                    cbVisiteur.Items.Add(visiteurs[i]);
                }
            }

        }

        private void btSuppr_Click(object sender, EventArgs e)
        {
            // Recuperer le nom du visiteur et son prenom
            string[] nomPrenom = cbVisiteur.Text.Split(' ');
            string nom = nomPrenom[0];
            string prenom = nomPrenom[1];

            // Supprimer le visiteur
            if (db.supprimerVisiteur(nom, prenom))
```

```
                {
                    MessageBox.Show("Visiteur supprimé !");
                    this.Close();
                    mainform.Maj_Affichage();
                }
                else
                {
                    MessageBox.Show("Erreur lors de la suppression
du visiteur !");
                }

            }
        }
}
```

pageSupprVisiteur.Designer.cs :

```
namespace AppCasier
{
    partial class pageSupprVisiteur
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components =
null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources
should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not
modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.btSuppr = new System.Windows.Forms.Button();
            this.lbTitre = new System.Windows.Forms.Label();
```

```
            this.cbVisiteur = new System.Windows.Forms.ComboBox();
            this.lbVisiteur = new System.Windows.Forms.Label();
            this.SuspendLayout();
            //
            // btSuppr
            //
            this.btSuppr.Location = new System.Drawing.Point(278, 62);
            this.btSuppr.Name = "btSuppr";
            this.btSuppr.Size = new System.Drawing.Size(75, 23);
            this.btSuppr.TabIndex = 0;
            this.btSuppr.Text = "Supprimer";
            this.btSuppr.UseVisualStyleBackColor = true;
            this.btSuppr.Click += new System.EventHandler(this.btSuppr_Click);
            //
            // lbTitre
            //
            this.lbTitre.AutoSize = true;
            this.lbTitre.Location = new System.Drawing.Point(12, 9);
            this.lbTitre.Name = "lbTitre";
            this.lbTitre.Size = new System.Drawing.Size(211, 13);
            this.lbTitre.TabIndex = 1;
            this.lbTitre.Text = "Veuillez selectionnez le Visteur à supprimer:";
            //
            // cbVisiteur
            //
            this.cbVisiteur.FormattingEnabled = true;
            this.cbVisiteur.Location = new System.Drawing.Point(82, 62);
            this.cbVisiteur.Name = "cbVisiteur";
            this.cbVisiteur.Size = new System.Drawing.Size(172, 21);
            this.cbVisiteur.TabIndex = 2;
            //
            // lbVisiteur
            //
            this.lbVisiteur.AutoSize = true;
            this.lbVisiteur.Location = new System.Drawing.Point(12, 65);
            this.lbVisiteur.Name = "lbVisiteur";
            this.lbVisiteur.Size = new System.Drawing.Size(47, 13);
            this.lbVisiteur.TabIndex = 3;
            this.lbVisiteur.Text = "Visiteur :";
            //
```

```
            // pageSupprVisiteur
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(385, 156);
            this.Controls.Add(this.lbVisiteur);
            this.Controls.Add(this.cbVisiteur);
            this.Controls.Add(this.lbTitre);
            this.Controls.Add(this.btSuppr);
            this.Name = "pageSupprVisiteur";
            this.Text = "pageSupprVisiteur";
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Button btSuppr;
        private System.Windows.Forms.Label lbTitre;
        private System.Windows.Forms.ComboBox cbVisiteur;
        private System.Windows.Forms.Label lbVisiteur;
    }
}
```

verifDate.cs :

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace AppCasier
{
    public partial class verifDate : Form
    {
        ChiffrageXOR ch = new ChiffrageXOR("ChiffrementXORApplication");
        DatabaseConnection db = new DatabaseConnection(); // Instance de la classe DatabaseConnection
        MainForm mainForm;
        public verifDate(string[] detailsAffectation, MainForm form)
        {
```

```
            this.FormBorderStyle = FormBorderStyle.FixedSingle; // Empêcher le redimensionnement de la fenêtre
            InitializeComponent();
            AffichageInfo(detailsAffectation);
            mainForm = form;
            db.OpenConnection();
        }

        private void btInfoAffectationFermer_Click(object sender, EventArgs e)
        {
            this.Close();
        }

        private void btInfoAffectationSupprimer_Click(object sender, EventArgs e)
        {
            // Recuperer les informations de l'affectation
            string casier = lbAffectationCasierINFO.Text;



            db.supprimerAffectation(casier);
            this.Close();

            // Mettre à jour l'affichage
            mainForm.Maj_Affichage();
        }

        private void AffichageInfo(string[] detailsAffectation)
        {
            lbAffectationTagINFO.Text = detailsAffectation[0];
            lbAffectationNomINFO.Text = ch.Decrypt(detailsAffectation[1]);
            lbAffectationPrenomINFO.Text = ch.Decrypt(detailsAffectation[2]);
            lbAffectationCompagnieINFO.Text = ch.Decrypt(detailsAffectation[3]);
            lbAffectationPlaqueINFO.Text = ch.Decrypt(detailsAffectation[4]);
            lbAffectationCasierINFO.Text = detailsAffectation[5];
            lbAffectationDateDebINFO.Text = detailsAffectation[6];
            lbAffectationDateFinINFO.Text = detailsAffectation[7];
            lbAffectationPaysINFO.Text = ch.Decrypt(detailsAffectation[8]);

        }

```

```
        private void btPerdu_Click(object sender, EventArgs e)
        {
            if(db.rendrePerdu(lbAffectationTagINFO.Text))
            {
                MessageBox.Show("Le tag a été marqué comme
perdu.");
                this.Close();
            }
            else
            {
                MessageBox.Show("Erreur lors de la mise à jour
du tag.");
            }
        }
    }
}
```

verifDate.Designer.cs :

```
namespace AppCasier
{
    partial class verifDate
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components =
null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources
should be disposed; otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not
modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
```

```
            this.label1 = new System.Windows.Forms.Label();
            this.lbAffectationCasierINFO = new System.Windows.
Forms.Label();
            this.lbAffectationCasier = new System.Windows.Forms
.Label();
            this.lbAffectationDateFinINFO = new System.Windows.
Forms.Label();
            this.lbAffectationDateDebINFO = new System.Windows.
Forms.Label();
            this.lbAffectationTagINFO = new System.Windows.
Forms.Label();
            this.lbAffectationPlaqueINFO = new System.Windows.
Forms.Label();
            this.lbAffectationCompagnieINFO = new System.
Windows.Forms.Label();
            this.lbAffectationNomINFO = new System.Windows.
Forms.Label();
            this.lbAffectationPrenomINFO = new System.Windows.
Forms.Label();
            this.lbAffectationDateFin = new System.Windows.
Forms.Label();
            this.lbAffectationDateDeb = new System.Windows.
Forms.Label();
            this.lbAffectationTag = new System.Windows.Forms.
Label();
            this.lbAffectationPlaque = new System.Windows.Forms
.Label();
            this.lbAffectationCompagnie = new System.Windows.
Forms.Label();
            this.lbAffectationNom = new System.Windows.Forms.
Label();
            this.lbAffectationPrenom = new System.Windows.Forms
.Label();
            this.btInfoAffectationSupprimer = new System.
Windows.Forms.Button();
            this.btInfoAffectationFermer = new System.Windows.
Forms.Button();
            this.btPerdu = new System.Windows.Forms.Button();
            this.lbAffectationPaysINFO = new System.Windows.
Forms.Label();
            this.lbAffectationPays = new System.Windows.Forms.
Label();
            this.SuspendLayout();
            //
            // label1
            //
            this.label1.AutoSize = true;
            this.label1.Font = new System.Drawing.Font("
Microsoft Sans Serif", 8.25F, System.Drawing.FontStyle.Bold,
System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.label1.Location = new System.Drawing.Point(12,
```

```
9);
            this.label1.Name = "label1";
            this.label1.Size = new System.Drawing.Size(244, 13);
            this.label1.TabIndex = 0;
            this.label1.Text = "Cette affectattion a atteint sa date limite :";
            //
            // lbAffectationCasierINFO
            //
            this.lbAffectationCasierINFO.AutoSize = true;
            this.lbAffectationCasierINFO.Location = new System.Drawing.Point(133, 196);
            this.lbAffectationCasierINFO.Name = "lbAffectationCasierINFO";
            this.lbAffectationCasierINFO.Size = new System.Drawing.Size(120, 13);
            this.lbAffectationCasierINFO.TabIndex = 36;
            this.lbAffectationCasierINFO.Text = "lbAffectationCasierINFO";
            //
            // lbAffectationCasier
            //
            this.lbAffectationCasier.AutoSize = true;
            this.lbAffectationCasier.Location = new System.Drawing.Point(28, 196);
            this.lbAffectationCasier.Name = "lbAffectationCasier";
            this.lbAffectationCasier.Size = new System.Drawing.Size(42, 13);
            this.lbAffectationCasier.TabIndex = 35;
            this.lbAffectationCasier.Text = "Casier :";
            //
            // lbAffectationDateFinINFO
            //
            this.lbAffectationDateFinINFO.AutoSize = true;
            this.lbAffectationDateFinINFO.Location = new System.Drawing.Point(133, 248);
            this.lbAffectationDateFinINFO.Name = "lbAffectationDateFinINFO";
            this.lbAffectationDateFinINFO.Size = new System.Drawing.Size(128, 13);
            this.lbAffectationDateFinINFO.TabIndex = 34;
            this.lbAffectationDateFinINFO.Text = "lbAffectationDateFinINFO";
            //
            // lbAffectationDateDebINFO
            //
            this.lbAffectationDateDebINFO.AutoSize = true;
            this.lbAffectationDateDebINFO.Location = new System.Drawing.Point(133, 225);
```

```
            this.lbAffectationDateDebINFO.Name = "
lbAffectationDateDebINFO";
            this.lbAffectationDateDebINFO.Size = new System.
Drawing.Size(134, 13);
            this.lbAffectationDateDebINFO.TabIndex = 33;
            this.lbAffectationDateDebINFO.Text = "
lbAffectationDateDebINFO";
            //
            // lbAffectationTagINFO
            //
            this.lbAffectationTagINFO.AutoSize = true;
            this.lbAffectationTagINFO.Location = new System.
Drawing.Point(133, 173);
            this.lbAffectationTagINFO.Name = "
lbAffectationTagINFO";
            this.lbAffectationTagINFO.Size = new System.Drawing
.Size(110, 13);
            this.lbAffectationTagINFO.TabIndex = 32;
            this.lbAffectationTagINFO.Text = "
lbAffectationTagINFO";
            //
            // lbAffectationPlaqueINFO
            //
            this.lbAffectationPlaqueINFO.AutoSize = true;
            this.lbAffectationPlaqueINFO.Location = new System.
Drawing.Point(133, 148);
            this.lbAffectationPlaqueINFO.Name = "
lbAffectationPlaqueINFO";
            this.lbAffectationPlaqueINFO.Size = new System.
Drawing.Size(124, 13);
            this.lbAffectationPlaqueINFO.TabIndex = 31;
            this.lbAffectationPlaqueINFO.Text = "
lbAffectationPlaqueINFO";
            //
            // lbAffectationCompagnieINFO
            //
            this.lbAffectationCompagnieINFO.AutoSize = true;
            this.lbAffectationCompagnieINFO.Location = new
System.Drawing.Point(133, 124);
            this.lbAffectationCompagnieINFO.Name = "
lbAffectationCompagnieINFO";
            this.lbAffectationCompagnieINFO.Size = new System.
Drawing.Size(144, 13);
            this.lbAffectationCompagnieINFO.TabIndex = 30;
            this.lbAffectationCompagnieINFO.Text = "
lbAffectationCompagnieINFO";
            //
            // lbAffectationNomINFO
            //
            this.lbAffectationNomINFO.AutoSize = true;
            this.lbAffectationNomINFO.Location = new System.
```

```
Drawing.Point(133, 73);
            this.lbAffectationNomINFO.Name = "
lbAffectationNomINFO";
            this.lbAffectationNomINFO.Size = new System.Drawing
.Size(113, 13);
            this.lbAffectationNomINFO.TabIndex = 29;
            this.lbAffectationNomINFO.Text = "
lbAffectationNomINFO";
            //
            // lbAffectationPrenomINFO
            //
            this.lbAffectationPrenomINFO.AutoSize = true;
            this.lbAffectationPrenomINFO.Location = new System.
Drawing.Point(133, 49);
            this.lbAffectationPrenomINFO.Name = "
lbAffectationPrenomINFO";
            this.lbAffectationPrenomINFO.Size = new System.
Drawing.Size(127, 13);
            this.lbAffectationPrenomINFO.TabIndex = 28;
            this.lbAffectationPrenomINFO.Text = "
lbAffectationPrenomINFO";
            //
            // lbAffectationDateFin
            //
            this.lbAffectationDateFin.AutoSize = true;
            this.lbAffectationDateFin.Location = new System.
Drawing.Point(28, 248);
            this.lbAffectationDateFin.Name = "
lbAffectationDateFin";
            this.lbAffectationDateFin.Size = new System.Drawing
.Size(65, 13);
            this.lbAffectationDateFin.TabIndex = 27;
            this.lbAffectationDateFin.Text = "Date de fin :";
            //
            // lbAffectationDateDeb
            //
            this.lbAffectationDateDeb.AutoSize = true;
            this.lbAffectationDateDeb.Location = new System.
Drawing.Point(28, 225);
            this.lbAffectationDateDeb.Name = "
lbAffectationDateDeb";
            this.lbAffectationDateDeb.Size = new System.Drawing
.Size(81, 13);
            this.lbAffectationDateDeb.TabIndex = 26;
            this.lbAffectationDateDeb.Text = "Date de début :";
            //
            // lbAffectationTag
            //
            this.lbAffectationTag.AutoSize = true;
            this.lbAffectationTag.Location = new System.Drawing
.Point(28, 173);
```

```
            this.lbAffectationTag.Name = "lbAffectationTag";
            this.lbAffectationTag.Size = new System.Drawing.
Size(32, 13);
            this.lbAffectationTag.TabIndex = 25;
            this.lbAffectationTag.Text = "Tag :";
            //
            // lbAffectationPlaque
            //
            this.lbAffectationPlaque.AutoSize = true;
            this.lbAffectationPlaque.Location = new System.
Drawing.Point(28, 148);
            this.lbAffectationPlaque.Name = "
lbAffectationPlaque";
            this.lbAffectationPlaque.Size = new System.Drawing.
Size(46, 13);
            this.lbAffectationPlaque.TabIndex = 24;
            this.lbAffectationPlaque.Text = "Plaque :";
            //
            // lbAffectationCompagnie
            //
            this.lbAffectationCompagnie.AutoSize = true;
            this.lbAffectationCompagnie.Location = new System.
Drawing.Point(28, 124);
            this.lbAffectationCompagnie.Name = "
lbAffectationCompagnie";
            this.lbAffectationCompagnie.Size = new System.
Drawing.Size(66, 13);
            this.lbAffectationCompagnie.TabIndex = 23;
            this.lbAffectationCompagnie.Text = "Compagnie :";
            //
            // lbAffectationNom
            //
            this.lbAffectationNom.AutoSize = true;
            this.lbAffectationNom.Location = new System.Drawing
.Point(28, 73);
            this.lbAffectationNom.Name = "lbAffectationNom";
            this.lbAffectationNom.Size = new System.Drawing.
Size(35, 13);
            this.lbAffectationNom.TabIndex = 22;
            this.lbAffectationNom.Text = "Nom :";
            //
            // lbAffectationPrenom
            //
            this.lbAffectationPrenom.AutoSize = true;
            this.lbAffectationPrenom.Location = new System.
Drawing.Point(28, 49);
            this.lbAffectationPrenom.Name = "
lbAffectationPrenom";
            this.lbAffectationPrenom.Size = new System.Drawing.
Size(49, 13);
            this.lbAffectationPrenom.TabIndex = 21;
```

```
            this.lbAffectationPrenom.Text = "Prénom :";
            //
            // btInfoAffectationSupprimer
            //
            this.btInfoAffectationSupprimer.Location = new
System.Drawing.Point(224, 281);
            this.btInfoAffectationSupprimer.Name = "
btInfoAffectationSupprimer";
            this.btInfoAffectationSupprimer.Size = new System.
Drawing.Size(75, 23);
            this.btInfoAffectationSupprimer.TabIndex = 20;
            this.btInfoAffectationSupprimer.Text = "Supprimer";
            this.btInfoAffectationSupprimer.
UseVisualStyleBackColor = true;
            this.btInfoAffectationSupprimer.Click += new System
.EventHandler(this.btInfoAffectationSupprimer_Click);
            //
            // btInfoAffectationFermer
            //
            this.btInfoAffectationFermer.Location = new System.
Drawing.Point(27, 281);
            this.btInfoAffectationFermer.Name = "
btInfoAffectationFermer";
            this.btInfoAffectationFermer.Size = new System.
Drawing.Size(75, 23);
            this.btInfoAffectationFermer.TabIndex = 19;
            this.btInfoAffectationFermer.Text = "Fermer";
            this.btInfoAffectationFermer.
UseVisualStyleBackColor = true;
            this.btInfoAffectationFermer.Click += new System.
EventHandler(this.btInfoAffectationFermer_Click);
            //
            // btPerdu
            //
            this.btPerdu.Location = new System.Drawing.Point
(119, 281);
            this.btPerdu.Name = "btPerdu";
            this.btPerdu.Size = new System.Drawing.Size(88, 23)
;
            this.btPerdu.TabIndex = 37;
            this.btPerdu.Text = "Rendre perdu";
            this.btPerdu.UseVisualStyleBackColor = true;
            this.btPerdu.Click += new System.EventHandler(this.
btPerdu_Click);
            //
            // lbAffectationPaysINFO
            //
            this.lbAffectationPaysINFO.AutoSize = true;
            this.lbAffectationPaysINFO.Location = new System.
Drawing.Point(133, 100);
            this.lbAffectationPaysINFO.Name = "
```

```
lbAffectationPaysINFO";
            this.lbAffectationPaysINFO.Size = new System.
Drawing.Size(114, 13);
            this.lbAffectationPaysINFO.TabIndex = 39;
            this.lbAffectationPaysINFO.Text = "
lbAffectationPaysINFO";
            //
            // lbAffectationPays
            //
            this.lbAffectationPays.AutoSize = true;
            this.lbAffectationPays.Location = new System.
Drawing.Point(28, 100);
            this.lbAffectationPays.Name = "lbAffectationPays";
            this.lbAffectationPays.Size = new System.Drawing.
Size(36, 13);
            this.lbAffectationPays.TabIndex = 38;
            this.lbAffectationPays.Text = "Pays :";
            //
            // verifDate
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF
(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.
AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(328, 325)
;
            this.Controls.Add(this.lbAffectationPaysINFO);
            this.Controls.Add(this.lbAffectationPays);
            this.Controls.Add(this.btPerdu);
            this.Controls.Add(this.lbAffectationCasierINFO);
            this.Controls.Add(this.lbAffectationCasier);
            this.Controls.Add(this.lbAffectationDateFinINFO);
            this.Controls.Add(this.lbAffectationDateDebINFO);
            this.Controls.Add(this.lbAffectationTagINFO);
            this.Controls.Add(this.lbAffectationPlaqueINFO);
            this.Controls.Add(this.lbAffectationCompagnieINFO);
            this.Controls.Add(this.lbAffectationNomINFO);
            this.Controls.Add(this.lbAffectationPrenomINFO);
            this.Controls.Add(this.lbAffectationDateFin);
            this.Controls.Add(this.lbAffectationDateDeb);
            this.Controls.Add(this.lbAffectationTag);
            this.Controls.Add(this.lbAffectationPlaque);
            this.Controls.Add(this.lbAffectationCompagnie);
            this.Controls.Add(this.lbAffectationNom);
            this.Controls.Add(this.lbAffectationPrenom);
            this.Controls.Add(this.btInfoAffectationSupprimer);
            this.Controls.Add(this.btInfoAffectationFermer);
            this.Controls.Add(this.label1);
            this.Name = "verifDate";
            this.Text = "verifDate";
            this.ResumeLayout(false);
```

```
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.Label label1;
        private System.Windows.Forms.Label lbAffectationCasierINFO;
        private System.Windows.Forms.Label lbAffectationCasier;
        private System.Windows.Forms.Label lbAffectationDateFinINFO;
        private System.Windows.Forms.Label lbAffectationDateDebINFO;
        private System.Windows.Forms.Label lbAffectationTagINFO;
        private System.Windows.Forms.Label lbAffectationPlaqueINFO;
        private System.Windows.Forms.Label lbAffectationCompagnieINFO;
        private System.Windows.Forms.Label lbAffectationNomINFO;
        private System.Windows.Forms.Label lbAffectationPrenomINFO;
        private System.Windows.Forms.Label lbAffectationDateFin;
        private System.Windows.Forms.Label lbAffectationDateDeb;
        private System.Windows.Forms.Label lbAffectationTag;
        private System.Windows.Forms.Label lbAffectationPlaque;
        private System.Windows.Forms.Label lbAffectationCompagnie;
        private System.Windows.Forms.Label lbAffectationNom;
        private System.Windows.Forms.Label lbAffectationPrenom;
        private System.Windows.Forms.Button btInfoAffectationSupprimer;
        private System.Windows.Forms.Button btInfoAffectationFermer;
        private System.Windows.Forms.Button btPerdu;
        private System.Windows.Forms.Label lbAffectationPaysINFO;
        private System.Windows.Forms.Label lbAffectationPays;
    }
}
```